Deep Learning

Elisa Ricci

a.y. 2025/2026

University of Trento

Davide Donà

July 2026

Notes Repository

Contents

1 Machine Learning Recap

In this section we will briefly introduce the definition of **Machine Learning** and **Deep Learning**, introducing the main types of learning that we will encounter in these notes.

1.1 Definitions

Artificial Intelligence (AI) is the discipline that aims to create machines that mimic human intelligence.

Machine Learning (ML) is a subset of AI that enables machines to learn from experience (i.e., data) to improve their performance on a specific task.

Deep Learning is in turn a subfield of ML that focuses on **Artificial Neural Networks** (ANNs); the term 'deep' refers to the use of multiple layers, which allows the network to learn complex representations of data with multiple levels of abstraction.

Machine Learning was born in contrast to the programming paradigm, which is based on the idea of writing explicit rules to solve a problem.

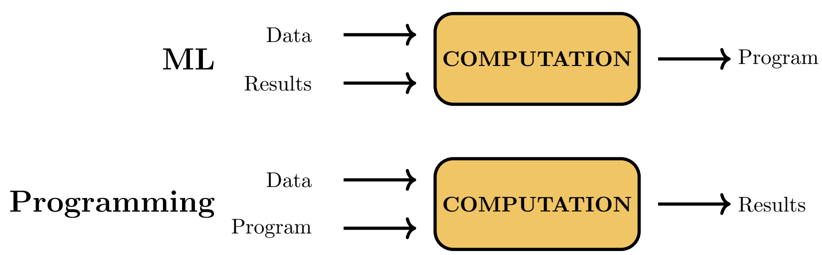


Figure 1: Programming paradigm vs Machine Learning paradigm.

In the Machine Learning paradigm, we are instead given the data, along with the results, and we try to learn the **model**. The learned model (e.g. splits in a decision tree, linear function for SVMs) represents the solution to the problem, and it can be used to make predictions on new data.

1.1.1 Why do we need Machine Learning

The main reason why we need Machine Learning can be summarized in the following points:

- **Complexity of the problem**: For many tasks, we have no human expertise to write explicit rules or we cannot write them down. The only plausible solution is to learn from data, which is often available in large quantities;

- **Personalized solutions**: For many tasks, we need to learn personalized solutions for each user, which is not feasible with the programming paradigm;
- **Huge amount of data**: Working with huge amount of data is not feasible, as it would require writing down explicit rules for each possible combination of inputs.

1.1.2 Machine Learning - Another definition

> **Machine Learning.** A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

This definition is more formal and precise than the previous one, and it highlights the importance of the three components: experience, task, and performance measure. Each machine learning algorithm can be characterized by these three components. For example, taking in consideration a spam email filter, we can define:

- T: Categorizing emails as spam or not spam;
- P: Accuracy of the classification. Other metrics may be more appropriate;
- E: Database of emails labeled as spam or not spam by human experts.

1.2 Types of learning

We can identify three main types of learning:

- **Supervised learning**: We are given a training set of input-output pairs, and we want to learn a function that maps inputs to outputs.
- **Unsupervised learning**: We are given a training set of inputs, and we want to learn a function that captures the underlying structure of the data.
- **Reinforcement learning**: We are given an environment, and we want to learn a policy that maximizes the cumulative reward.

1.2.1 Supervised Learning

Supervised learning can be further divided into subcategories. In particular, we are interested in the following:

- **Classification**: The output is a discrete label. More formally, given a training set $\mathcal{D} = \{(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_n, y_n)\}$ we can define it as:

$$f : \mathcal{X} \to \{1, \ldots, k\} \tag{1.1}$$

  where $\mathcal{X}$ is the input space, and $\{1, \ldots, k\}$ is the set of possible labels.

  Based on the number of classes, we can further divide classification into **binary classification** ($k = 2$) and **multi-class classification** ($k > 2$).

- **Regression**: The output is a continuous value. More formally, given a training set $\mathcal{D} = \{(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_n, y_n)\}$ we can define it as:

$$f : \mathcal{X} \to \mathbb{R} \tag{1.2}$$

- **Structured output**: **supervised learning** doesn't necessarily require a **vector** as input and a **scalar** as output. We can have more complex input and output structures, such as images, graphs, or sequences. In all cases the output is a structured object, with relationships between its components.

  Some practical example of **structured input regression** problems are:

  - **Semantic segmentation**: Given an image of input size $M \times N \times 3$, we want to predict a mask of size $M \times N$, where each pixel is labeled with a class.

    Before the advent of deep learning, the original solution to this kind of problem was to use **feature engineering** to extract features from neighborhoods of pixels. After that, the extracted features were passed to an SVM to predict the class of each pixel.

    Neural Networks show their real power in this kind of problems, as they are able to learn features directly from the data, without any human intervention.

  - **Machine translation**: given a sentence (sequence of words) in a source language, we want to translate it to a target language. This is even more complex than the previous example, as not only the output is related to its adjacent components, but also grammatical and semantic relationships between words need to be captured. We will see how **Recurrent Neural Networks** (RNNs) and **Transformers** are able to capture these relationships and produce state-of-the-art results in this kind of problems.

1.2.2 Unsupervised Learning

In unsupervised learning, we are given a training set of inputs, without any corresponding output labels, and we want to learn the underlying structure of the data. Some examples of unsupervised learning tasks are:

- **Clustering**: grouping similar data points together;
- **Deep clustering**: a technique that combines deep learning with classical clustering algorithms, such as k-means, to learn better representations of the data and improve clustering performance.

  It is based on the idea of using a deep neural network to learn a low-dimensional representation of the data. This is then used as input to a clustering algorithm, producing a set of **pseudo-labels** that are then used to train the neural network using classical backpropagation. This process is repeated iteratively until convergence;

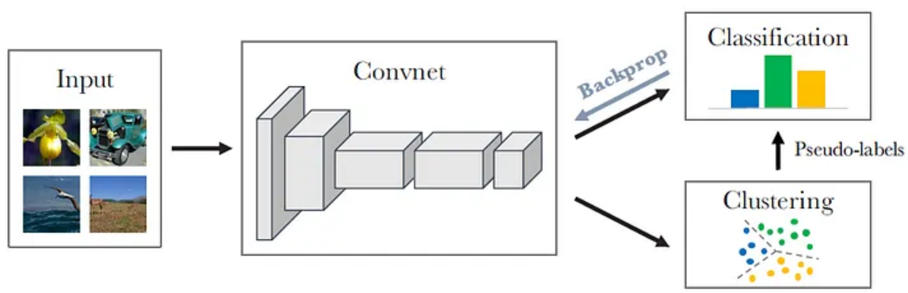


Figure 2: Deep clustering example

- **Generative Models**: models learn the underlying distribution of the data, and can be used to generate new data points, similar to the ones in the training set.

  Also in this case, we can have **deep generative models**, which try to map examples extracted from a simple distribution Z to a more complex distribution $g_\theta(Z)$, which is similar to the distribution of the training data X.

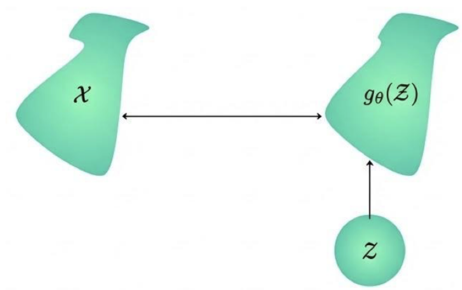


Figure 3: Deep generative models.

- **Dimensionality reduction**: try to map data to a lower-dimensional space in which visualization and interpretation are easier. This is often used as a preprocessing step for other machine learning algorithms.
- **Autoencoders**: a type of encoder-decoder architecture, where:
  - The **encoder** maps the input data to a **latent space** of lower dimensionality;
  - The **decoder** maps the latent representation back to the original input.

  The two components are trained together to minimize the reconstruction error, which is the difference between the original input and the reconstructed output. Once trained, the encoder can be used alone to obtain lower-dimensional representations of the data.

1.2.3 Reinforcement Learning

In Reinforcement Learning, an **agent** learn to interact with an **environment** in order to maximize a cumulative reward. We are not given any data, but rather the agent learns from its own experience, by exploring the environment and receiving feedback in the form of rewards.

In **deep reinforcement learning** the agent uses a deep neural network to learn a policy that maps states of the environment to actions. This allows the agent to learn complex behaviors and solve tasks that are difficult to solve with traditional reinforcement learning algorithms.

1.3 Designing a Learning System

To design a learning system, it's important to consider the following steps:

1. **Choose the training experience** E: This is the data that will be used to train the model. It should be representative of the problem we want to solve, and it should be of good quality;
2. **Choose the model**: This is the function that we want to learn from the data. It heavily depends on the task we want to solve,
3. **Choose how to represent the model**: the form of the model, what kind of function we allow. For example, we can have:
   - **Instance-based functions**: predictions are made based on the similarity between the input and the training examples. For example, k-nearest neighbors;
   - **Numerical functions**: predictions are made based on a mathematical function that maps inputs to outputs. For example, Neural Networks, SVMs;
   - **Probabilistic functions**: predictions are made based on a probability distribution over the possible outputs. For example, Naive Bayes;
   - **Symbolic functions**: predictions are made based on a set of rules that are learned from the data. For example, decision trees.
4. **Choose the learning algorithm**: This is the algorithm that will be used to learn the model from the data. For example, for SVMs we can decide between different optimization algorithms, such as stochastic gradient descent or quadratic optimization.

   The success of the machine learning system also depends on the search and optimization algorithm used to find the best model.
5. **Choose the performance measure** P: This is the metric that will be used to evaluate the performance of the model. Different tasks require different performance measures and it's not always a trivial choice. Ideally we would like to choose a performance measure that is closely related to the loss function used to train the model.

1.4 Assumptions in Machine Learning

When developing a machine learning model, we usually assume that we are given a training set of examples that are **observed** and we want to learn a model that can make predictions on new, unseen examples, the **test set**.

The challenge is that the algorithm has to learn a model that performs well on new examples. This ability is known as **generalization**.

1.4.1 Empirical risk minimization

Classical optimization algorithms are different from machine learning algorithms. In classical optimization, the only goal is to find the best solution to a problem, reducing the **training error** as much as possible. In machine learning instead, we want to minimize the **test error**, which is the error on new, unseen examples. This is a much harder problem, as we don't have direct access to the test set during training.

To tackle this problem, we assume that both training and test examples are drawn from the same distribution. This is known as the **independent and identically distributed** (i.i.d.) assumption, and it allows us to use the training set to learn a model that generalizes well to the test set.

If the training set is representative of the underlying unknown distribution, then the **empirical loss** (the average loss on the training set) is a good estimate of the **expected loss** (the average loss on the test set). This is the principle of **empirical risk minimization**.

1.4.2 Assumption violations

If the training set is not representative of the underlying distribution or if the i.i.d. assumption is violated, then the model may not generalize well to the test set, and we may end up with a model that performs well on the training set but poorly on the test set. This is known as **overfitting**.

1.4.3 Underfitting

If the model is too simple, it may perform poorly on both the training and test set. This is known as **underfitting**. To avoid underfitting, we need to choose a more complex model, or to use a more powerful learning algorithm.

1.4.4 Overfitting

If the model is too complex, it may perform well on the training set but poorly on the test set. When the gap between the two errors is large, it means that the model has learned to fit the noise in the training data, rather than the underlying distribution. This is known as **overfitting**.

To avoid overfitting, we can use techniques such as:

- **Simpler models**: choosing a simpler model with fewer parameters, which is less likely to overfit the training data;

- **Regularization**: any modification made to a learning algorithm that is intended to reduce the generalization error, but not the training error. Usually, this is done by adding a penalty term to the loss function, which is proportional to the complexity of the model (e.g., adding a term that penalizes large weights in a neural network).

1.5 How to choose the algorithm

To choose the right algorithm for a given problem, we need to consider the following factors:

- **Size of the training set**: Some algorithms perform better with small training sets, while others require large training sets to perform well;
- **Address problem difficulty**: are data linearly separable? Is the problem a regression or a classification problem? Is the output structured?
- **Prior knowledge**: if prior knowledge about the problem is available, it can be encoded in the model, which can improve performance;
- **Computational requirements**: do we have any requirements on the computational resources available for training and inference?
- **Online vs batch learning**: based on the kind of data we have, we may need to choose different algorithms. For example, for streaming data, Neural Network aren't the best choice;
- **Interpretability**: do we need to understand how the model makes predictions?

2 Feedforward Neural Networks

Before we can understand the architecture of a **feedforward neural network** (FFNN), we first need to introduce how we got here. In this chapter, we will cover the historical background of neural networks, the basic architecture of FFNNs, and the key components that make them work.

2.1 From Logistic Regression to Feedforward NN

To fit a model to some data distribution, a simple and common approach is to use **Maximum Likelihood Estimation** (MLE).

2.1.1 Maximum Likelihood Estimation

In MLE, we want to estimate the parameters of an assumed probability distribution, given some observed data. For example, if we assume that the data is generated from a Gaussian distribution, we can estimate the mean and variance of that distribution using MLE.

More formally, given a dataset $\boldsymbol{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\}$ and a known probability distribution $p_{\text{model}}(\boldsymbol{x}|\boldsymbol{\theta})$ we want to find the parameters $\boldsymbol{\theta}$ that better explain the data.

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} p_{\text{model}}(\boldsymbol{X}|\boldsymbol{\theta}) \tag{2.1}$$

Assuming that the data points are independent and identically distributed (i.i.d.), we can express the likelihood as a product of individual probabilities:

$$p_{\text{model}}(\boldsymbol{X}|\boldsymbol{\theta}) = \prod_{i=1}^{N} p_{\text{model}}(\boldsymbol{x}_i|\boldsymbol{\theta})$$

To simplify the optimization process, we often take the logarithm of the likelihood function, which transforms the product into a sum:

$$\log p_{\text{model}}(\boldsymbol{X}|\boldsymbol{\theta}) = \sum_{i=1}^{N} \log p_{\text{model}}(\boldsymbol{x}_i|\boldsymbol{\theta})$$

We can show that this is equivalent to considering the expected value of the log-likelihood over the empirical (observed) data distribution:

$$\log p_{\text{model}}(\boldsymbol{X}|\boldsymbol{\theta}) = N \cdot \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}[\log p_{\text{model}}(\boldsymbol{x}|\boldsymbol{\theta})]$$

where p_{data} is the empirical distribution of the observed data and $\mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}$ denotes the expected value sampled from this distribution.

To find the optimal parameters $\boldsymbol{\theta}^*$, we usually differentiate the log-likelihood with respect to $\boldsymbol{\theta}$ and set it to zero. We can then resolve the resulting equation to find the optimal parameters.

The equation 2.1 can also be extended to the case of conditional probability distributions ($p_{\text{model}}(y|\boldsymbol{x}, \boldsymbol{\theta})$) in order to predict a target variable y given an input variable $\boldsymbol{x}$:

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} p_{\text{model}}(y|\boldsymbol{x}, \boldsymbol{\theta})$$

2.1.2 Example: Logistic Regression

One of the simplest examples of MLE is **Logistic Regression**, which is a linear model used for binary classification tasks. In Logistic Regression, we assume that the conditional probability distribution of the target variable y given the input variable $\boldsymbol{x}$ follows a logistic function:

$$p_{\text{model}}(y = 1|\boldsymbol{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^T \boldsymbol{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \boldsymbol{x}}} \tag{2.2}$$

We can clearly see that the logistic function $\sigma(z)$ can be interpreted as a **neuron**:

- The input to the neuron is the linear combination of the input features $\boldsymbol{x}$ and the parameters $\boldsymbol{\theta}$.
- The sigmoid activation function $\sigma(z)$ is a non-linear function that maps the input to a probability value between 0 and 1.

2.2 From the Perceptron to Deep Networks

Before introducing the architecture of a feedforward neural network, it is useful to recall the building block from which it originated: the artificial neuron, also known as the **perceptron**.

2.2.1 Electronic Brain

In the first perceptron model, the inputs form a vector of **binary features** that are simply summed together and passed through a **threshold function**, without any **weights** involved.

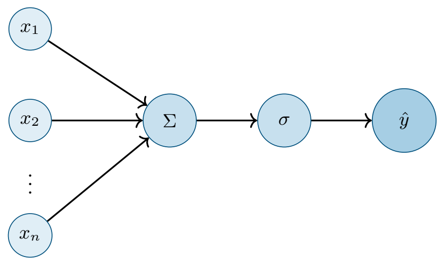


Figure 4: Architecture of the electronic brain model. Inputs x_i are summed together and passed through an activation function σ to produce the output $\hat{y}$.

It can be shown that this model is capable of classifying linearly separable data, such as the AND, OR, and NOT functions, but it fails to classify non-linearly separable data, such as the XOR function.

2.2.2 The Perceptron

The **Perceptron** is an extension of the electronic brain model. The key difference is the introduction of **weights** for each input, which allows the model to learn from data and adjust its parameters accordingly.

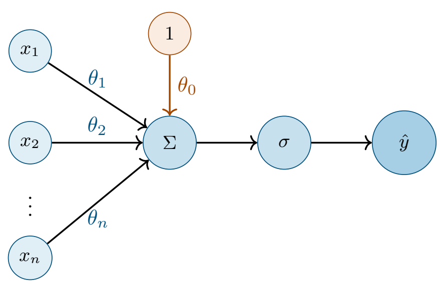


Figure 5: Architecture of a single layer perceptron. Inputs x_i are weighted by w_i, summed together, and passed through an activation function σ to produce the output y.

The prediction $\hat{y}$ of the perceptron is given by the following formula:

$$\hat{y} = \sigma\left(\sum_{i=1}^{n} \theta_i x_i + \theta_0\right) \tag{2.3}$$

where θ_i are the weights associated with each input x_i, and σ is the activation function. We can also express the prediction in vector form as follows:

$$\hat{y} = \sigma\left(\boldsymbol{\theta}^T \boldsymbol{x} + \theta_0\right) \tag{2.4}$$

The most important intuition is that the weights θ_i are not fixed, but rather learned from data.

The bias term is introduced to allow the model to shift the decision boundary (hyperplane) away from the origin.

> **Perceptron.** An artificial neuron, i.e., a perceptron is a non-linear parametrized function with a restricted output range.

Today, the activation function σ is typically a non-linear function, such as the sigmoid, ReLU, or tanh function, which allows the model to learn complex patterns in the data.

Perceptron Learning Algorithm The weights of the perceptron are learned from data using the **Perceptron Training Rule**. Given a set of training examples $\mathcal{T} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_N, y_N)\}$, $y_i \in \{0, 1\}$ and a learning rate η, the weights are updated as follows:

1. Initialize the weights θ_i to small random values;
2. Until convergence, repeat:
   (a) Select randomly a training example $(\boldsymbol{x}_i, y_i)$ from $\mathcal{T}$;
   (b) Compute the prediction $\hat{y}_i$ using the current weights:
   $$\hat{y}_i = \sigma\left(\boldsymbol{\theta}^T \boldsymbol{x}_i + \theta_0\right)$$
   (c) Update the weights based on the error between the prediction and the true label:
   $$\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)} + \eta(y_i - \hat{y}_i)\boldsymbol{x}_i$$
   (d) Update the bias:
   $$\theta_0^{(t+1)} \leftarrow \theta_0^{(t)} + \eta(y_i - \hat{y}_i)$$

Note that we are assuming that the training data is linearly separable, otherwise we would have to introduce a stopping criterion to prevent the algorithm from running indefinitely.

2.2.3 MLP

A single perceptron cannot learn non-linearly separable data, such as the XOR function. This limitation motivated the **Multilayer Perceptron** (MLP), which stacks several layers of neurons.

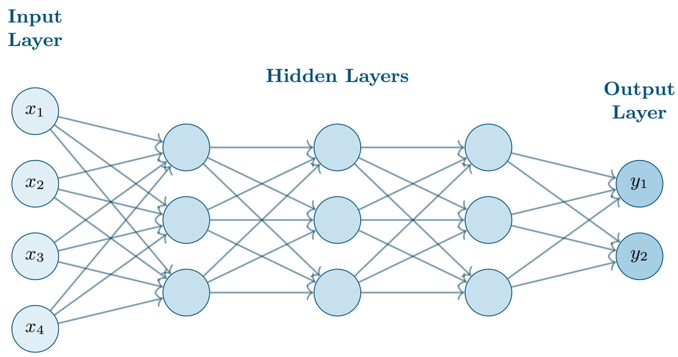


Figure 6: Architecture of an MLP with three hidden layers, fully connected in a feed-forward fashion.

This structure is very flexible: multiple nodes can be used in the output layer, so it can be applied to multi-class classification problems, and also to regression.

Backpropagation The perceptron learning algorithm is not suitable for training MLPs, as no expected output is available for the hidden layers, but just for the output layer.

Backpropagation is an efficient algorithm for computing the gradients also for large datasets. At a high level, the algorithm can be summarized as follows:

1. **Forward propagation**: Given an input $\boldsymbol{x}$, compute the output of the network $\hat{y}$ by propagating the input through the layers of the network;
2. **Loss computation**: Compute the loss $\mathcal{L}(y, f(\boldsymbol{x}, \boldsymbol{\theta}))$, where y is the true label and $f(\boldsymbol{x}, \boldsymbol{\theta})$ is the output of the network with parameters $\boldsymbol{\theta}$;
3. **Backward propagation**: the error signal is propagated backwards through the network and it's used to update the weights. We are basically computing how much each weight contributed to the error, and we can use this information to update the weights in the direction that reduces the error.

This concept is covered in more detail later in this chapter.

Cybenko's Theorem Cybenko's theorem states that a 2-layer feed-forward neural network with linear output can approximate any continuous function on a compact subset of $\mathbb{R}^n$ with arbitrary accuracy.

In practice, the more hidden layers we have, the more complex functions we can learn. Deep neural networks need a lot of labeled data to be trained, and cannot be trained easily.

2.3 Feedforward Neural Networks

An FFNN is a type of artificial neural network that define a mapping from the input space to an output space. The learned mapping $f(\boldsymbol{x})$ is defined as a composition of several functions, each representing a layer of the network.

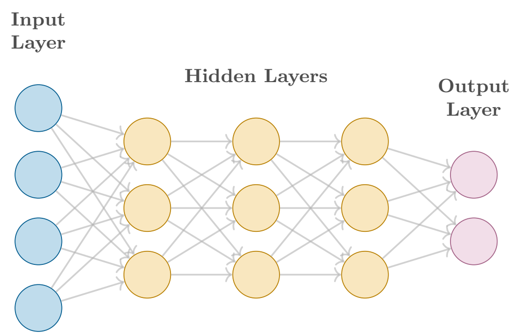


Figure 7: Example of a feedforward neural network with three hidden layers.

For example, taking in consideration the FFNN architecture shown in the figure 7 with three layers, we can express the mapping as follows:

$$f(\boldsymbol{x}) = f^{(5)}(f^{(4)}(f^{(3)}(f^{(2)}(f^{(1)}(\boldsymbol{x})))))$$

- The first layer $f^{(1)}(\boldsymbol{x})$ is usually referred to as the **input layer**, each node corresponds to a feature of the input data. Usually no computations are performed in this layer, as it simply serves as a conduit for the input data to be fed into the network;
- The layers $f^{(2)}$, $f^{(3)}$, and $f^{(4)}$ are called **hidden layers**, and they perform computations on the input data, applying a linear transformation followed by a non-linear activation function. The number of hidden layers and the number of neurons in each hidden layer depend on the architecture of the network and the complexity of the problem being solved.
- The last layer $f^{(5)}$ is called the **output layer**, and it produces the final output of the network. Depending on the type of problem, different activation functions can be used (e.g., linear for regression, softmax for classification) and different number of neurons can be used (e.g., one for binary classification and regression, one per class for multi-class classification).

The function composition can be described by a **directed acyclic graph** (DAG), where the nodes represent the functions (layers) and the edges represent the flow of data through the network. The term "feedforward" comes from the fact that the data flows in one direction, from the input layer to the output layer, without any cycles or loops in the graph.

2.3.1 Difference from Linear Models

Linear models are generally very easy to optimize (e.g., closed-form solutions or convex optimization), but they are limited in their expressiveness and cannot capture interactions between features or non-linear relationships in the data. FFNNs allow to overcome the limitations of linear models by introducing non-linear activation functions and multiple layers of computation.

How can we improve Linear Models? FFNNs are not the only way to improve linear models, but they are one of the most powerful and widely used approaches.

In general, to boost the performance of linear models, we can apply a non-linear transformation to the input data $\boldsymbol{x} \to \phi(\boldsymbol{x})$, where ϕ is a non-linear function. We can then apply the same linear model as before to the transformed data $\phi(\boldsymbol{x})$. The transformation ϕ can be implemented in various ways, such as:

- **Kernel Methods**: map the input data into a high-dimensional feature space (possibly infinite-dimensional) where linear models can be applied. This has enough capacity to capture complex relationships in the data, but achieves poor generalization performance for highly variable target functions.
- **Feature Engineering**: manually design and select features that capture relevant information from the input data. This was the traditional approach before the rise of deep learning, in particular for computer vision tasks, but it is time-consuming and requires domain expertise.
- **Neural Networks**: learn the transformation ϕ from the data itself. This shifts the engineering effort from designing features to designing the architecture of the network, but throwhs away the convexity and interpretability.

2.3.2 Training a FFNN

To train a Neural Network, we need to optimize the parameters of the network (weights $\boldsymbol{\theta}$) to drive the output of the network $f(\boldsymbol{x};\boldsymbol{\theta})$ to match the target variable y for a given input $\boldsymbol{x}$.

The training process is nothing different from training any other machine learning model. We need to define a loss function $\mathcal{L}(f(\boldsymbol{x};\boldsymbol{\theta}), y)$ that measures the discrepancy between the predicted output and the true target variable.

The largest difference with respect to linear models comes from the fact that most loss functions are non-convex, which removes the guarantee of finding a global minimum and makes the optimization process more challenging.

2.3.3 Modelling Choices

When designing a FFNN, we need to make several modelling choices, such as:

- **Cost Function**: the choice of the loss function $\mathcal{L}$ depends on the type of problem we are trying to solve (e.g., regression, classification) and the properties of the data. Each function has its own advantages and disadvantages, and the choice can significantly impact the performance of the model.
- **Form of Output**: the function applied in the output layer. Similarly to the cost function, the choice of the output units depends on the type of problem we are trying to solve;
- **Activation Function**: the non-linear function applied in the hidden layers. It's independent from the type of problem we are trying to solve, but it can significantly impact the performance of the model and the convergence during training.
- **Architecture**: the number of hidden layers, the number of neurons in each hidden layer, etc. The architecture of the network can significantly impact the performance of the model and the convergence during training.
- **Optimizer**: how we update the parameters of the network during training.
- **Regularizer**: how we prevent the model from overfitting the training data and improve generalization performance.

2.4 Loss Functions

The purpose of loss functions, also known as cost functions, is to measure the discrepancy between the predicted output of the network and the true target variable, guiding the parameter optimization process. Various functions can be defined based on the type of problem.

2.4.1 Classification

- **Cross-Entropy Loss**: used for multi-class classification problems. It measures the dissimilarity between the predicted probability of class $\hat{y}_i$ and the true class label. For a single data point, the loss can be expressed as:

  $$\mathcal{L}(y, \hat{y}) = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

  where C is the number of classes, y_i is the true label (one-hot encoded), and $\hat{y}_i$ is the predicted probability for class i.

- **Binary Cross-Entropy Loss**: special case of the categorical cross-entropy loss for binary classification problems. For a single data point, the loss can be expressed as:

$$\mathcal{L}(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

2.4.2 Regression

- **Mean Squared Error (MSE)**: commonly used for regression problems. It measures the average squared difference between the predicted and true values. For a single data point, the loss can be expressed as:

$$\mathcal{L}(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$$

  The squared term ensures that the function is convex (easy to optimize), but penalizes larger errors more than smaller ones, as the difference is squared. This causes the loss function to be sensitive to outliers in the data, as they can have a disproportionate influence on the overall loss.

- **Mean Absolute Error (MAE)**: similar to MSE but uses the absolute difference instead of the squared difference. For a single data point, the loss can be expressed as:

$$\mathcal{L}(y, \hat{y}) = |y - \hat{y}|$$

  This one penalizes large errors less than MSE, making it more robust to outliers.

- **Huber Loss**: combines the properties of MSE and MAE. It behaves like MSE for small errors and like MAE for large errors, providing a balance between sensitivity to outliers and smoothness of the loss function. Formally, for a single data point, the loss can be expressed as:

$$\mathcal{L}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{for } |y - \hat{y}| \leq \delta \\ \delta(|y - \hat{y}| - \frac{1}{2}\delta) & \text{for } |y - \hat{y}| > \delta \end{cases}$$

  where δ is a hyperparameter that determines the threshold between the MSE and MAE behavior.

Many more loss functions exist, and can be tryed out based on the specific problem and data characteristics.

2.4.3 Contrastive Learning

Another branch that is worth mentioning is the one used to train models in a self-supervised way, such as contrastive learning. As shown in the figure 8, the idea is to learn a representation of the data that captures the underlying structure and relationships between data points, that is invariant to certain transformations.

The loss function used in contrastive learning is designed to encourage the model to learn representations that are similar for similar data points and dissimilar for different data points. A possible loss function for contrastive learning is the **Noise Contrastive Estimation** (NCE) loss, which can be expressed as follows:

$$NCE_{Loss} = -\log \frac{e^{\text{sim}(\boldsymbol{z}_i,\boldsymbol{z}_j)}}{e^{\text{sim}(\boldsymbol{z}_i,\boldsymbol{z}_j)} + \sum_{k=1}^{N} e^{\text{sim}(\boldsymbol{z}_i,\boldsymbol{z}_k)}}$$

where $\boldsymbol{z}_i$ and $\boldsymbol{z}_j$ are the representations of two similar data points (anchor and positive), $\boldsymbol{z}_k$ are the representations of all data points (negatives), $\text{sim}(\cdot,\cdot)$ is a similarity function (e.g., cosine similarity).

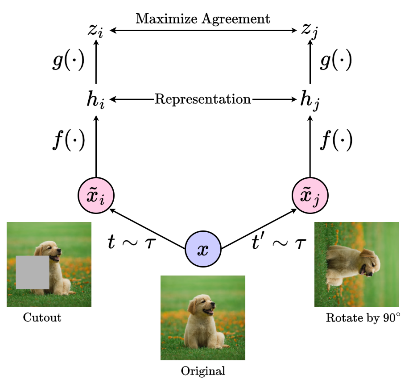


Figure 8: Contrastive learning Model

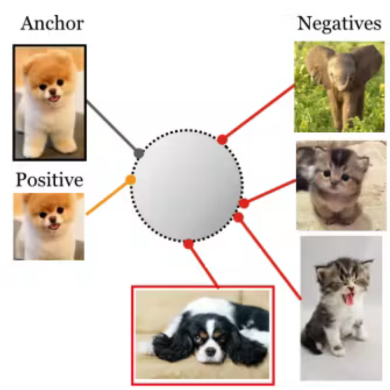


Figure 9: Contrastive learning loss function

2.5 Output Units

The output units are the last layer of the network, and their choice depends on the type of problem we are trying to solve.

- **Linear Units**: used for regression problems, where the output is simply a linear combination of the last hidden layer. For a given input $\boldsymbol{x}$, the output of the linear layer can be expressed as:

  $$y = \boldsymbol{\theta}^T \boldsymbol{x} + \theta_0$$

  Linear units usually do not saturate, which makes them effective for gradient-based optimization methods, as they do not suffer from the vanishing gradient problem.

- **Softmax Units**: used for multi-class classification problems. A softmax layer takes the unnormalized output of the last hidden layer and converts it into a probability distribution over the classes. For a given input $\boldsymbol{x}$, the output of the softmax layer can be expressed as:

  $$\hat{y}_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}$$

  where z_i is the output of the last hidden layer for class i, and K is the total number of classes.

  The *exp* function ensures that the output is always positive, and the normalization term ensures that the outputs sum to 1, making them naturally interpretable as probabilities.

2.6 Activation Functions

Unlike output layers, which are strictly dictated by the specific problem being solved, hidden units generally employ task-agnostic activation functions.

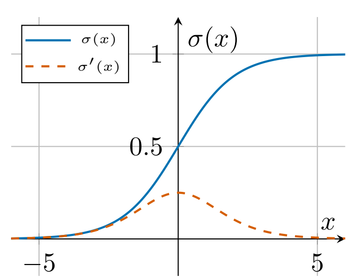
When selecting an activation function, its derivative is often more critical than the function itself. Because backpropagation relies entirely on these gradients to update the network's weights, a well-behaved derivative is essential for efficient learning.

Another highly desirable property is a zero-centered output. Zero-centering ensures that gradients do not constantly share the same sign, which allows for more balanced weight updates and significantly accelerates convergence during training.

2.6.1 Sigmoid

Maps the input to a value between 0 and 1, which can be interpreted as a probability. It is defined as:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



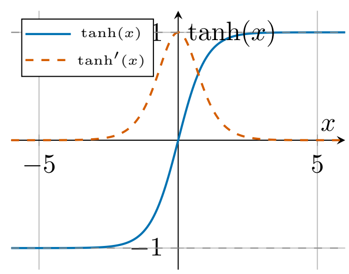
The function is smooth anddifferentiable everywhere, which is important for gradient-based optimization methods. However, its derivative **saturate** (approaches 0) across most of its domain, which lead to the problem of vanishing gradients during training.

As of today, the sigmoid function is rarely used in hidden layers, but it is still widely as building block in some architectures, such as in the gates of LSTM and GRU cells, or to compute the predicted position in local attention mechanisms.

2.6.2 Hyperbolic Tangent (tanh)

Similar to sigmoid but maps the input to a value between -1 and 1. It is defined as:

$$\tanh(x) = 2\sigma(2x) - 1 = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

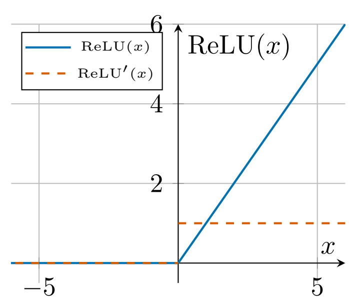


This is better than the sigmoid function as the outputs are zero-centered, helping with convergence during training. It still suffers from the vanishing gradient problem as for large positive or negative inputs the derivative still approaches 0.

2.6.3 ReLU

The most commonly used activation function in hidden layers. It is defined as:

$$\mathrm{ReLU}(x) = \max(0, x)$$



Born to solve the saturation problem of sigmoid and tanh, the ReLU function is in fact non-saturating for positive values (derivative is 1) which allows for faster convergence during training. It is not differentiable at 0, but in practice this is not a problem as the derivative can be defined as 0 or 1 at that point, with no significant impact on the training process.

However, this function has still a few negative aspects:

- **Non zero centered output**: similarly to the sigmoid, also the output of the ReLU is non-zero centered;
- **Dying ReLU**: If a neuron's inputs are predominantly negative, it outputs zero and produces a zero gradient. Consequently, its weights cannot be updated during backpropagation, leaving the neuron permanently inactive.

Different solutions have been proposed to solve the dying ReLU problem, such as:

- **Stochastic Gradient Descent (SGD)**: the randomness introduced by SGD can help to prevent neurons from dying, as it allows for some updates to the weights even for neurons that are currently inactive.
- **Lower Learning Rates**: using too high learning rates can cause the weights to update too aggressively, possibly leading to weights in the highly negative range;
- **Generalized ReLU**: A family of variants of the ReLU function that introduce a non-zero slope for negative inputs, allowing for a small gradient even when the input is negative.

  Formally, the Generalized ReLU can be expressed as:

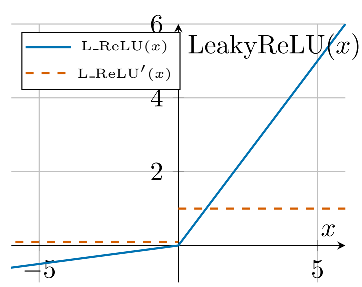
  $$\mathrm{GReLU}(x) = \begin{cases} x & x \geq 0 \\ \alpha x & x < 0 \end{cases}$$

  where α is a hyperparameter that controls the slope for negative inputs. Different choices of α lead to different variants of the ReLU function, such as:

  - **Leaky ReLU**: a variant of ReLU that allows a small, non-zero gradient for negative values. It is defined as:

$$\mathrm{LeakyReLU}(x) = \begin{cases} x & x \geq 0 \\ \alpha x & x < 0 \end{cases}$$

where $\alpha = 0.01$



- **Parametric ReLU (PReLU)**: where α is a learnable parameter that is updated during training;
- **Randomized ReLU (RReLU)**: where α is randomly sampled from a uniform distribution during training, and fixed to a value during testing;
- **Exponential Linear Unit (ELU)**: where the function is defined as:

$$\text{ELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

  This has all the benefits of ReLU while avoiding the dying ReLU problem, but requires more computational resources due to the exponential function.

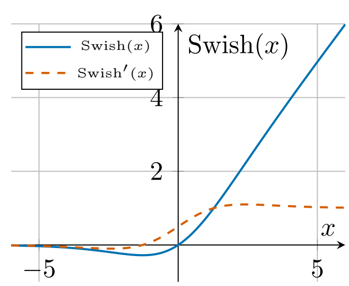
All this proposed solutions have the common property of being monotonic for positive values.

2.6.4 More Recent Activation Functions

Recent research has focused on developing activation functions that are non-monotonic, which can potentially lead to better performance in certain scenarios. Some of the most notable non-monotonic activation functions include:

- **Swish**: defined as $f(x) = x \cdot \sigma(x)$, where $\sigma(x)$ is the sigmoid function. The resulting function is non-monotonic and smooth everywhere, which has shown to perform better than ReLU in some cases, but it is more computationally expensive due to the sigmoid function.

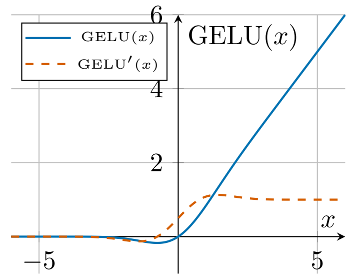
$$\text{Swish}(x) = x \cdot \sigma(x)$$



- **GelU**: defined as $f(x) = x \cdot \Phi(x)$, where $\Phi(x)$ is an approximation of the cumulative distribution function of the standard normal distribution. This function integrates the properties of ReLU and regularizations techniques, such as dropout and zoneout.

$$\text{GELU}(x) = x \cdot \Phi(x)$$

$$\approx 0.5x\left(1 + \tanh\left[\sqrt{\tfrac{2}{\pi}}\left(x + 0.044715x^3\right)\right]\right)$$



2.7 Architectures

When designing a FFNN, we need to make several choices regarding the architecture of the network, such as:

- **Number of Layers**: how deep should the network be?
- **Number of Neurons per Layer**: how many neurons should each layer have?
- **Expressiveness of the Network**: how complex should the network be to capture the underlying structure of the data?

We know that a 2-layer Neural network are universal function approximators, which means that they can approximate any continuous function to an arbitrary degree of accuracy, given enough neurons in the hidden layer. This implies that we could use a shallow network with a single hidden layer, by increasing the number of neurons in that layer (wide network).

However, in practice, it's not guaranteed that our training algorithm will find the optimal parameters that allow the network to approximate the target function, as we have to take into consideration the problems in optimization and generalization. It has been shown that deeper networks can generally capture more complex functions and achieve better generalization performance than shallower networks.

However, increasing the depth of the network is not always the best solution. Other approaches instead focused on architecutral design choices rather than depth. In particular, this idea was first explored in the context of Convolutional Neural Networks (CNNs), where the use of convolutional layers allows to get better performance with fewer parameters compared to FFNNs with fully connected layers.

3 Optimization

As shown in the previous chapter, to train a FFNN, we need to optimize its parameters in order to minimize the loss function. This is usually done using gradient-based optimization algorithms, such as gradient descent and its variants.

These algorithms are based on the idea of iteratively updating the parameters of the network in the direction of the steepest descent of the loss function, which is given by the negative of the gradient of the loss function with respect to the parameters. To better understand this, let's first introduce the concept of gradients.

3.1 Gradients

Gradients are a vector of partial derivatives that indicate the **direction** and **magnitude** of the **steepest ascent** of a function. Given the loss function $\mathcal{L}(\boldsymbol{W})$, the gradient with respect to the parameters w_i is given by:

$$\nabla_{w_i}\mathcal{L} = \left[\frac{\partial \mathcal{L}}{\partial w_1}, \frac{\partial \mathcal{L}}{\partial w_2}, \ldots, \frac{\partial \mathcal{L}}{\partial w_N}\right]$$

Each component of the gradient vector $\frac{\partial \mathcal{L}}{\partial w_i}$ measures the rate of change of the loss function with respect to that parameter.

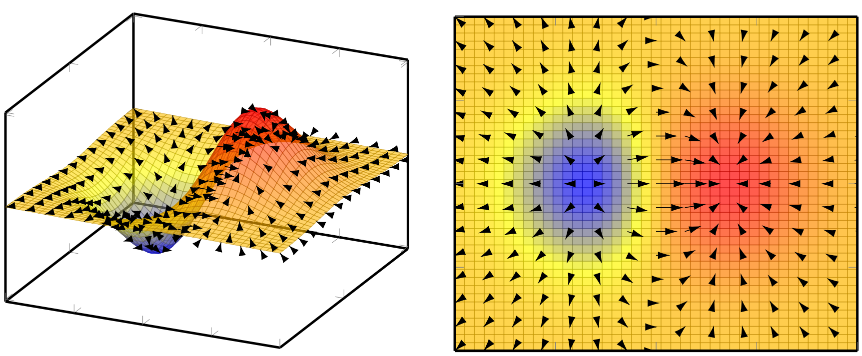


Figure 10: Visualization of derivatives and gradients.

By moving in the opposite direction of the gradient, we follow the path of **steepest descent**. This allows the optimization algorithm to iteratively navigate the parameter space and converge on the weights that minimize the loss function.

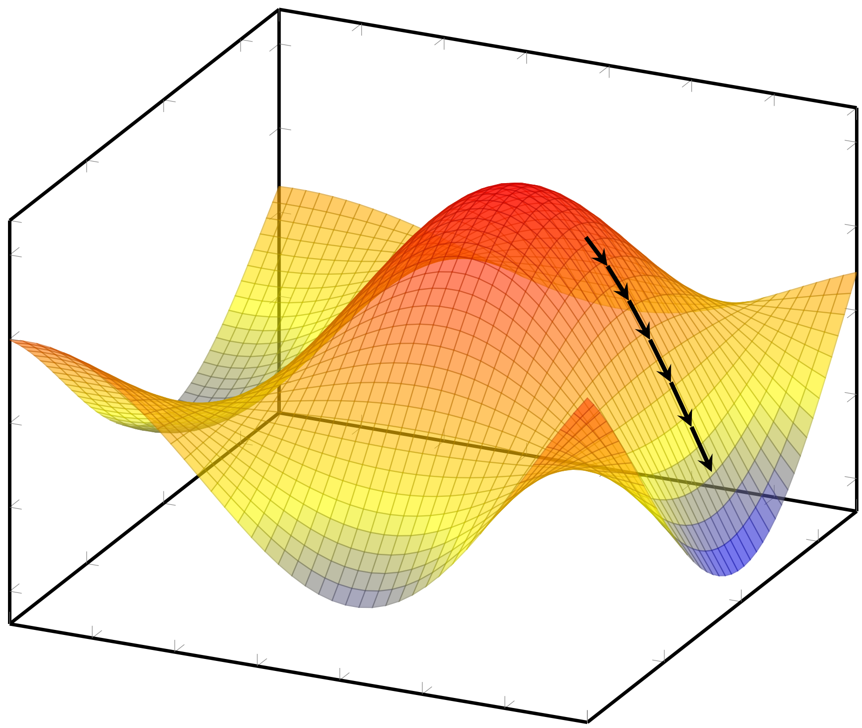


Figure 11: Visualization of Gradient Descent.

3.2 Problems with gradients

By now we know that in neural networks, the loss function is usually non-convex, which means that it can have multiple **local minima** and **saddle points**.

> **Local minima.** Local minima are points in the parameter space where the loss function has a lower value than in the surrounding area, but it is not the global minimum. This can lead to suboptimal performance of the model.

> **Saddle points.** Similarly, saddle points are points in the parameter space where the loss function has a gradient of zero, but does not correspond to a local minimum.

Both local minima and saddle points are issues that can arise during the optimization process. Altough they can lead to suboptimal performance, there are still other issues regarding optimization, that are much more important.

3.3 Gradient Descent

In the following sections, we will discuss different variants of gradient descent, which is the most commonly used optimization algorithm for training neural networks.

3.3.1 Batch Gradient Descent

Batch Gradient Descent is an optimization algorithm that minimizes a model's loss function by iteratively updating its parameters in the direction of the negative gradient. Because the gradient is computed over the entire training dataset, the parameters are updated only once per full data pass.

In the general case, we can express the update rule for the parameters $\boldsymbol{\theta}$ as follows:

Algorithm 1 Batch Gradient Descent

Require: Learning rate η
Require: Initial parameters $\boldsymbol{\theta}$

```
1: while stopping criteria not met do
2:     Compute gradient estimate over the entire dataset:
3:     ∇ ← (1/N) Σ_{i=1}^{N} ∇_θ L(f(x^(i); θ), y^(i))
4:     θ ← θ − η∇
5: end while
6: Output: Learned parameters θ
```

Here, η represents the learning rate, which dictates the step size of the updates, and some parameter initialization is required to start the optimization process. While computing the gradient over the full dataset guarantees a stable convergence trajectory, it can become computationally expensive for very large datasets.

> **Remarks on Learning Rate.** For better result, the learning rate η should be adjusted during training, starting with a higher value and gradually decreasing it as the optimization process progresses. This can be defined trough the Learning Rate Schedule:
>
> $$\eta_k = (1-\alpha)\eta_0 + \alpha\eta_\tau \quad \alpha = \frac{k}{\tau}$$
>
> The starting value of the learning must be choosen by trying different values and analyzing the learning curves on the validation set.

- If the Learning Rate is too large, the curves will show big oscillations.
- If it is too low, learning proceeds too slowly.

3.3.2 Stochastic Gradient Descent

SGD is a highly efficient variant of gradient descent that updates model parameters using a single, randomly selected training example per iteration. By bypassing the full dataset evaluation, SGD enables much faster parameter updates.

The update rule for SGD is defined as follows:

Algorithm 2 Stochastic Gradient Descent

Require: Learning rate η
Require: Initial parameters $\boldsymbol{\theta}$
while stopping criteria not met **do**
  $i \leftarrow$ randomly selected index from $\{1, 2, \ldots, N\}$
  $\nabla \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla$
end while
Output: Learned parameters $\boldsymbol{\theta}$

While evaluating a single sample accelerates each iteration, it introduces high variance into the gradient estimate, leading to a noisy, non-smooth optimization path. However, this stochasticity acts as a natural mechanism for exploration, frequently helping the algorithm escape saddle points and shallow local minima, even if it induces oscillations near the global optimum.

3.3.3 Mini Batch

To mitigate the issues of both Batch Gradient Descent and Stochastic Gradient Descent, we can use **Mini Batch Gradient Descent**.

Instead of using the entire dataset or a single training example, we can use a small batch of training examples to compute the gradient and update the parameters. The size of the mini batch is a hyperparameter that can be tuned based on the specific problem, dataset size, and hardware constraints.

This has many advantages, such as:

- **computational efficiency**: by using mini batches, the gradient computation can be vectorized and parallelized, optimizing the use of hardware resources. It also becomes independent of the dataset size.

- **balance between stability and speed**: mini batches provide a good balance between the stability of Batch Gradient Descent and the speed of Stochastic Gradient Descent.

3.4 Momentum

A major limitation of standard gradient-based optimization is its extreme sensitivity to the loss function's topography. In long, flat regions, gradients are exceptionally small, causing painfully slow convergence. Conversely, in steep ravines, large gradients lead to inefficient oscillations or even divergence. To mitigate these issues and stabilize the updates, the concept of **momentum** was introduced.

3.4.1 SGD with Momentum

The idea is to introduce a **velocity** term v that accumulates the gradients over time, allowing for faster convergence and smoother updates. This can be represented by exponentially decaying moving average of the negative gradients, which can be expressed as follows:

$$v \leftarrow \alpha v - \eta \nabla_{\boldsymbol{\theta}}(\mathcal{L}(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)}))$$

Where $\alpha \in [0, 1)$ is the momentum hyperparameter that controls the contribution of the previous velocity to the current update.

If $\alpha = 0$, the update reduces to standard SGD. Otherwise, the update incorporates to the current gradient a fraction of the historical gradients.

The update rule for the parameters can then be expressed as follows:

Algorithm 3 Stochastic Gradient Descent with Momentum

Require: Learning rate η
Require: Momentum hyperparameter α
Require: Initial velocity v
Require: Initial parameters $\boldsymbol{\theta}$

while stopping criteria not met **do**
  $i \leftarrow$ randomly selected index from $\{1, 2, \ldots, N\}$
  $\nabla \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
  $v \leftarrow \alpha v - \eta \nabla$
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + v$
end while
Output: Learned parameters $\boldsymbol{\theta}$

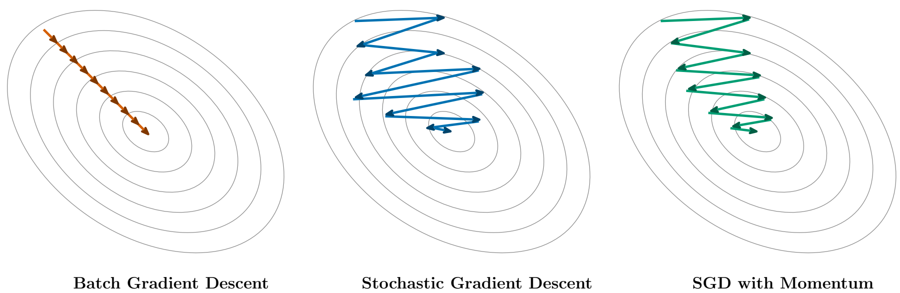


Figure 12: Different optimization algorithms.

3.4.2 SGD with Nesterov Momentum

While standard momentum accelerates gradient descent, its accumulated velocity can frequently lead to overshooting the minimum, leading to inefficient oscillations, especially in the presence of steep ravines.

Nesterov Momentum (or Nesterov Accelerated Gradient) mitigates this by introducing a **look-ahead** mechanism. Instead of computing the gradient at the current position, it first takes a partial step using the accumulated velocity, effectively "peeking" ahead. It then computes the gradient at this look-ahead position.

By evaluating the gradient ahead of time, the algorithm applies a preemptive correction that dampens oscillations before they occur, resulting in a more stable and efficient convergence.

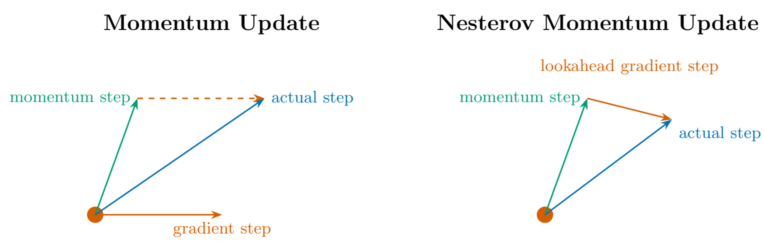


Figure 13: Comparison of different optimization algorithms and their update trajectories.

The update rule for Nesterov Momentum can be expressed as follows:

Algorithm 4 Stochastic Gradient Descent with Nesterov Momentum

Require: Learning rate η
Require: Momentum hyperparameter α
Require: Initial velocity v
Require: Initial parameters $\boldsymbol{\theta}$

```
while stopping criteria not met do
    i ← randomly selected index from {1, 2, ..., N}
    ∇ ← ∇_θ L(f(x^(i); θ + αv), y^(i))
    v ← αv − η∇
    θ ← θ + v
end while
Output: Learned parameters θ
```

Ultimately, this anticipatory update enables faster convergence and greater stability compared to standard momentum techniques.

3.5 Adaptive Learning Rates Methods

So far, we have seen optimization algorithms that use a fixed global learning rate η for all model parameters. This means that each of the parameters has the same importance during the optimization process, which is not always the case.

Adaptive learning rate methods are a family of optimization algorithms that adjust the learning rate for each parameter based on the historical gradients.

3.5.1 Adagrad

Adagrad (Adaptive Gradient Algorithm) dynamically adjusts the learning rate for each individual model parameter during training. It maintains a running sum of the squared gradients for every parameter and scales the learning rate by the inverse square root of this accumulated value.

Consequently, the algorithm automatically shrinks the learning rate for frequently updated features, while preserving larger updates for rare features to ensure they receive meaningful adjustments throughout the training process.

The update rule for Adagrad can be expressed as follows:

Algorithm 5 Adagrad

Require: Initial learning rate η
Require: Initial parameters $\boldsymbol{\theta}$
Require: Initial accumulated squared gradients $r = 0$

while stopping criteria not met **do**
  $i \leftarrow$ randomly selected index from $\{1, 2, \ldots, N\}$
  $\nabla \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
  $r \leftarrow r + \nabla^2$
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{\sqrt{r+\delta}} \nabla$
end while
Output: Learned parameters $\boldsymbol{\theta}$

where δ is a small smoothing constant added to prevent division by zero.

While Adagrad pioneered the use of adaptive learning rates and sparked a major trend in Deep Learning, it suffers from a critical flaw. Because the squared gradients are strictly positive, the accumulator r grows monotonically during training. This causes the effective learning rate to decay far too aggressively, often shrinking to a point where the model stops learning completely before reaching convergence.

3.5.2 RMSProp

The problem of Adagrad can be mitigated in non-convex settings by replacing the accumulated squared gradients with an **exponentially decaying average** of the gradient.

In non-convex optimization, the Adagrad accumulator r grows indefinitely, eventually shrinking the learning rate to a point where the model stops learning before reaching a local minimum. RMSProp "forgets" some of the past, allowing it to adapt to the local curvature of the loss surface.

The update rule for RMSProp can be expressed as follows:

Algorithm 6 RMSProp

Require: Initial learning rate η
Require: Initial parameters $\boldsymbol{\theta}$
Require: Initial accumulated squared gradients $r = 0$

while stopping criteria not met **do**
  $i \leftarrow$ randomly selected index from $\{1, 2, \ldots, N\}$
  $\nabla \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
  $r \leftarrow \beta r + (1 - \beta) \nabla^2$
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{\sqrt{r+\delta}} \nabla$
end while
Output: Learned parameters $\boldsymbol{\theta}$

where β is a decay factor.

It's important to note that there is also a variant of RMSProp that includes the Nesterov Momentum, which can further improve convergence and performance.

3.5.3 Adam

Adam (Adaptive Moment Estimation) is an optimization algorithm that combines the benefits of both momentum and adaptive learning rates. To do so, it maintains an exponentially decaying average of the gradients, analogous to the velocity v from Momentum, together with an exponentially decaying average of the squared gradients, analogous to the accumulator r from RMSProp. These are referred to as the **first moment** (mean) and **second moment** (uncentered variance) of the gradients, respectively.

Since both moments are initialized at zero, they are biased towards zero, especially during the initial stages of training. To compensate for this, Adam introduces a **bias-corrected estimate** for each of the two moments.

The update rule for Adam can be expressed as follows:

Algorithm 7 Adam

Require: Initial learning rate η
Require: Decay rates $\beta_1, \beta_2 \in [0, 1)$
Require: Initial parameters $\boldsymbol{\theta}$
Require: Initial first and second moment estimates $s = 0$, $r = 0$
Require: Initial timestep $t = 0$

while stopping criteria not met **do**
    $i \leftarrow$ randomly selected index from $\{1, 2, \ldots, N\}$
    $\nabla \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
    $t \leftarrow t + 1$
    $s \leftarrow \beta_1 s + (1 - \beta_1)\nabla$ ▷ Momentum
    $r \leftarrow \beta_2 r + (1 - \beta_2)\nabla^2$ ▷ RMSProp
    $\hat{s} \leftarrow \dfrac{s}{1 - \beta_1^t}$ ▷ **first moment** bias correction
    $\hat{r} \leftarrow \dfrac{r}{1 - \beta_2^t}$ ▷ **second moment** bias correction
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \dfrac{\eta}{\sqrt{\hat{r}} + \delta}\hat{s}$ ▷ Update parameters
end while
Output: Learned parameters $\boldsymbol{\theta}$

where δ is the same small smoothing constant used in Adagrad and RMSProp, added to prevent division by zero.

We can see Adam as a combination of all the techniques we have seen so far: s plays the role of the velocity v from Momentum, while r plays the role of the accumulator r from RMSProp. This combination makes Adam a very powerful optimization algorithm that is widely used in practice.

3.6 Batch Normalization

Batch Normalization (BN) is a technique originally introduced to address the problem of *internal covariate shift*. As a neural network trains, the distribution of each layer's inputs changes because the parameters of the preceding layers are constantly updating. This shifting distribution can slow down training, as the network must continuously adapt to new input scales and ranges.

Further research has shown that BN stabilizes the learning process by normalizing the inputs of a layer to have zero mean and unit variance, which allows for much faster convergence. It is typically implemented as a standalone layer and is usually applied to the pre-activations (before the non-linear activation function), though it can also be applied post-activation.

3.6.1 Standard Batch Normalization

To perform batch normalization, we first compute the mean and variance of a specific feature across a mini-batch. Let $H = \{H_1, \ldots, H_m\}$ denote the values of a specific feature for a mini-batch of size m. The batch mean μ_B and batch variance σ_B^2 are computed as follows:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} H_i$$

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(H_i - \mu_B)^2$$

To standardize the inputs, we then substitute the original values H_i with the normalized values $\hat{H}_i$:

$$\hat{H}_i = \frac{H_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

where ϵ is a small constant added for numerical stability to prevent division by zero.

Learnable Parameters While standardizing the inputs helps stabilize training, strictly forcing features to have zero mean and unit variance can limit the representational power (expressiveness) of the network.

To resolve this, we introduce two learnable parameters: γ (scale) and β (shift). These parameters allow the network to scale and shift the normalized inputs back to any distribution if it is optimal to do so. The final output of the batch normalization layer is:

$$H_i' = \gamma \hat{H}_i + \beta$$

This ensures the network learns a linear transformation of the normalized inputs, preserving expressiveness while still benefiting from the stabilization that BN provides.

- **During training:** The backpropagation algorithm updates γ and β based on the gradients of the loss function. Meanwhile, the layer maintains an exponential moving average of the batch means (μ_B) and variances (σ_B^2).
- **During inference (testing):** The running averages of μ and σ collected during training are used to normalize the inputs. We do not compute statistics from the test mini-batch, ensuring that the model's prediction for a specific test sample is completely independent of the other samples in that batch.

Properties While Batch Normalization was initially proposed to fix internal covariate shift, subsequent research suggests its primary benefit lies in making the optimization landscape significantly smoother. Key benefits include:

- Improving gradient flow through the network, preventing vanishing or exploding gradients;
- Enabling the use of much higher learning rates, which accelerates training;
- Reducing the network's sensitivity to sub-optimal weight initialization;
- Acting as a mild form of regularization, sometimes reducing the need for techniques like Dropout.

3.6.2 Layer Normalization

Alternative normalization schemes have been proposed for scenarios where Batch Normalization struggles (e.g., small batch sizes or Recurrent Neural Networks). One prominent alternative is Layer Normalization.

While Batch Normalization computes statistics across the *batch* dimension for a single feature, Layer Normalization computes statistics across the *feature* dimensions for a single sample:

$$\mu_L = \frac{1}{d} \sum_{j=1}^{d} H_j$$

$$\sigma_L^2 = \frac{1}{d} \sum_{j=1}^{d} (H_j - \mu_L)^2$$

where d is the total number of features (or hidden units) in the given sample, and H_j represents the j-th feature of that sample. The normalization step then proceeds identically to BN, applying a learned scale and shift.

This allows to normalize the inputs, without introducing any dependency between training examples. It has been shown that Layer Normalization is particularly effective in recurrent neural networks and transformers architectures.

Instance Normalization Instance Normalization (IN) is a normalization technique specifically designed for convolutional neural networks. While Batch Normalization computes statistics across the *batch* dimension for a single feature, and Layer Normalization computes statistics across the *feature* dimensions for a single sample, Instance Normalization computes statistics across the *spatial* dimensions of a single feature map, i.e., a single channel, of a single sample.

Let $H_1, \ldots, H_k$ denote the values of a specific feature map for a given sample, where k is the number of spatial locations in that feature map. The instance mean μ_I and instance variance σ_I^2 are computed as follows:

$$\mu_I = \frac{1}{k} \sum_{i=1}^{k} H_i$$

$$\sigma_I^2 = \frac{1}{k} \sum_{i=1}^{k} (H_i - \mu_I)^2$$

As with BN and LN, the values are then standardized using μ_I and σ_I^2, followed by a learned scale and shift.

Because these statistics are computed independently for each sample and each feature map, Instance Normalization is independent of both the other samples in the mini-batch and the other feature maps of the same sample.

By normalizing each feature map separately, Instance Normalization reduces variations in image contrast and illumination while preserving the structural information encoded by the convolutional features. This property has made it particularly effective in image generation and image-to-image translation tasks, such as neural style transfer, where the appearance of an image is modified while its semantic content is preserved.

Normalization Example To illustrate the differences between the various normalization techniques, consider a mini-batch of images represented by a tensor of dimensions (N, C, H, W), where:

- N denotes the batch size, i.e., the number of images in the mini-batch (e.g., $N = 32$);

- C denotes the number of channels (or feature maps). For an RGB image, $C = 3$, corresponding to the red, green, and blue channels. After one or more convolutional layers, C corresponds to the number of feature maps learned by the network;
- H and W denote the height and width of each feature map, respectively.

The main difference between Batch Normalization, Layer Normalization, and Instance Normalization lies in the dimensions over which the normalization statistics (mean and variance) are computed.

- **BN:** For each channel, the mean and variance are computed across the entire mini-batch and all spatial locations, i.e., over the dimensions $N \times H \times W$. Consequently, all samples in the mini-batch share the same normalization statistics for a given channel, making the normalization dependent on the other samples in the batch.
- **Layer Normalization (LN):** For each individual sample, the mean and variance are computed across all of its features, i.e., over the dimensions $C \times H \times W$. As a result, each sample is normalized independently of the other samples in the mini-batch.
- **IN:** For each sample and each channel independently, the mean and variance are computed only across the spatial dimensions, i.e., over the dimensions $H \times W$. Therefore, each feature map of each sample is normalized independently, making the normalization independent of both the other samples and the other feature maps.

4 Backpropagation

The perceptron learning algorithm (see Section 2.2.2) is designed for single-layer perceptrons and therefore cannot be directly applied to FFNNs. In a multilayer network, only the output layer can be directly compared with the ground truth, while the hidden layers do not have target values from which their errors can be computed.

To train an FFNN, the error observed at the output layer must be propagated backward through the network to determine how each parameter contributes to the overall loss. This is achieved by the **backpropagation** algorithm, which efficiently computes the gradient of the loss function with respect to every trainable parameter by repeatedly applying the chain rule of calculus.

Once these gradients have been computed, they are used by an optimization algorithm, such as **SGD**, to update the network parameters and minimize the loss function.

4.1 Idea behind Backpropagation

Before backpropagation, a few ideas were proposed to learn the weights inside a FFNN:

- **Randomly perturb the weights**: randomly perturb one weight at a time and check if the loss function decreases. This is inefficient and does not scale well with the number of parameters in the network.
- **Parallel Perturbation**: randomly perturb all the weights at the same time and check if the loss function decreases. This doesn't help at all, as we would need many perturbations to get a good estimate of the gradient;
- **Perturb the activations**: randomly perturb the activations of the hidden layers and check if the loss function decreases. This is more efficient than perturbing the weights, as we have fewer activations than weights, but it still does not scale well with the number of parameters in the network.

From the last idea, the backpropagation algorithm was born, which instead measure how the loss function changes with respect to the activations of the hidden layers.

4.2 The algorithm

The backpropagation algorithm, introduced in the section 2.2.3, consists of three main steps:

- **Forward Pass**: compute the output of the network for a given input $\boldsymbol{x}$ and the current parameters $\boldsymbol{\theta}$;
- **Error Computation**: compute the error at the output layer by comparing the predicted output with the true target variable y;
- **Backward Pass**: propagate the error signal back through the network, allowing to update the parameters of the network using the computed gradients.

As discussed previously, the expected outputs of the hidden units are not available in the training data. Instead, backpropagation determines how changes in the activations of the hidden units affect the overall loss. By repeatedly applying the chain rule, the algorithm computes the contribution of each hidden unit to the loss and, consequently, the gradients required to update the network parameters during the optimization step.

4.3 Single Neuron Case

Let's first consider a neural network with just one neuron in each layer. After performing the forward pass, the computation performed by layer l can be expressed as follows:

$$z^{(l)} = w^{(l)} a^{(l-1)} + b^{(l)}$$

followed by the activation function:

$$a^{(l)} = \sigma(z^{(l)})$$

where $a^{(l)}$ is the output of layer l, σ is a non-linear activation function, $w^{(l)}$ and $b^{(l)}$ are respectively the weight and bias of layer l, and $a^{(l-1)}$ is the output of the previous layer.

Output layer Let's now consider the output layer L.

$$a^{(L)} = \sigma(z^{(L)}) = \sigma(w^{(L)} a^{(L-1)} + b^{(L)})$$

Our goal is to understand how a small change in the parameters $w^{(l)}$ will affect the loss function $\mathcal{L}$. Since the loss function depends on the parameter through a sequence of intermidiate variables, we apply the chain rule of calculus:

$$w^{(l)} \longrightarrow z^{(l)} \longrightarrow a^{(l)} \longrightarrow \mathcal{L}$$

Applying the chain rule, we can express the derivative of the loss function with respect to the parameter $w^{(l)}$ as follows:

$$\frac{\partial \mathcal{L}}{\partial w^{(l)}} = \frac{\partial z^{(l)}}{\partial w^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial \mathcal{L}}{\partial a^{(l)}}$$

The three terms have a very intuitive interpretation:

- $\frac{\partial z^{(l)}}{\partial w^{(l)}}$ measures how a small change in the weight $w^{(l)}$ affects the pre-activation $z^{(l)}$;
- $\frac{\partial a^{(l)}}{\partial z^{(l)}}$ measures how a small change in the pre-activation $z^{(l)}$ affects the output of the activation function $a^{(l)}$;
- $\frac{\partial \mathcal{L}}{\partial a^{(l)}}$ measures how a small change in the output of the activation function $a^{(l)}$ affects the loss function $\mathcal{L}$.

Deeper layers As we move to earlier layers in the network, the chain of dependencies to the loss function grows longer. Consider a weight parameter $w^{(l-1)}$ in layer $l-1$. As illustrated in Figure 14, its path to the loss $\mathcal{L}$ is given by:

$$w^{(l-1)} \longrightarrow z^{(l-1)} \longrightarrow a^{(l-1)} \longrightarrow z^{(l)} \longrightarrow a^{(l)} \longrightarrow \mathcal{L}$$

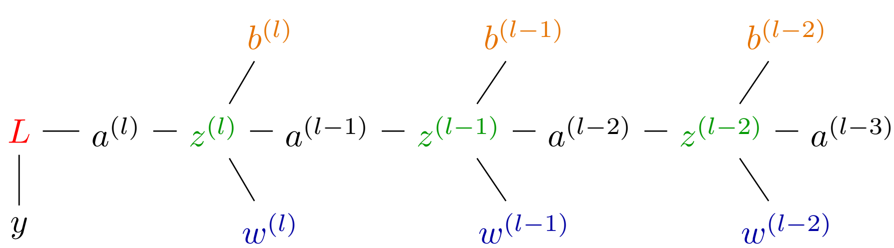


Figure 14: Dependency chain for a parameter in a deep network.

Applying the chain rule, the derivative of the loss with respect to $w^{(l-1)}$ is:

$$\frac{\partial \mathcal{L}}{\partial w^{(l-1)}} = \frac{\partial z^{(l-1)}}{\partial w^{(l-1)}} \cdot \frac{\partial a^{(l-1)}}{\partial z^{(l-1)}} \cdot \frac{\partial \mathcal{L}}{\partial a^{(l-1)}}$$

We can further expand the incoming gradient term, $\frac{\partial \mathcal{L}}{\partial a^{(l-1)}}$, as follows:

$$\frac{\partial \mathcal{L}}{\partial a^{(l-1)}} = \frac{\partial z^{(l)}}{\partial a^{(l-1)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial \mathcal{L}}{\partial a^{(l)}}$$

Crucially, the term $\frac{\partial \mathcal{L}}{\partial a^{(l)}}$ on the right-hand side is not computed from scratch; it was already evaluated one step earlier during the backward pass for layer l. Backpropagation exploits this by proceeding backward, from output to input, caching and reusing the gradient produced by the previous step. This recursive reuse eliminates the need to redundantly retrace the ever-lengthening path back to the loss for every single layer, which is exactly what makes the naive perturbation approaches from Section 2.2.3 computationally intractable.

4.4 General Case

Transitioning from a single neuron to a full network primarily involves tracking multiple paths, as a single parameter can now influence the loss function through several downstream connections.

To accommodate multiple neurons per layer, we update our notation: let $w_{ij}^{(l)}$ represent the weight connecting the j-th neuron in layer $l-1$ to the i-th neuron in layer l. The pre-activation (weighted sum) for the i-th neuron in layer l is then:

$$z_i^{(l)} = \sum_j w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$$

For the output layer, the dependency chain mirrors the single-neuron scenario, as each output neuron connects directly to the loss function:

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(L)}} = \frac{\partial z_i^{(L)}}{\partial w_{ij}^{(L)}} \cdot \frac{\partial a_i^{(L)}}{\partial z_i^{(L)}} \cdot \frac{\partial \mathcal{L}}{\partial a_i^{(L)}}$$

For a weight $w_{ij}^{(l-1)}$ in a hidden layer, the same reasoning applies unchanged from the single-neuron case, as each weight still only affects the loss through its corresponding neuron:

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l-1)}} = \frac{\partial z_i^{(l-1)}}{\partial w_{ij}^{(l-1)}} \cdot \frac{\partial a_i^{(l-1)}}{\partial z_i^{(l-1)}} \cdot \frac{\partial \mathcal{L}}{\partial a_i^{(l-1)}}$$

However, for hidden layers, an activation $a_j^{(l-1)}$ feeds into *every* neuron in the subsequent layer l. To evaluate how this activation affects the overall loss, we must sum the gradients across all these divergent paths:

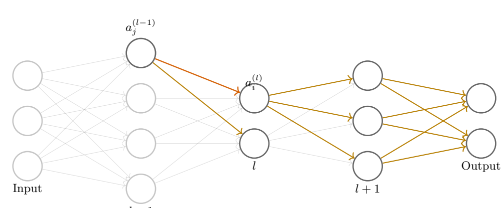


Figure 15: Dependency chain for a parameter feeding into h_2.

$$\frac{\partial \mathcal{L}}{\partial a_j^{(l-1)}} = \sum_i \frac{\partial z_i^{(l)}}{\partial a_j^{(l-1)}} \cdot \frac{\partial a_i^{(l)}}{\partial z_i^{(l)}} \cdot \frac{\partial \mathcal{L}}{\partial a_i^{(l)}}$$

Just as in the single-neuron case, backpropagation handles this elegantly through recursive caching. The term $\frac{\partial \mathcal{L}}{\partial a_i^{(l)}}$ inside the summation is not recomputed from scratch; it is the exact quantity already obtained while processing layer l. Instead of exhaustively tracing every divergent path back to the loss, layer $l-1$ simply aggregates these pre-computed gradients via a single weighted sum.

5 Regularization

Regularization techniques are methods used to reduce the generalization error of a model, but not necessarily its training error.

Regularization encompasses a variety of strategies designed to mitigate overfitting in machine learning models. This section explores the most common approaches applied to FFNNs.

While our primary focus here is on FFNNs, it is worth noting that many of these core methods easily generalize to other architectures, such as CNNs, which also rely on their own domain-specific regularization mechanisms.

5.1 Model Capacity

Model capacity refers to the ability of a model to fit a wide variety of functions. We have that:

- **Low capacity models** may underfit the data, as they are not able to capture the underlying structure of the data;
- **High capacity models** may overfit the data, by memorizing properties of the training data that are not generalizable to unseen data.

We can control the capacity of a model by adjusting the hypotheses space, which is the set of all possible functions that the model can represent. To deal with overfitting, we need to find a good balance between the capacity of the model and its ability to generalize to unseen data.

5.2 Simple approach

In addition to controlling the model capacity, many techniques have been proposed to control overfitting, some of which where already known in the context of machine learning, such as:

- **Collecting more data**: acquiring more training data can help to reduce overfitting. It allows to better represent the underlying distribution of the data. However, this is not always a valiable option, as it might be too expensive or time-consuming to collect more data;
- **Ensemble methods**: combining the predictions of multiple models can help to reduce overfitting. It allows to average out the errors of individual models, leading to better generalization performance. However, this approach is computationally and time-consuming, as it requires training multiple models independently.

5.3 Regularization Methods

For deeper architectures, controlling the complexity of the model is not as simple as adjusting the number of parameters. Many techniques have been proposed to regularize such complex models, without reducing their capacity. Commonly, many regularization methods can be combined to achieve better performance, as they often have complementary effects on the model.

In this section, we will discuss some of the most common regularization methods used in the context of FFNNs.

5.3.1 Parameter Norm Penalties

One of the most common regularization techniques is to add a penalty term to the loss function that is propotional to the norm of the model parameters. This encourages the model to have smaller weights, which has been shown to reduce overfitting.

The general formulation of this regularization technique can be expressed as follows:

$$\mathcal{L}_{\text{reg}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}) + \lambda\Omega(\boldsymbol{\theta})$$

where $\mathcal{L}(\boldsymbol{\theta})$ is the original loss function, $\Omega(\boldsymbol{\theta})$ is the regularization term, and λ is a hyperparameter that controls the strength of the regularization.

The most common choices for the regularization term $\Omega(\boldsymbol{\theta})$ are:

- **L2 Regularization**: also known as Ridge Regression, it adds a penalty term that encourages the model to have smaller weights by adding the squared norm of the weights to the loss function. The regularization term can be expressed as follows:

$$\Omega(\boldsymbol{\theta}) = \frac{1}{2}\sum_i ||\theta_i^2||$$

- **L1 Regularization**: also known as Lasso Regression, it adds a penalty term proportional to the absolute value of the weights. Differently from L2 regularization, L1 regularization favors the sparsity of the weights, setting the weights of less important features to zero. The regularization term can be expressed as follows:

$$\Omega(\boldsymbol{\theta}) = \sum_i ||\theta_i||$$

5.3.2 Data Augmentation

Data augmentation is a technique used to artificially expand a dataset by applying various transformations to the original samples. By exposing the model to a wider variety of inputs, this approach encourages better generalization. This method is particularly effective for visual data, where operations like rotation, scaling, or flipping can be applied without altering the underlying identity of the object (invariance).

However, transformations must be carefully tailored to the specific task to avoid corrupting the labels. For instance, in the MNIST digit classification dataset, rotating a "6" by 180 degrees transforms it into a "9," inadvertently changing its class.

Furthermore, data augmentation does not guarantee improved performance. Because the synthetic examples are merely variations of existing data rather than genuinely novel information, they may not always introduce sufficient functional diversity to the model.

5.3.3 Noise Injection

Noise injection is a regularization technique that involves adding random noise (variance scaled wrt the model weights) to the inputs, outputs, or weights of the model during training.

- **Input Noise**: Adding Gaussian noise to each input example during training can achieve effects similar to L_2 regularization. Each example can be represented as:
  $$x_i' = x_i + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$
  where $\mathcal{N}(0, \sigma^2)$ is Gaussian noise with mean 0 and variance σ^2.

  For a linear layer, the noise variance is propagated through the weights. If $y = \sum w_i x_i$, the added variance at the node level is proportional to $\sum w_i^2 \sigma^2$. Consequently, to minimize the expected loss, the network is encouraged to maintain smaller weights, mimicking the effect of L_2 regularization. High levels of noise typically generate a smoother mapping function, improving generalization performance.

  This concept is also foundational to **Denoising Autoencoders**, where the network is trained to recover the original input from a corrupted version, forcing it to learn more robust latent representations.

- **Weight Noise**: This technique adds noise to the weights during the forward pass, forcing the model to remain performant even when parameters are slightly perturbed. This encourages the training process to converge to a **flat minimum** of the loss function, where the model's performance is less sensitive to small changes in weight values.

- **Output Noise**: This discourages the model from making overconfident predictions by smoothing out the label distribution (often referred to as label smoothing). This prevents the network from outputting extremely high logits for a single class, which can lead to better calibration and reduced overfitting.

5.3.4 Early Stopping

A simple yet effective regularization technique is to stop the training process before the model starts to overfit the training data. This can be done by monitoring the performance of the model on a validation set during training, and stopping the training process when the performance on the validation set starts to degrade.

This procedure can be summarized as follows:

1. Start the training process with small weights;
2. Every time the error on the validation set decreases, save a copy of the model parameters;
3. When the training process is stopped, restore the model parameters to the values that achieved the best performance on the validation set.

5.3.5 Multi-Task Learning

Multi-task learning is a regularization technique that involves training a model on multiple related tasks simultaneously.

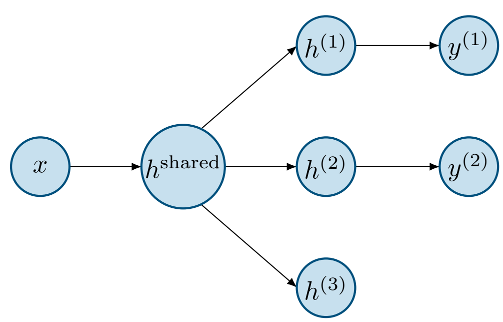


Figure 16: Example of a multi-task learning architecture. The model is trained on multiple related tasks simultaneously, sharing some intermediate representations.

Usually, we have several tasks that share the same set of input data $\boldsymbol{x}$, as well as some intermediate representations $h^{(shared)}$, but may have different output layers $f^{(1)}$, $f^{(2)}$, ..., $f^{(n)}$ for each task.

By sharing the intermediate representations, the model has 2 advantages:

- It can learn more robust and generalizable features, as it is forced to learn representations that are useful for multiple tasks;
- It can leverage the additional data from the other tasks, which can help to reduce overfitting and improve generalization performance.

5.3.6 Parameter Sharing

Parameter sharing (or parameter tying) is a regularization strategy that reduces model complexity by forcing different parts of the network to use the same weights. This can be achieved softly, by adding a penalty to the loss function that encourages weight similarity, or explicitly, by tying the parameters together during optimization so they remain identical.

This approach is particularly powerful when prior knowledge about the data structure suggests that certain patterns are universal. The prime example is the **CNN**, which applies a single, shared set of weights (a filter) across all spatial locations of an input. This structural constraint dramatically reduces the parameter count and allows the model to efficiently learn translation-invariant features.

5.3.7 Dropout

We have already seen that the ensemble of multiple models can help to reduce overfitting by averaging out the errors of individual models. In deep learning, this approach would be computationally and time-consuming, as it would require training multiple models independently.

Dropout is a regularization technique born from the idea of ensemble learning, which allows to approximate the effect of training multiple models by randomly dropping out a subset of the neurons during training.

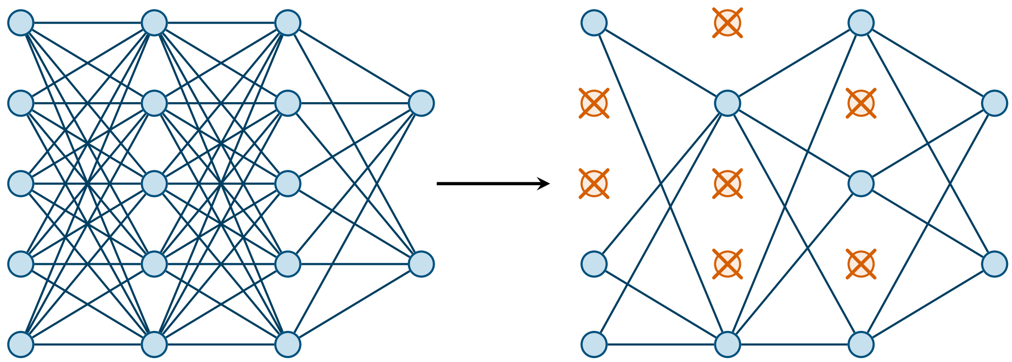


Figure 17: Example of a dropout regularization technique.

During the training process, each neuron is randomly dropped out (set to zero) with a certain probability p, which is a hyperparameter that controls the strength of the regularization. This forces the model to learn redundant representations, which allows the model to be more robust and less sensitive to the specific weights of individual neurons, thus improving generalization performance.

Dropout is similar to a *bagging* approach, where we train multiple models on different subsets of the data and average their predictions. We can see each trainging iteration as training a different model. At test time, we use the full network, but we scale the weights of the neurons by the dropout probability p to account for the fact that during training, each neuron was only active with probability p.

5.3.8 Implicit Regularization

Some techinques, such as SGD, have been shown to have an implicit regularization effect on the model. In particular, its stochastic nature can favor flatter minima of the loss function, which are associated with better generalization performance.

Also, using loss functions that are less sensitive to outliers, such as the Huber loss, can have an implicit regularization effect by reducing the influence of outliers on the training process.

6 Convolutional Neural Network

Some real world application of AI, such as image recognition, naturally work with an input space that is **locally structured**.

CNNs are a type of neural network architecture that is particularly well-suited for processing data with a 2D structure, such as images. Similarly to FFNNs, the idea for CNNs has some biological inspiration, as they are loosely based on the structure of the visual cortex in animals. Neurons are organized in a hierarchical manner, with each layer processing increasingly complex features of the input image, starting from simple edges and textures in the early layers to more complex shapes and objects in the deeper layers.

If we were to use a standard FFNN for processing images, we would need to flatten the 2D image into a 1D vector, which would result in a loss of spatial information and a large number of parameters to learn, making the model prone to overfitting.

CNNs design is based on 2 principles that allow them to effectively process images while keeping the number of parameters manageable:

- **Local Connectivity**: each neuron in a CNN is connected only to a small region of the layer before it. If we were to instead consider a FFNN, usually each neuron is connected to all the neurons in the previous layer;
- **Parameter Sharing**: the same set of weights is shared across different spatial locations in the input image. This allows the model to learn **shift-invariant filters**, while also significantly **reducing the number of parameters** to learn.

6.1 Convolution

Convolutional Neural Networks basically replace general matrix multiplications with convolution operations in at least one layer of the network.

6.1.1 1-D Convolution

Even though CNNs are typically used for processing 2D images, it's important to remember that the convolution operation can be applied to any type of data, including 1D sequences (e.g., time series, text)

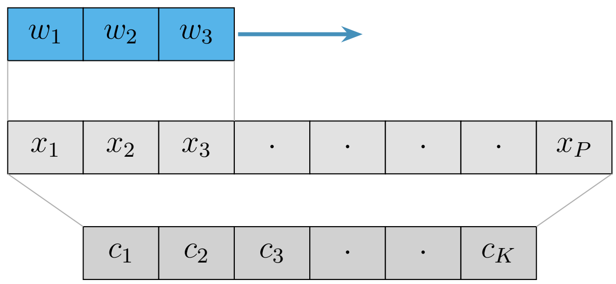


Figure 18: Example of a 1D convolution operation.

Given the example shown in the figure 18, we can express the convolution operation as follows:

$$c_1 = w_1x_1 + w_2x_2 + w_3x_3$$

$$c_2 = w_1x_2 + w_2x_3 + w_3x_4$$

And so on, where w_i are the weights of the filter and x_i are the input values.

6.1.2 2-D Convolution

In the 2D case, which is the one actually used in CNNs, the input is a 2D image, and the convolution operation is performed by sliding a small **filter** (also called **kernel**) across the image and computing the **dot product** between the filter and the local region of the image. Formally, the output of a convolution operation for a given input image $\boldsymbol{I}$ and filter $\boldsymbol{K}$ is defined as:

$$(\boldsymbol{I} * \boldsymbol{K})(x, y) = \sum_i \sum_j \boldsymbol{I}(x + i, y + j) \cdot \boldsymbol{K}(i, j)$$

where $*$ denotes the convolution operation.

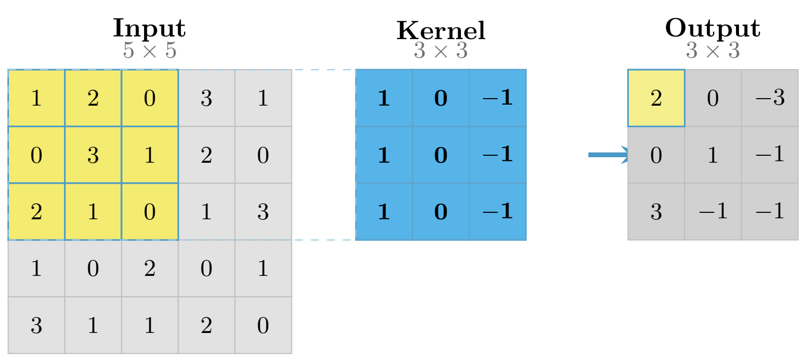


Figure 19: Example of a 2D convolution operation.

In the figure 19, we can see an example of a 2D convolution operation, where the input image is a 5×5 grid of pixel values, and the filter is a 3×3 kernel with specific values. The output of the convolution operation is a new 3×3 grid of values, where each value is computed as the dot product between the kernel and the corresponding local region of the input image.

6.2 Inside a CNN

A CNN is typically composed of several stacked layers, each performing a specific operation on the input data. Usually, low-level layers of the network learn to extract simple local features (e.g., edges, textures), while deeper layers learn to combine these features into more complex and abstract representations (e.g., shapes, objects).

Different types of layers can be used in a CNN, each one serving a specific purpose. In the following sections, we will briefly describe the most common types of layers used in CNNs.

6.2.1 Convolution Layer

The **convolutional layer** is the core building block of a CNN. It is responsible for performing the convolution operation to extract spatial features from the input data. Each layer contains a set of learnable filters (or kernels). These filters are spatially small but extend through the full depth of the input volume. The specific region of the input that a filter processes at any given moment is known as the local **receptive field**.

During the forward pass, the filter is convolved with the input. This process produces a two-dimensional **feature map** that highlights the spatial presence of specific visual patterns, such as edges or textures.

Rather than relying on hand-engineered features, CNNs automatically learn the optimal filter weights for the task at hand through backpropagation and gradient descent.

The behavior and output volume of a convolutional layer are governed by three primary hyperparameters:

- **Kernel Size**: The spatial dimensions of the filter (e.g., 3×3 or 5×5). This dictates the size of the receptive field and determines the extent of local information captured at each step.
- **Stride**: The step size used when sliding the filter across the input. A stride of 1 means the filter moves one pixel at a time. A stride of 2 moves the filter by two pixels at a time, which skips positions and effectively downsamples the spatial dimensions of the resulting feature map.

- **Padding**: Because a standard filter cannot center itself on the outermost edge pixels without extending beyond the input boundary, the convolution operation naturally shrinks the spatial dimensions of the output. To preserve the original spatial dimensions and retain information at the borders, we can apply **padding** (typically zero-padding), which artificially adds a border of zeros around the perimeter of the input before applying the convolution.

6.2.2 Non-Linearity Layer

After the convolution layer, a non-linearity layer is typically applied to introduce non-linear transformations to the feature maps. This is usually done using activation functions such as ReLU (Rectified Linear Unit), similar to what we have seen in FFNNs. The non-linearity layer allows the network to learn more complex and non-linear relationships between the input data and the output.

6.2.3 Pooling Layer

The pooling layer serves to downsample the feature maps generated by the preceding convolutional layer, reducing their spatial dimensions while preserving critical structural information.

This dimensionality reduction is typically achieved through operations such as *max pooling* or *average pooling*, which extract the maximum or mean value from localized regions of the feature map to construct a more compact output.

Crucially, pooling operations contribute to the translation invariance inherent in CNNs, rendering the network less sensitive to the precise spatial positioning of features within the original input image.

6.3 Different Architectures

This section outlines several seminal CNN architectures, detailing their unique structural designs and foundational contributions to computer vision.

6.3.1 LeNet (1998)

Proposed in 1998, LeNet is one of the earliest CNN architectures, originally designed for handwritten digit recognition.

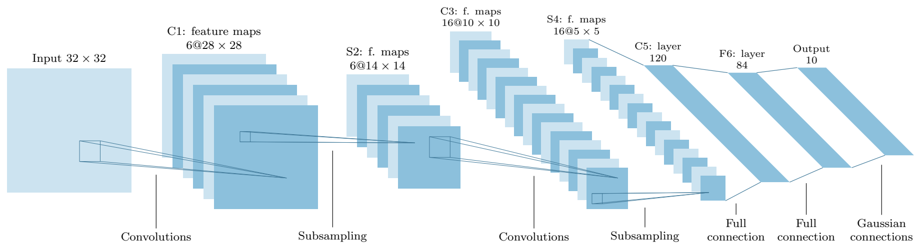


Figure 20: LeNet architecture.

It follows a straightforward structural pattern: two convolutional layers (C1 and C3), each immediately followed by a pooling layer (S2 and S4). The network concludes with two fully connected layers (C5 and F6) that feed into an output layer, producing a probability distribution across the ten possible digit classes (0-9).

6.3.2 AlexNet (2012)

Introduced during the 2012 ImageNet Large Scale Visual Recognition Challenge (ILSVRC), AlexNet significantly advanced the foundational concepts established by LeNet.

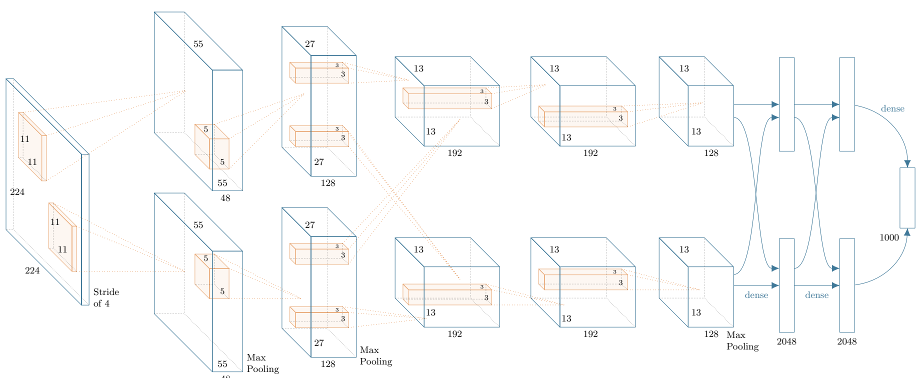


Figure 21: AlexNet architecture.

While retaining a similar architecture to LeNet, AlexNet dramatically scaled up the network's complexity, featuring five convolutional layers and three fully connected layers. This increased capacity was supported by a massive leap in available training data, utilizing the ImageNet dataset comprised of over one million labeled images across 1,000 categories.

AlexNet represented a watershed moment for deep learning. By decisively outperforming traditional methods on ImageNet, it demonstrated the immense power of deep convolutional networks and catalyzed the research community's shift toward increasingly deep architectures.

Beyond raw performance, AlexNet fundamentally transformed how the field approached feature extraction. Previously, image processing relied heavily on hand-crafted descriptors (such as SIFT or GIST) paired with traditional classifiers like SVMs. AlexNet proved that an end-to-end trained CNN could automatically learn hierarchical, robust feature representations directly from raw pixels.

Researchers soon discovered that these learned representations were highly transferable, eventually outperforming manual features across diverse applications, including object detection, segmentation, and natural language processing.

6.3.3 Zeiler and Fergus (2013)

After the success of AlexNet, a natural question was: how can we understand what the network has learned?

The Zeiler and Fergus architecture, proposed in 2013, introduced a method for visualizing and understanding the features learned by deep convolutional networks. This allowed to gain insights into the inner workings of CNNs, allowing for an easier parameter tuning and architecture design. This approach involved deconvolutional networks, that allowed to invert the operations of a CNN, visualizing the activations of the different layers and understanding which features were being captured by the network.

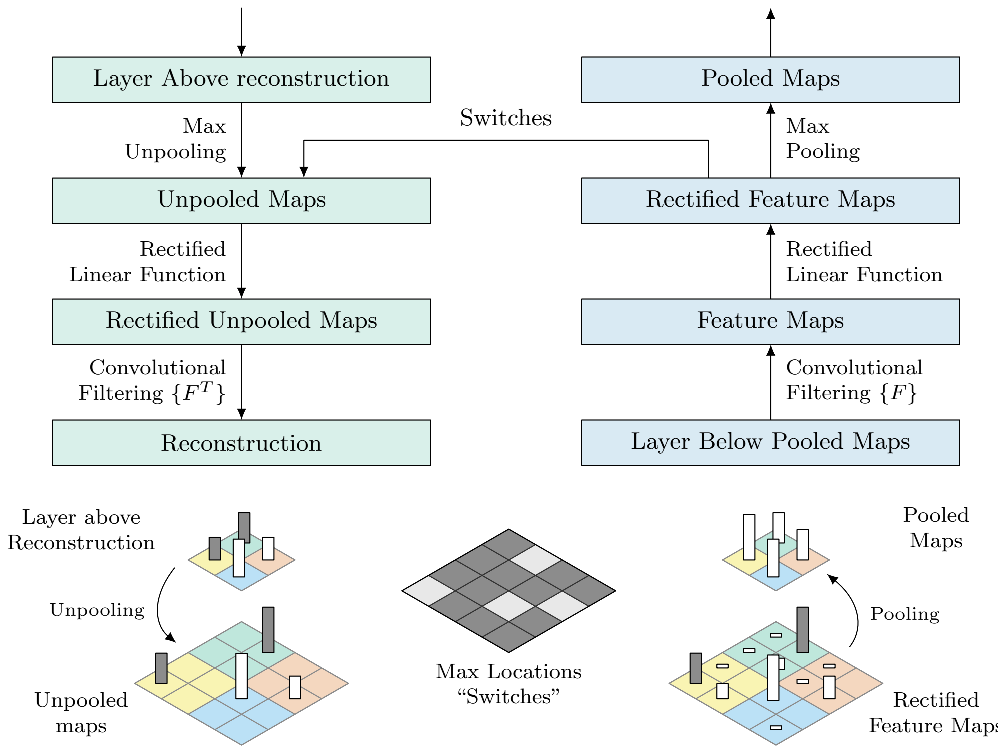


Figure 22: Zeiler and Fergus architecture for visualizing learned features.

How it works The deconvolutional network is designed to reverse the operations of a CNN, for each layer of the CNN. It is built using the same types of layers as a CNN, but in reverse order, and with the operations inverted. Obviously, the convolution operation and the activation functions are easily inverted, but the pooling operation is not invertible, as it removes information from the input feature map.

To address this issue, the authors introduced a method called "unpooling", which is used to approximate the inverse of the pooling operation. It basically uses the indices of the maximum values from the pooling operation to place the values back in their original positions, while filling the rest of the positions with zeros. This allows the deconvolutional network to reconstruct an approximation of the original input image, based on the activations of the different layers of the CNN.

Other approaches to explainability Other approaches to explainability in CNNs include:

- **Image Occlusion**: this method involves occluding different parts of the input image and observing how the output of the network changes, to understand which parts of the image are most important for the network's predictions;
- **Class Model Visualization**: more rigorous than the previous one, this method involves finding the input image that maximizes the activation of a specific class in the output layer, to understand which features are most important for that class;
- **Class Activation Mapping (CAM)**: uses each weight of the output layer to create a saliency map that highlights the regions of the input image that are most important for the network's predictions. The images are then averaged to create a heatmap that shows the most important regions of the input image for the network's predictions.

  This approch only works for fully convolutional networks, with no fully connected layers.
- **Grad-CAM**: An extension of CAM that can be applied to any CNN architecture. It uses the gradients of the output with respect to the feature maps of a specific convolutional layer to create a heatmap that highlights the important regions of the input image for the network's predictions.

6.3.4 VGGNet (2014)

VGGNet follows the same structural pattern as AlexNet but scales up the depth significantly, utilizing up to 19 layers. While several configurations of VGGNet were proposed with varying numbers of convolutional and fully connected layers, they all suffered from a common limitation: an enormous number of parameters, predominantly concentrated in the fully connected layers. This massive parameter count made the network computationally expensive and highly prone to overfitting.

Consequently, VGGNet was one of the last major architectures to rely heavily on this traditional sequential pattern. Subsequent architectures introduced novel design paradigms specifically to mitigate this bottleneck.

Notably, VGGNet replaced the larger convolutional filters (7×7 or 11×11) used in earlier models with stacks of smaller 3×3 filters. This design choice became a staple in later architectures, as stacking smaller filters achieves the same effective receptive field while substantially reducing the number of parameters and allowing for greater network depth.

6.3.5 GoogLeNet (2014)

Introduced in 2014, GoogLeNet debuted the **Inception Module**, a novel architectural component designed to capture features at multiple scales and levels of abstraction by applying various filter sizes in parallel within the same layer.

The key innovations of GoogLeNet include:

- **Parameter Efficiency:** By replacing computationally expensive fully connected layers with **Global Average Pooling**, GoogLeNet reduced the parameter count from approximately 60 million in AlexNet to just 6.7 million, despite being significantly deeper.
- **Depth:** The network consists of 22 layers, making it substantially deeper than its predecessors. This depth allows it to learn more complex and abstract features from the input data, contributing to its improved performance on the ImageNet dataset.
- **Dimensionality Reduction:** To keep the computational cost manageable, 1×1 convolutions are used to reduce the number of input channels before the expensive 3×3 and 5×5 convolutions.
- **Intermediate Supervision:** To mitigate the **vanishing gradient problem** caused by the increased depth, GoogLeNet introduced auxiliary classifiers at intermediate layers. These branches provide additional gradient signals back to the earlier layers during training.

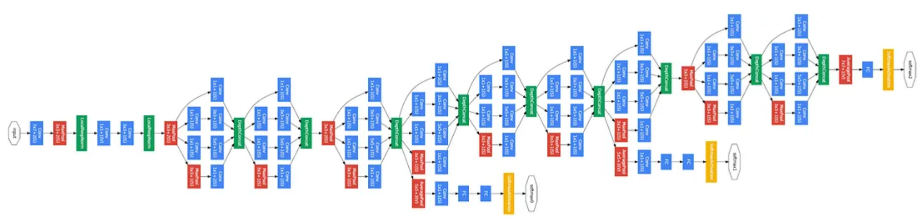


Figure 23: GoogLeNet architecture.

1×1 **Convolutions** Before detailing the Inception Module, it is crucial to understand the 1×1 convolution, the key mechanism that makes the module computationally feasible.

A 1×1 convolution is a filter with a spatial extent of a single pixel. Because it spans no spatial neighborhood, it does not aggregate information from adjacent pixels. Instead, at each spatial location, it computes a linear combination *across the input channels*. It effectively functions as a small fully connected layer applied identically across every spatial position, a technique often referred to as **cross-channel pooling** or *network-in-network*.

Its most critical property lies in manipulating the channel dimension. Because the number of filters dictates the number of output channels, a 1×1 convolution maps an $H \times W \times C_{in}$ tensor to an $H \times W \times C_{out}$ tensor. The spatial resolution

$(H \times W)$ remains untouched while the channel depth is adjusted. This makes it an efficient **bottleneck** layer capable of expanding or shrinking the channel dimension as needed. It serves two primary purposes:

- **Dimensionality Reduction:** Placing a 1×1 convolution before a heavy 3×3 or 5×5 convolution reduces the number of channels the larger filter must process, drastically lowering both parameter count and computational operations.
- **Added Non-linearity:** When followed by a ReLU activation function, it injects an additional non-linear transformation, deepening the representational capacity of the network with virtually no spatial computational cost.

To illustrate these computational savings, consider an input feature map of size $28 \times 28 \times 192$ where we wish to apply a 5×5 convolution to produce 32 output channels:

- **Direct convolution:** The computational cost is $28 \times 28 \times 32 \times 5 \times 5 \times 192$, which equals approximately 120 million multiply-add operations.
- **With a 1×1 reduction:** First, we reduce the input from 192 to 16 channels using a 1×1 convolution, and then apply the 5×5 convolution to go from 16 to 32 channels. The total cost is the sum of both operations: $(28 \times 28 \times 16 \times 192) + (28 \times 28 \times 32 \times 5 \times 5 \times 16)$, which is approximately 2.4 million + 10 million = 12.4 million multiply-adds.

This strategy yields roughly a 10x reduction in computation while generating the exact same output shape. This precise technique allows the multi-branch Inception Module to execute several parallel convolutions without an explosion in computational overhead.

Inception Modules The Inception Module is the fundamental building block of GoogLeNet. It is specifically engineered to learn diverse features across multiple scales while keeping the parameter count strictly regulated. Rather than forcing the network to choose a single filter size at a given layer, the module applies 1×1, 3×3, and 5×5 convolutions, alongside a 3×3 max-pooling operation, all in parallel. The resulting feature maps from these parallel operations are then concatenated along the channel dimension to form the final output of the module.

This parallel design offers two main advantages:

- **Multiscale Feature Extraction:** By processing multiple filter sizes simultaneously, the network automatically determines which spatial scale is most relevant for identifying a specific feature, making it exceptionally adept at handling objects of varying sizes within complex datasets like ImageNet.

- **Feature Redundancy and Generalization:** Processing the same input through diverse parallel paths creates structural redundancy in the learned feature maps. This redundancy pushes the model toward more robust, scale-invariant representations, which enhances generalization and acts as an implicit regularizer during training.

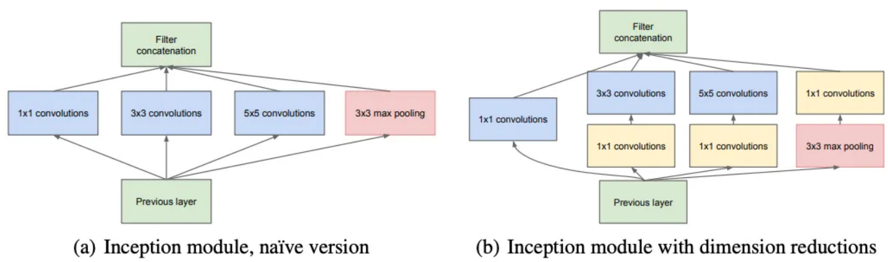


Figure 24: Inception Module.

6.3.6 ResNet (2015)

As Convolutional Neural Networks grew increasingly deep, researchers encountered the **degradation problem**: beyond a certain depth, network accuracy begins to saturate and then degrades rapidly. Crucially, this degradation is not a result of overfitting, as deeper models counterintuitively exhibit higher **training error** than their shallower counterparts.

ResNet (Residual Network), introduced in 2015, addressed this bottleneck by pioneering the use of **residual connections** (also known as skip connections). These connections bypass one or more layers, enabling the network to learn an identity mapping, meaning the input is passed through entirely unchanged if the intermediate non-linear layers are deemed unnecessary for the given task.

This simple yet profound architectural innovation provided several critical benefits:

- **Implicit Ensemble Learning:** Skip connections create multiple paths for information flow, effectively forming an implicit ensemble of numerous shallower networks embedded within the deeper architecture.

- **Smoother Loss Landscape:** The residual paths help to smooth out the loss landscape, making it significantly easier for optimization algorithms (like SGD) to navigate toward a global minimum, thereby improving convergence during training.

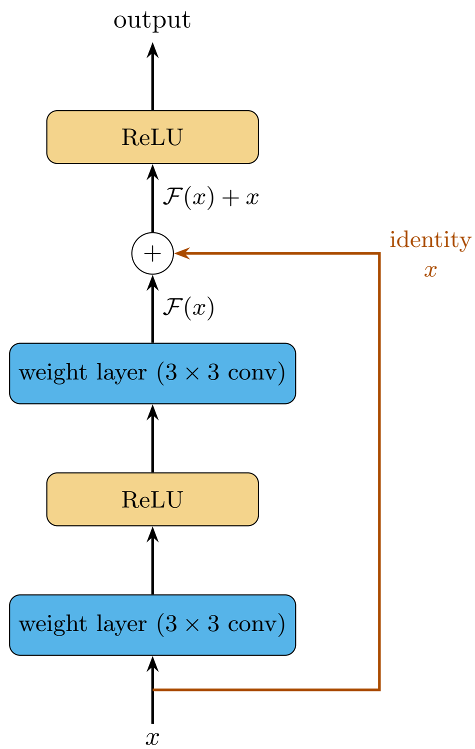


Figure 25: A residual block: the identity skip connection adds the block's input $\boldsymbol{x}$ back to the transformed output $\mathcal{F}(\boldsymbol{x})$.

Handling Dimension Mismatches Direct identity connections cannot be applied when the input and output dimensions (either spatial or channel-wise) of a residual block differ. To resolve this, the concept of a 1×1 convolution is utilized along the skip connection. This projection matches the dimensions of the input to the output feature maps while simultaneously introducing a learned linear transformation.

Variants of Residual Blocks In subsequent years, several modifications to the standard residual block were proposed to further optimize training and computational efficiency:

- **Pre-activation Residual Block**: This design alters the traditional sequence of operations by applying batch normalization and activation functions *before* the convolutional layers. This creates a cleaner, unobstructed identity mapping path that is free of non-linear interruptions, resulting in superior gradient flow during backpropagation.

- **Bottleneck Residual Block**: Designed to make exceedingly deep networks computationally feasible, this block uses a 1×1 convolution to compress the channel dimension, applies the standard 3×3 convolution on this reduced space, and then uses a final 1×1 convolution to expand the channels back to their original depth.
- **Wide Residual Block**: Rather than strictly scaling depth, this approach focuses on increasing the *width* (the number of channels) of the residual blocks. By utilizing wider convolutions and fewer layers overall, this architecture systematically explores the depth-to-width trade-off, demonstrating that wider, shallower residual networks can often match or exceed the performance of their deeper, thinner counterparts.

Following the profound success of ResNet, numerous modern architectures were developed that build upon the foundational concept of residual learning. The following two architectures are prominent examples of this continuing trend.

6.3.7 DenseNet (2018)

DenseNet are a type of CNN architecture that builds upon the idea of residual connections. Instead of using skip connections that add the input to the output of a layer, DenseNet introduces the **Dense Block**.

In a Dense Block, each layer is connected to all the other layers that compose the block. This means that for each layer, the feature maps of all the previous layers are concatenated together and used as input for the current layer.

This approach has a few advantages, such as:

- **Feature Reuse:** Since feature maps are preserved via concatenation, there is no need to relearn redundant features in subsequent layers.
- **Parameter Efficiency:** Despite the dense connectivity, DenseNets often require fewer parameters than ResNets for similar performance because they do not need large, wide layers to capture complex information.

6.3.8 SENet (2019)

SENet (Squeeze-and-Excitation Networks) introduces the concept of **channel attention** to CNN architectures. Rather than assuming all feature map channels contribute equally, SENet adaptively recalibrates channel-wise responses, learning to emphasize informative features and suppress irrelevant ones based on the task at hand.

This dynamic recalibration is achieved through a lightweight module called the SE block, which operates in two distinct steps:

- **Squeeze**: Spatial information is aggregated using global average pooling. This compresses the spatial dimensions of the input feature map, producing a $1 \times 1 \times C_{in}$ vector that serves as a global, statistical summary of each channel's activations.
- **Excitation**: The squeezed vector is passed through a gating mechanism (typically a small multi-layer perceptron) followed by a sigmoid activation function. This step learns the relative importance of each channel, generating a dynamic weight between 0 and 1 for each one. This is conceptually similar to the attention mechanism in Transformers; however, SENet applies weights channel-wise rather than token-wise.

The final output of the SE block is obtained by multiplying the original input feature maps by these learned weights, effectively scaling each channel by its relevance.

6.3.9 ConvNext (2022)

In the later years, the trend of transformer-based architectures has dominated the field of computer vision, with models such as **Vision Transformer (ViT)** and **Swin Transformer** achieving state-of-the-art performance on various vision tasks.

Transformers Vision Transformers usually work by splitting the input image into a grid of non-overlapping patches. Each patch is then flattened, along with its embedded positional information. The transformer architecturet then processes the entire sequence of patches, produing a global latent representation of the image, which is then used for downstream tasks such as classification or object detection.

Differently from what we have seen in CNNs, the spatial reletionships between the patches are not explicitly modeled, but rather learned through the self-attention mechanism of the transformer architecture.

ConvNext is a CNN architecture that was proposed in 2022. It showed that with some modifications to the standard CNN architecture, it is possible to achieve performance comparable to transformer-based architectures, while still maintaining the efficiency and simplicity of CNNs.

6.4 Regularization

To prevent overfitting, various regularization techniques were developed specifically for CNNs, such as:

- **Cutout**: this method involves randomly masking out a square portion of the input image during training, replacing it with the mean pixel value of the dataset. This approach forces the network to learn more robust features that are not reliant on specific parts of the image;
- **DropBlock**: an extension of dropout, where instead of randomly dropping individual neurons, contiguous regions of the feature maps are dropped during training, for all channels.
- **CutMix**: this method involves randomly cutting and pasting patches between training images. Class labels are also updated proportionally to the area of the patches. This encourages the model to learn features from different parts of the images, providing a regularization effect and improving generalization.

7 Expanding CNN Beyond Classification

In the previous chapter, we have seen how CNNs can be used for image classification tasks.

CNNs can however be applied to a wide range of other tasks beyond classification, such as **object detection**, **semantic segmentation**, and **captioning**, among others. In this chapter, we will explore some of these applications and how CNNs can be adapted to solve them.

7.1 Object Detection

Object detection is the task of identifying and localizing objects within an image. It can be seen as a combination of **regression** (where is the object located)and **classification** (what object is present). The task input would be an image, and the output would be a set of bounding boxes around the detected objects, along with their corresponding class labels.

This task is more complex than basic image classification, as it requires the model not only to recognize the presence of objects, but also to accurately localize them within the image. This makes also the training process more challenging, as several issues can arise, such as:

- **Data scarcity**: Object detection datasets are often smaller than classification datasets. Also, being the task more complex, it requires more data to train a model that can generalize well.
- **Poor localization**: the deep CNN proposed for classification may not be able to localize the objects accurately, because of their large receptive field and the pooling layers that reduce the spatial resolution of the feature maps.

To address these issues, several architectures have been proposed, such as **R-CNN**, **Fast R-CNN**, and **YOLO**, among others.

7.1.1 Before CNNs: DPM

Before the rise of deep learning and CNNs, one of the most popular approaches for object detection was the **Deformable Part Model** (DPM). DPM is a method that models objects as a collection of parts, where each part can be described by a set of features. The model learns to detect these parts and their spatial relationships to identify the presence of an object in an image.

This approach was quite effective for its time, but reached a plateau in performance already in 2010, before the advent of deep learning.

7.1.2 Selective Search

One of the key challenges in object detection is to efficiently search for objects in an image. **Selective Search** is a method that generates a set of candidate regions (also called **proposals**) in an image that are likely to contain objects.

Given the proposals, the process of object detection can be simplified to a classification task, where the model only needs to classify the proposals as containing an object or not, which is much easier than searching for objects in the entire image.

To detect objects at any scale, Selective Search uses a hierarchical grouping of superpixels, which are small regions of an image that have similar features (such as color, texture, or intensity).

The algorithm starts by generating all the possible candidate objects. Similar regions are then recursively merged together based on their similarity. Once the merging process is complete, the algorithm uses the resulting regions as candidate object proposals for the object detection task.

7.1.3 R-CNN

R-CNN (Region-based Convolutional Neural Networks) is one of the first architectures that successfully applied CNNs to object detection. The R-CNN architecture consists of three main steps:

- **Region Proposal**: The first step is to generate a set of candidate regions (proposals) in the image using Selective Search.
- **Feature Extraction**: Each proposal is then first resized to a fixed size (e.g., 224x224) and then passed through a CNN to extract a fixed-length feature vector.
- **Classification**: the extracted features are then used to classify each proposal as containing an object or not, and to assign a class label to the detected objects. This was typically done using a Support Vector Machine (SVM) classifier, which was trained on the features extracted from the proposals.

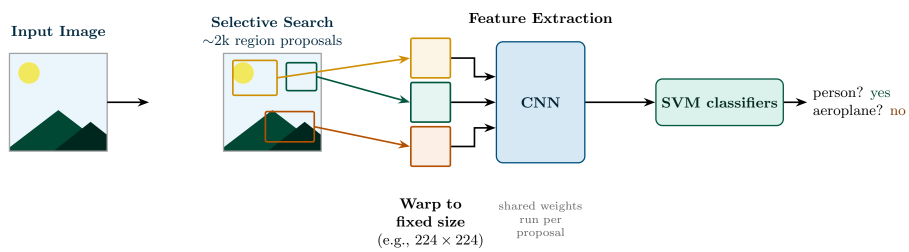


Figure 26: The R-CNN pipeline: Selective Search extracts ∼2k region proposals from the input image, each of which is warped to a fixed size and passed independently through a CNN. The resulting features are then classified by per-class SVMs.

This approach was a significant improvement over previous methods, but it still had a few major limitations, such as:

- **NOT an end-to-end architecture**: The R-CNN architecture is not an end-to-end architecture, as it requires separate training for the region proposal, feature extraction, and classification steps. Deep learning models are known to perform better when trained end-to-end, as they can learn to optimize the entire process jointly.
- **Image resizing**: Each proposal must be resized to a fixed size (e.g., 224x224) before being fed into the CNN for feature extraction. This can lead to a loss of information, especially for small objects, and can also introduce distortions in the aspect ratio of the proposals.
- **Computationally expensive**: The R-CNN architecture is computationally expensive, as it requires running the CNN for each proposal, which can be in the order of thousands per image (takes around 47 seconds per image on a GPU).
- **Selective Search**: The region proposal step relies on Selective Search, which is a hand-crafted and fixed algorithm. It may not be optimal for all types of images and objects, and can never be improved through learning.

7.1.4 Fast R-CNN

Fast R-CNN improves upon the original R-CNN by addressing its heavy computational redundancy and disjointed training pipeline. Instead of processing thousands of individual region proposals through a CNN, it utilizes a **shared computation strategy**.

The main innovations of Fast R-CNN are:

- **Region Proposals (unchanged)**: Fast R-CNN still relies on Selective Search to generate the ∼2k region proposals for each image, exactly as in R-CNN. What changes is how these proposals are processed afterward.
- **Shared Convolutional Architecture**: Instead of running the CNN separately for each proposal, Fast R-CNN processes the entire image through a convolutional network to produce a global feature map.
- **RoI Pooling Layer**: To handle the variable sizes of the region proposals, Fast R-CNN introduces a **Region of Interest** (RoI) **pooling** layer, which converts each region proposal into a fixed-size feature representation regardless of its original size. It works as follows:
  1. Each region proposal is mapped onto the convolutional feature map computed for the entire image.
  2. The mapped region is divided into a fixed-size grid (e.g., 7×7).
  3. Max pooling is performed independently within each grid cell, rather than over the region as a whole.
  4. The pooled values from all cells are concatenated into a fixed-size descriptor (e.g., a $7 \times 7 \times C$ tensor, where C is the number of feature channels), which is then fed to the fully connected layers.
- **Multi-Task Loss**: Fast R-CNN also introduces a multi-task loss function that combines classification and bounding box regression into a single optimization problem. This allows the model to learn both tasks simultaneously, improving the overall performance and efficiency of the training process.

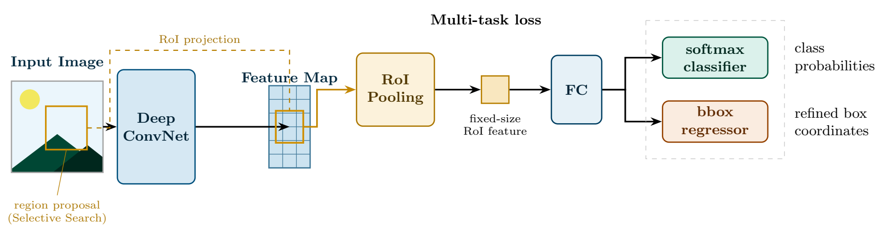


Figure 27: The Fast R-CNN architecture: the whole image is passed once through a deep ConvNet to produce a shared feature map. Each region proposal is projected onto this feature map and reshaped by the RoI pooling layer into a fixed-size feature vector, which is then fed to fully connected layers ending in a softmax classifier and a bounding box regressor, jointly optimized with a multi-task loss.

This approach allowed reducing inference time significantly (to around 0.3 seconds per image on a GPU, from the 47 seconds of R-CNN), while also slightly improving the accuracy of the model.

7.1.5 Faster R-CNN

While Fast R-CNN significantly reduced the computational cost of the detection head, it remained bottlenecked by external region proposal methods like **Selective Search**, which typically took ~ 2 seconds per image on a CPU. Not only did this slow down the inference time, but it also prevented the model from being trained end-to-end, as the region proposal step was not learnable.

Faster R-CNN addresses this by introducing a **Region Proposal Network (RPN)**, allowing for a truly unified, end-to-end object detection framework.

The architecture of Faster R-CNN consists of the following components:

- **Shared Convolutional Network**: Like its predecessor, it processes the entire image once to generate a high-level feature map.
- **RPN**: A small, fully convolutional network that slides over the feature map. At each sliding window location, it predicts objectness scores and bounding box offsets for k **anchor boxes** of varying scales and aspect ratios. This implies that, for each position in the feature map, the RPN generates k proposals, which are then filtered using non-maximum suppression (NMS) to remove redundant boxes.
- **RoI Pooling Layer**: This layer crops and reshapes the regions proposed by the RPN from the shared feature map into fixed-size feature vectors.
- **Classification and Bounding Box Regression**: The final stage consists of fully connected layers that predict the specific object class and perform fine-grained coordinates refinement.

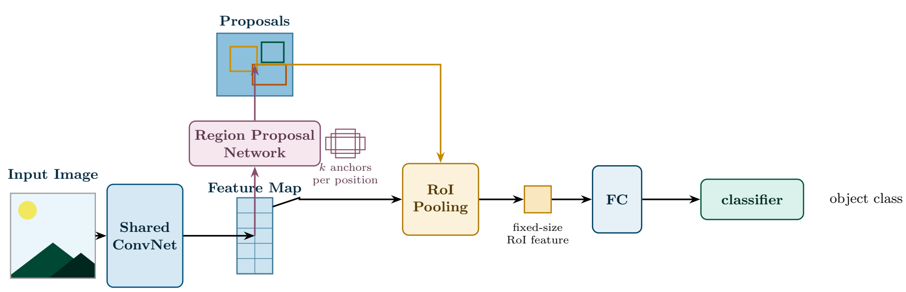


Figure 28: The Faster R-CNN architecture: a shared ConvNet produces a feature map that feeds both the Region Proposal Network, which slides over it to output k anchor-based proposals per position, and the RoI pooling layer, which pools the proposed regions into fixed-size features for the final classifier.

Performance By replacing Selective Search with the RPN, Faster R-CNN achieves an inference time of approximately **0.2 seconds** per image on a GPU.

Furthermore, because the RPN is trained specifically for the task, it typically yields higher-quality proposals than traditional computer vision algorithms, leading to improved mAP.

7.1.6 You Only Look Once

YOLO (You Only Look Once) is a different approach to object detection. Instead of using a two-stage process like R-CNN and its variants, which was very slow and hard to train, YOLO treats object detection as a single regression problem, directly predicting bounding boxes and class probabilities from the input image in one pass through the network.

The process of YOLO can be summarized as follows:

- **Grid Division**: The input image is divided into an $S \times S$ grid. Each grid cell is responsible for predicting m bounding boxes;
- **Bounding Box Prediction**: For each bounding box, the model predicts the coordinates of the box and a class confidence score that indicates the likelihood of the box containing an object and the class of the object;
- **Bounding Box Selection**: During inference, the model selects only the bounding boxes which have a confidence score above a certain threshold.

This design allows YOLO to achieve real-time object detection, with inference times as low as 0.02 seconds per image on a GPU, while still maintaining competitive accuracy compared to two-stage detectors. Even YOLO has some limitations, such as:

- **Small Object Detection**: YOLO can struggle with detecting small objects, as the grid cell responsible for predicting the object may not have enough information to accurately localize it.
- **Different Aspect Ratios**: YOLO uses a fixed set of anchor boxes, which may not be optimal for all object shapes and sizes, leading to suboptimal performance for objects with unusual aspect ratios.
- **Loss Function**: The original YOLO loss is very complex and can be difficult to optimize, as it combines classification and localization losses, which can have different scales and gradients.

7.2 Instance Segmentation

A different task that can be tackled with CNNs is **instance segmentation**. Instance segmentation is the task of assigning a class label to each pixel in an image, while also distinguishing between different instances of the same class.

For example, in an image containing multiple people, instance segmentation would not only identify the pixels belonging to the ”person” class, but also differentiate between each individual person.

This task is more complex than object detection, as it requires not only localizing objects but also accurately segmenting them at the pixel level.

Mask R-CNN

One of the most popular architectures for instance segmentation is **Mask R-CNN**, showing again the flexibility of CNNs to be adapted to different tasks. Mask R-CNN builds upon the Faster R-CNN architecture by adding a branch for predicting segmentation masks on each Region of Interest (RoI).

To achieve this, Mask R-CNN introduces:

- **RoIAlign Layer**: Instead of using the RoI Pooling layer from Faster R-CNN, which can quantize the RoI and lead to misalignments between the RoI and the extracted features, Mask R-CNN uses a RoIAlign layer that preserves the spatial alignment of the features, allowing for more accurate mask predictions.
- **Mask Branch**: In addition to the classification and bounding box regression branches, Mask R-CNN includes a separate branch that predicts a binary mask for each RoI. This branch is typically implemented as a small **fully convolutional network** (FCN) that takes the aligned features from the RoIAlign layer and outputs a mask of fixed size for each class, in a pixel-to-pixel manner that preserves the spatial layout of the RoI.

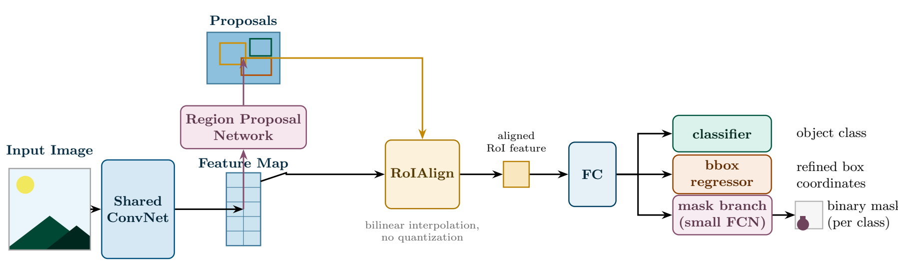


Figure 29: The Mask R-CNN architecture: it extends Faster R-CNN by replacing RoI Pooling with RoIAlign, which uses bilinear interpolation instead of quantization to preserve precise spatial alignment, and by adding a third sibling head, the mask branch, that predicts a binary mask for each RoI alongside the classifier and bounding box regressor.

8 CLIP

CLIP (Contrastive Language-Image Pretraining) is a state-of-the-art **Vision-Language Model (VLM)** developed by OpenAI. It leverages natural language supervision to learn a wide range of visual concepts by simply predicting the correct image-caption pairings during pre-training.

The architecture consists of two main components:

- An **image encoder** (typically a CNN or Vision Transformer).
- A **text encoder** (typically a standard Transformer).

CLIP's key innovation is training both encoders simultaneously using a **contrastive learning approach**. This mechanism maps images and text into a single, shared embedding space, forcing semantically similar pairs close together while pushing dissimilar pairs apart. This unified representation is what allows the model to easily perform versatile downstream tasks, such as zero-shot learning, image classification, and object detection.

Before CLIP Prior to CLIP, traditional computer vision relied on heavily curated datasets of images and fixed, categorical labels. This supervised approach had significant limitations:

- **Data collection bottleneck:** Acquiring large volumes of high-quality, human-annotated data is expensive and time-consuming.
- **Poor generalization:** Models were confined to their specific training tasks and struggled to adapt to new domains.
- **Inflexible classes:** Introducing even a single new class required inefficiently retraining the model from scratch.

By replacing fixed labels with open-ended textual descriptions, CLIP overcomes these hurdles, enabling zero-shot classification of novel visual concepts without any task-specific retraining.

8.1 Ideas that led to CLIP

Rather than introducing an entirely new paradigm, CLIP successfully combined and scaled existing machine learning concepts:

- **Webly supervised learning**: Exploiting the vast, readily available pools of data on the internet to train models without manual annotation.

- **Weak supervision from multimodal data**: Leveraging naturally occurring pairings of visual and textual information to learn a joint embedding space.
- **Scaling laws**: Natural Language Processing (NLP) had already demonstrated massive performance leaps by scaling up models and datasets (e.g., GPT-3). CLIP proved this principle translates effectively to computer vision.

8.2 Dataset: WebImageText

Previous vision-language research relied on relatively small datasets like MS-COCO, which contains roughly 100,000 images. To test whether their approach could truly scale, OpenAI constructed a massive new dataset called *WebImage-Text* (WIT).

WIT consists of 400 million image-text pairs scraped from the internet. To ensure diversity and broad conceptual coverage, the collection process utilized 500,000 distinct search queries encompassing common terms, word combinations, and random concepts. To maintain an approximate class balance, the dataset was capped at 20,000 image-text pairs per query.

While WIT is orders of magnitude larger than prior datasets, it is also inherently noisy. Because the web data was minimally curated, the quality of the image-text pairings is not strictly guaranteed, meaning the resulting model inevitably inherits the biases and limitations of the internet.

8.3 Contrastive Pretraining

The core objective of CLIP's training phase is to **maximize** the cosine similarity between the embeddings of the N matching image-text pairs in a batch, while **minimizing** the similarity of the $N^2 - N$ incorrect pairings.

This is achieved using a symmetric, softmax-based contrastive loss function:

$$\mathcal{L} = -\frac{1}{2N} \sum_{i=1}^{N} \left(\underbrace{\log \frac{\exp(\mathrm{sim}(\boldsymbol{v}_i, \boldsymbol{t}_i)/\tau)}{\sum_{j=1}^{N} \exp(\mathrm{sim}(\boldsymbol{v}_i, \boldsymbol{t}_j)/\tau)}}_{\mathit{Image}\to\mathit{Text}} + \underbrace{\log \frac{\exp(\mathrm{sim}(\boldsymbol{t}_i, \boldsymbol{v}_i)/\tau)}{\sum_{j=1}^{N} \exp(\mathrm{sim}(\boldsymbol{t}_i, \boldsymbol{v}_j)/\tau)}}_{\mathit{Text}\to\mathit{Image}} \right), \tag{8.1}$$

where $\boldsymbol{v}_i$ and $\boldsymbol{t}_i$ represent the embeddings of the i-th image and text pair, and sim is the cosine similarity function.

The two terms of the sum play a symmetric but distinct role:

- **Image → Text**: for each image embedding $\boldsymbol{v}_i$, this term is a softmax over all N text embeddings in the batch. It pulls $\boldsymbol{v}_i$ closer to its matching text embedding $\boldsymbol{t}_i$, while pushing it away from all the other $N-1$ text embeddings.
- **Text → Image**: symmetrically, for each text embedding $\boldsymbol{t}_i$, this term is a softmax over all N image embeddings, pulling $\boldsymbol{t}_i$ closer to its matching image embedding $\boldsymbol{v}_i$, while pushing it away from all the other $N-1$ image embeddings.

Averaging both directions ensures the embedding space is aligned consistently, regardless of whether an image is used to query a text or vice versa.

> **Temperature.** The hyperparameter τ modulates the sharpness of the probability distribution. A lower temperature sharpens the distribution, which boosts confidence but increases the risk of overfitting. Conversely, a higher temperature softens the distribution, regularizing the model at the potential cost of peak performance.

> **Training Phase.** Due to the immense scale of the WebImageText dataset, both the image and text encoders are trained entirely from scratch without relying on pre-trained weights. Only basic data augmentation techniques are applied to the visual inputs during this process.

Limitations of the training approach This training approach has some limitations regarding the computational resources required. In particular, since we need to compute the cosine similarity between all pairs of images and text in a batch (N^2 pairs), the size of the batch is limited by the available memory.

8.4 Inference

During inference, CLIP can be used for various tasks such as image classification, object detection, and zero-shot learning.

- **Image classification**: Given an image, CLIP can be used to classify it by comparing its embedding to the embeddings of a set of candidate labels (text descriptions);
- **Text to Image retrieval**: Given a text query, CLIP can be used to retrieve relevant images by comparing the text embedding to the embeddings of a set of candidate images. First, the text query is encoded using the text encoder to obtain its embedding. Then, the cosine similarity between the text embedding and the embeddings of a set of candidate images is computed. The images with the highest similarity scores are returned as the most relevant results.

- **Image to Image retrieval**: Given an image query, CLIP can be used to retrieve relevant images by comparing the image embedding to the embeddings of a set of candidate images. Similarly to text to image retrieval, the image query is encoded using the image encoder to obtain its embedding. Then, the cosine similarity between the image embedding and the embeddings of a set of candidate images is computed. The images with the highest similarity scores are returned as the most relevant results.
- **Zero-shot learning**: CLIP enables zero-shot learning, meaning it can recognize and classify images based on textual descriptions without needing to be retrained on specific classes.

8.5 Fine Tuning

Once the power of CLIP was demonstrated on zero-shot learning tasks, the next step was to explore how to further improve its performance on specific tasks.

A first approach was to fine-tune either the image encoder, the text encoder, or both on a specific dataset. This was found to be ineffective, as the model had already learned a powerful shared embedding space, and fine-tuning on a specific dataset often led to overfitting and a decrease in performance on other tasks.

Other approaches based on prompt engineering were found to be more effective in improving the performance of CLIP on specific tasks, without the need for fine-tuning the model itself. Here, we will discuss some of the most effective techniques for improving CLIP's performance through prompt engineering.

8.5.1 Basic Prompt Engineering

Prompt engineering is the process of designing and optimizing the input prompts to improve a model's performance on specific tasks. Since CLIP learns a shared embedding space, the phrasing of the text prompts significantly impacts the results.

For example, since images are rarely described with a single word in the wild, using a template like `"a photo of a [class]"` often outperforms using the raw class name alone. On ImageNet, this simple modification yielded a gain of 1.3% in accuracy.

8.5.2 Prompt Ensembling

Further improvements can be achieved by using multiple prompts for each class and averaging their embeddings. For the class "dog," one might ensemble prompts like *"a photo of a dog," "a picture of a dog,"* and *"a photo of a cute* *dog."* On ImageNet, this approach provided an additional 3.5% gain in accuracy compared to a single prompt.

8.5.3 Advanced Approaches

Prompt engineering can still be difficult and time-consuming. Many times a slight change in the prompt can lead to a significant change in performance, with no clear way to predict which prompts will work best for a given task.

To address this, more advanced approaches have been proposed.

CoOp CoOp (Context Optimization) is a method that learns continuous **prompt vectors** instead of using discrete text prompts. These prompt vectors are then fed into the text encoder to generate the class embeddings. Different choices can be made, such as the number of prompt vectors, their dimensionality, where they are inserted in the text encoder (e.g., before the class name, after the class name, or both), and whether they are shared across classes or specific to each class.

CoOp optimizes the prompt vectors using a small amount of labeled data from the target task, allowing it to adapt the prompts to better suit the specific task at hand. This approach has been shown to significantly improve the performance of CLIP on various tasks, especially when the amount of labeled data is limited.

Even though the results are quite impressive, they are still not humanly interpretable, as the learned prompt vectors may not correspond to any meaningful words or phrases. Also, the approach was shown to have poor generalization performance to new classes, as the learned prompt vectors may be overfitted to the specific classes seen during training.

CoCoOp CoCoOp (Contextualized Context Optimization) is an extension of CoOp that addresses the issue of generalization to new classes.

CoCoOp learns a **meta-network** that generates some meta-prompts based on the output of the image encoder. These meta-tokens are then summed with the learned prompt vectors to generate the final prompts that are fed into the text encoder. This allows CoCoOp to adapt the prompts based on the specific image being processed, improving generalization to new classes and reducing overfitting to the classes seen during training.

The problem with this approach is that it is more computationally expensive than CoOp, as it requires an additional forward pass through the meta-network for each image, during the **training phase**.

Visual Prompt Tuning All the approaches described so far have focused on optimizing the text prompts, while keeping the image encoder fixed. Visual Prompt Tuning is a method that optimizes the input to the image encoder instead of the text encoder. Similarly to CoOp, it learns a set of continuous prompt vectors that are instead added to the input image before being fed into the image encoder.

Multimodal prompt Learning Sometimes, optimizing only the text prompts or only the image prompts leads to suboptimal performance, as ther's no general rule to determine which one is more effective for a given task. Multimodal Prompt Learning is a method that optimizes both the text and image prompts simultaneously, allowing for a more holistic approach to prompt engineering.

Using a small neatwork (for example, a transformer), the method learns to generate both text and image prompts based on the input image and the target task. The prompts are then fed into the respective encoders to generate the final embeddings, which are used for classification or other downstream tasks.

8.6 Beyond CLIP - SigLIP

To resolve the limitations of CLIP, presented in Equation 8.3, a new method called SigLIP (Sigmoid Loss for Language-Image Pretraining) was proposed. Instead of using a softmax-based contrastive loss, where each pair of image and text is compared to all other pairs in the batch, SigLIP uses a sigmoid-based loss that compares each pair independently, treating the problem as a binary classification.

For each pair of image and text, the model predicts a score that indicates whether the pair is a match or not (e.g., whether the image and text are semantically related). Because each pair is treated independently, the batch size can be much larger without running into memory issues, allowing for more efficient training on large datasets.

9 Recurrent Neural Networks

Unlike the architectures previously discussed, which only process data represented as fixed-size vectors for both input and output, **RNNs** are designed to handle sequential data.

Standard feed-forward models face significant limitations with real-world data types such as text, audio, and video, which are inherently sequential and vary in length.

RNNs were the first architectures specifically designed to handle this kind of data, enabling solutions to a wide range of tasks, such as:

- **Document classification**: Given a document, classify it into one of several categories (e.g., spam vs. non-spam). We know that the document size cannot be fixed a priori, as it can vary significantly from one document to another.
- **Sentiment analysis**: similar to document classification, but the goal is to classify the sentiment of a text.
- **Image captioning**: Given an image, generate a caption that describes the content of the image. This is a more complex task, as it requires the model to generate a sequence of words that describe the image, which can have variable length.
- **Dynamical Systems**: dynamical systems are systems that evolve over time, based on the current state, some external input, and a predetermined set of rules that define how the system transitions from one state to another. RNNs can be used to learn a non-fixed transition function that captures the underlying dynamics of the system, allowing for accurate predictions of future states based on past observations.

9.1 RNN Architecture types

While classical FFNNs are categorized as **one-to-one** (or single-to-single) architectures, taking one input and producing one output, RNNs are classified based on how they map input sequences to output sequences. The primary types include:

- **Many-to-One**: This type of RNN takes a sequence of inputs and produces a single output. This is the case of document classification, where the input is a sequence of words and the output is a single class label.

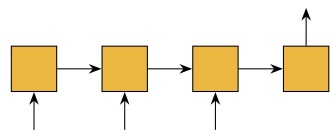


Figure 30: Many-to-One RNN architecture.

- **One-to-Many**: This type of RNN takes a single input and produces a sequence of outputs. This is the case of image captioning, where the input is an image and the output is a sequence of words that describe the image.

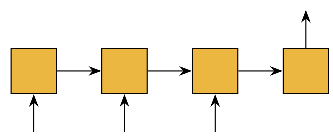


Figure 31: One-to-Many RNN architecture.

- **Many-to-Many**: This type of RNN takes a sequence of inputs and produces a sequence of outputs. We can have 2 subtypes of this architecture:
  - **Synchronized**: The input and output sequences have the same length, and an output is generated for every input (e.g., Frame-level video classification or Part-of-Speech tagging).
  - **Encoder-Decoder**: The model first processes the entire input sequence (encoding) before generating an output sequence of a potentially different length (decoding), as seen in Machine Translation.

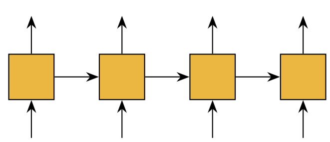


Figure 32: Many-to-Many Synchronized RNN architecture.

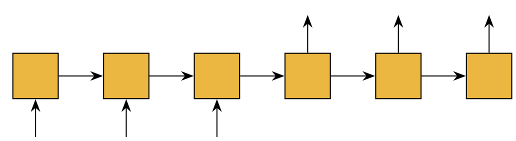


Figure 33: Many-to-Many Encoder-Decoder RNN architecture.

9.2 Vanilla RNNs

The simplest form of RNN is the **Vanilla RNN**, which consists of a single hidden layer that is recurrently connected to itself.

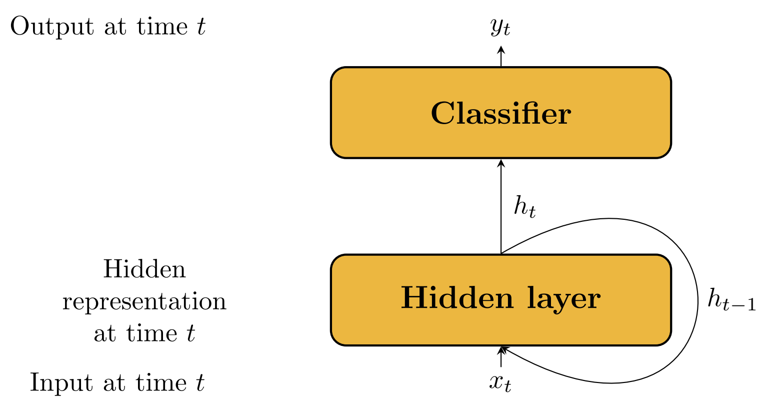


Figure 34: Vanilla RNN architecture.

At each time step t, the model combines the current input $\boldsymbol{x}_t$ with the previous hidden representation $\boldsymbol{h}_{t-1}$ to compute a new hidden representation $\boldsymbol{h}_t$. The updated state $\boldsymbol{h}_t$ is then used to produce the output y_t and is passed forward to the next step $t+1$. By managing this recurrent connection, the model creates a memory across time steps, enabling it to capture the temporal dependencies in the input sequence.

Hidden representation The hidden representation $\boldsymbol{h}_t$ can be computed as:

$$\boldsymbol{h}_t = \sigma_{\boldsymbol{W}}(\boldsymbol{x}_t, \boldsymbol{h}_{t-1}) \tag{9.1}$$

Where $\sigma_{\boldsymbol{W}}$ is a non-linear activation function (e.g., ReLU, Tanh) applied to a linear transformation of the current input $\boldsymbol{x}_t$ and the previous hidden state $\boldsymbol{h}_{t-1}$, with $\boldsymbol{W}$ representing the learnable parameters of the model.

Taking the Tanh activation function as an example, we can express the hidden representation as:

$$\boldsymbol{h}_t = \tanh(\boldsymbol{W}_x\boldsymbol{x}_t + \boldsymbol{W}_h\boldsymbol{h}_{t-1}) \tag{9.2}$$

where $\boldsymbol{W}_x$ and $\boldsymbol{W}_h$ are the weight matrices associated with the input and hidden state, respectively.

Unrolled Representation The RNN can also be unrolled in time, which means that we can represent the RNN as a feed-forward network where each layer corresponds to a time step in the sequence.

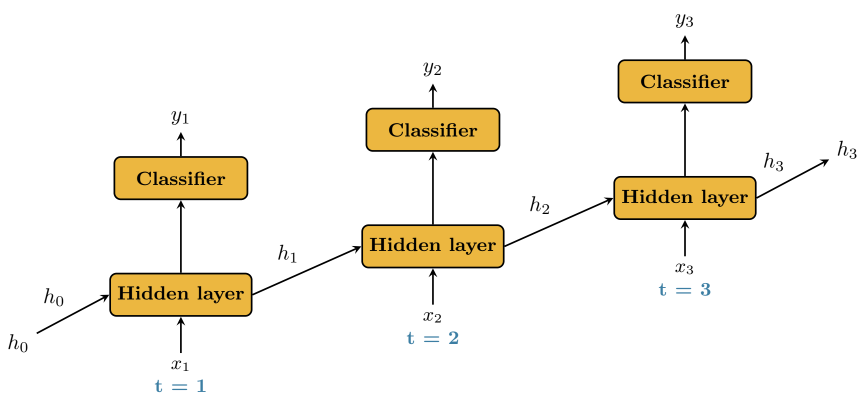


Figure 35: Unrolled RNN architecture.

9.2.1 Backpropagation Through Time

To train RNNs, we use a variant of backpropagation called **Backpropagation Through Time** (BPTT), which follows the same principles as standard backpropagation.

The unfolded RNN is treated as a deep feed-forward network, where each layer corresponds to a time step in the sequence. The weights are shared across all time steps, and the error is backpropagated through the entire sequence, allowing the model to learn from the temporal dependencies in the data.

BPTT Example Let's see a simple example of how BPTT works in a Vanilla RNN:

- **Forward Pass**: The input sequence $(\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3)$ is fed into the RNN, producing hidden states $(\boldsymbol{h}_1, \boldsymbol{h}_2, \boldsymbol{h}_3)$ and outputs (y_1, y_2, y_3).
- **Loss Calculation**: The loss is computed based on the outputs and the true labels for each time step. The error is expressed as the sum of the losses at each time step:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t(y_t, \hat{y}_t) \tag{9.3}$$

  In our example, with $T = 3$, we have:

$$\mathcal{L} = \mathcal{L}_1(y_1, \hat{y}_1) + \mathcal{L}_2(y_2, \hat{y}_2) + \mathcal{L}_3(y_3, \hat{y}_3) = \mathcal{L}_1 + \mathcal{L}_2 + \mathcal{L}_3 \tag{9.4}$$

- **Backward Pass**: in this example we will compute the gradient of the loss with respect to the weights associated with the hidden layer $\boldsymbol{W}_h$. Note that the process is analogous for the weights associated with the input layer $\boldsymbol{W}_x$ and the output layer $\boldsymbol{W}_y$.

The gradient of the loss with respect to these weights is computed by summing the contributions from all time steps:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_h} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{W}_h} \tag{9.5}$$

Where each term $\frac{\partial \mathcal{L}_t}{\partial \boldsymbol{W}_h}$ is computed as:

$$\frac{\partial \mathcal{L}_t}{\partial \boldsymbol{W}_h} = \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{h}_t} \cdot \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{W}_h} \tag{9.6}$$

Not only that, but the gradient of the loss with respect to the hidden state at time step t is also influenced by the hidden states at subsequent time steps. This implies that when computing $\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t}$, we also need to consider the chain of time steps dependencies. Considering for example the hidden state at time step $t = 1$, we have:

$$\boldsymbol{h}_1 \to \boldsymbol{h}_2 \to \boldsymbol{h}_3 \to \mathcal{L} \tag{9.7}$$

This gives us:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_1} = \frac{\partial \boldsymbol{h}_2}{\partial \boldsymbol{h}_1} \cdot \frac{\partial \boldsymbol{h}_3}{\partial \boldsymbol{h}_2} \cdot \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_3} \tag{9.8}$$

9.2.2 Challenges with BPTT: The Vanishing Gradient

When generalizing to a sequence of length T, the gradient of the loss with respect to the initial hidden state $\boldsymbol{h}_1$ is computed using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_1} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_T} \cdot \prod_{t=2}^{T} \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{h}_{t-1}} \tag{9.9}$$

While BPTT allows RNNs to learn from sequential data, this multiplicative chain introduces a critical flaw known as the **vanishing gradient** problem. The issue can be broken down as follows:

- **The Mechanism:** Early time steps in a sequence receive gradients through a long chain of multiplications. If the sequence is long, these repeated multiplications cause the gradients to shrink exponentially as they propagate backward.
- **The Consequence:** The gradients for early states (like $\boldsymbol{h}_1$) become vanishingly small, which effectively prevents the model from updating its weights and learning long-term dependencies in the data.

- **The Real-World Impact:** This is particularly problematic for tasks that require a deep understanding of long-range context, such as language modeling or machine translation, where the model must capture relationships between words that are far apart in the input sequence.

9.2.3 A partial solution: Truncated BPTT

A common practical solution to the vanishing gradient problem and the high computational cost of standard BPTT is **Truncated Backpropagation Through Time** (TBPTT). The core idea is to approximate the computation by processing the sequence in smaller chunks rather than the entire history at once.

The algorithm can be characterized by two parameters:

- k_1: The number of time steps between gradient updates (the forward pass length).
- k_2: The maximum number of time steps backpropagated (the truncation window).

This approach allows the model to learn from recent history while mitigating the vanishing gradient problem, as we only backpropagate through a limited number of time steps, thus. It also reduces the computational cost, allowing for more efficient training of RNNs on long sequences.

This approach is particularly effective for tasks where long-term dependencies are less critical, as in fact it fails to capture dependencies that span more than k_2 time steps, but it can still be useful for many applications where recent context is more relevant.

9.3 Long Short-Term Memory

To address the limitations of Vanilla RNNs, specifically the **vanishing gradient problem**, **Long Short-Term Memory** (LSTM) networks were introduced. LSTMs utilize a gated architecture designed to maintain a persistent state over long sequences, allowing the model to learn when to forget old information and when to update with new inputs.

9.3.1 The Cell State

The core innovation of the LSTM is the **cell state** (denoted as $\boldsymbol{C}_t$), which acts as a long-term memory conveyor belt. Unlike the hidden state $\boldsymbol{h}_t$, which undergoes a non-linear transformation at every time step, the cell state $\boldsymbol{C}_t$ interacts with the rest of the network primarily through linear operations.

This linear path allows gradients to flow through many time steps with minimal decay, effectively creating a "highway" for gradients during backpropagation, which is crucial for learning long-term dependencies in sequential data.

9.3.2 Gating Mechanisms

LSTMs regulate the cell state using three specialized "gates." Each gate consists of a sigmoid layer (σ) and an element-wise multiplication operation ($\odot$). These gates are learned parameters that determine the flow of information.

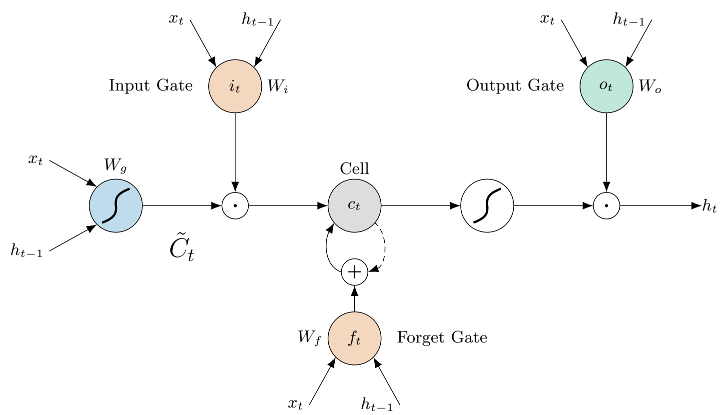


Figure 36: Internal gated architecture of an LSTM cell.

Input Gate ($\boldsymbol{i}_t$): The input gate decides which values in the cell state will be updated. Simultaneously, a *candidate* cell state $\tilde{\boldsymbol{C}}_t$ is created using a tanh layer to represent new information that could be added to the state.

$$\boldsymbol{i}_t = \sigma\left(\boldsymbol{W}_i \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_i\right) \tag{9.10}$$

The input gate takes the previous hidden state $\boldsymbol{h}_{t-1}$ and the current input $\boldsymbol{x}_t$ as input, applies a linear transformation followed by a sigmoid activation function, and outputs a value between 0 and 1 for each element in the cell state, where 0 means "do not update" and 1 means "update".

On the other hand, the candidate cell state $\tilde{\boldsymbol{C}}_t$, similarly to the hidden state in a Vanilla RNN, is computed as a non-linear transformation of the current input and the previous hidden state:

$$\tilde{\boldsymbol{C}}_t = \tanh\left(\boldsymbol{W}_C \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_C\right) \tag{9.11}$$

Forget Gate ($\boldsymbol{f}_t$): This gate decides what information from the previous cell state $\boldsymbol{C}_{t-1}$ is no longer useful and should be discarded.

$$\boldsymbol{f}_t = \sigma\left(\boldsymbol{W}_f \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_f\right) \tag{9.12}$$

It takes the previous hidden state $\boldsymbol{h}_{t-1}$ and the current input $\boldsymbol{x}_t$ as input, applies a linear transformation followed by a sigmoid activation function, and outputs a value between 0 and 1 for each element in the cell state. The output can be interpreted as a soft mask, where 0 means "completely forget," while 1 means "completely retain."

Output Gate ($\boldsymbol{o}_t$): This gate decides what information to output from the cell state.

$$\boldsymbol{o}_t = \sigma\left(\boldsymbol{W}_o \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_o\right) \tag{9.13}$$

It takes the previous hidden state $\boldsymbol{h}_{t-1}$ and the current input $\boldsymbol{x}_t$ as input, applies a linear transformation followed by a sigmoid activation function, and outputs a value between 0 and 1 for each element in the hidden state. The output can be interpreted as a soft mask, where 0 means "completely forget," while 1 means "completely retain."

9.3.3 Memory Cell Update

The new cell state $\boldsymbol{C}_t$ is updated by combining the gated previous state and the gated candidate state. This is the "update" step where the network actually forgets and learns:

$$\boldsymbol{C}_t = \boldsymbol{f}_t \odot \boldsymbol{C}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{C}}_t \tag{9.14}$$

By using the forget gate $\boldsymbol{f}_t$, the network can reset the memory of a specific unit if it is no longer relevant, preventing the internal state from growing indefinitely.

Using the input gate $\boldsymbol{i}_t$ and the candidate cell state $\tilde{\boldsymbol{C}}_t$, the network can add new information to the cell state, allowing it to learn from new inputs while retaining important past information.

9.3.4 Hidden State Generation

Finally, the hidden state $\boldsymbol{h}_t$ is updated. We pass the cell state through a tanh (to push the values between -1 and 1) and multiply it by the output gate $\boldsymbol{o}_t$:

$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{C}_t) \tag{9.15}$$

While $\boldsymbol{C}_t$ stores long-term information, $\boldsymbol{h}_t$ serves as the "working memory" used for the immediate prediction and as an input for the next time step.

9.3.5 Gradients and Backpropagation

The architectural beauty of the LSTM lies in its additive update rule for $\boldsymbol{C}_t$. Even though the gates introduce non-linearities, the cell state $\boldsymbol{C}_t$ itself is updated through a combination of linear operations (element-wise multiplication and addition), which provides a nearly linear path for gradients to flow during backpropagation. This significantly mitigates the vanishing gradient problem inherent in the multiplicative chain of standard RNNs.

9.4 Gated Recurrent Unit

Gated Recurrent Units (GRUs) are a simplified version of LSTMs that tries to reduce the number of parameters and computational complexity while maintaining similar performance. GRUs achieve this by combining the forget and input gates into a single "update gate" and merge the cell state and hidden state into a single state. This results in a more streamlined architecture that is computationally more efficient while still capturing long-term dependencies effectively.

9.4.1 GRU Cell Architecture

In a GRU cell, we have only two gates: the **update gate** ($\boldsymbol{z}_t$) and the **reset gate** ($\boldsymbol{r}_t$).

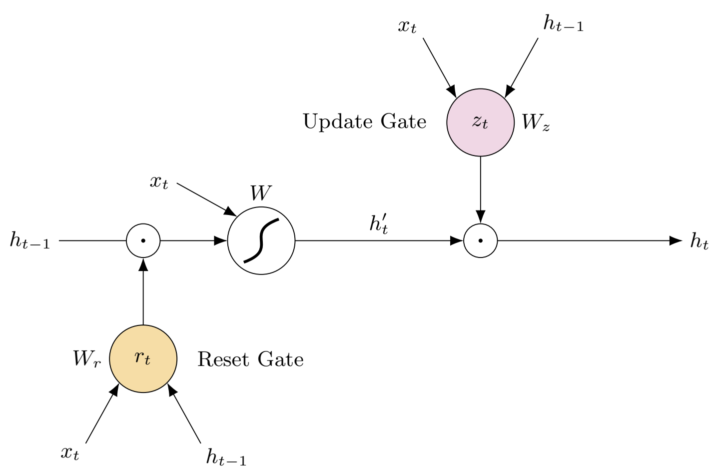


Figure 37: Internal gated architecture of a GRU cell.

Reset Gate ($\boldsymbol{r}_t$): The reset gate determines how much of the previous hidden state $\boldsymbol{h}_{t-1}$ should be used when computing the candidate hidden state $\tilde{\boldsymbol{h}}_t$.

$$\boldsymbol{r}_t = \sigma\left(\boldsymbol{W}_r \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_r\right) \tag{9.16}$$

It takes the previous hidden state $\boldsymbol{h}_{t-1}$ and the current input $\boldsymbol{x}_t$ as input, applies a linear transformation followed by a sigmoid activation function, and outputs a value between 0 and 1 for each element in the hidden state, where 0 means "completely reset" and 1 means "completely retain."

Candidate Hidden State ($\tilde{\boldsymbol{h}}_t$): The candidate hidden state is computed based on the reset gate and the previous hidden state.

$$\tilde{\boldsymbol{h}}_t = \tanh\left(\boldsymbol{W}_h \cdot [\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_h\right) \tag{9.17}$$

The reset gate $\boldsymbol{r}_t$ controls how much of the previous hidden state $\boldsymbol{h}_{t-1}$ is used in the computation of the candidate hidden state $\tilde{\boldsymbol{h}}_t$. If $\boldsymbol{r}_t$ is close to 0, the previous hidden state is effectively ignored, allowing the model to reset its memory when necessary.

Update Gate ($\boldsymbol{z}_t$): The update gate determines how much of the previous hidden state $\boldsymbol{h}_{t-1}$ and the candidate hidden state $\tilde{\boldsymbol{h}}_t$ should be combined to form the new hidden state $\boldsymbol{h}_t$.

$$\boldsymbol{z}_t = \sigma\left(\boldsymbol{W}_z \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_z\right) \tag{9.18}$$

It takes the previous hidden state $\boldsymbol{h}_{t-1}$ and the current input $\boldsymbol{x}_t$ as input, applies a linear transformation followed by a sigmoid activation function, and outputs a value between 0 and 1 for each element in the hidden state, where 0 means "completely update" and 1 means "completely retain."

The new hidden state $\boldsymbol{h}_t$ is then computed as a linear interpolation between the previous hidden state $\boldsymbol{h}_{t-1}$ and the candidate hidden state $\tilde{\boldsymbol{h}}_t$, weighted by the update gate $\boldsymbol{z}_t$:

$$\boldsymbol{h}_t = (1 - \boldsymbol{z}_t) \odot \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \odot \tilde{\boldsymbol{h}}_t \tag{9.19}$$

This allows the GRU to adaptively capture long-term dependencies while maintaining a simpler architecture compared to LSTMs.

LSTM vs GRU The LSTM and GRU architectures both address the vanishing gradient problem and are capable of capturing long-term dependencies in sequential data. However, they differ in complexity and performance characteristics:

- **Complexity:** LSTMs have three gates and a separate cell state, while GRUs have two gates and combine the cell and hidden states. This allows GRUs to have a lower number of parameters.
- **Training Speed:** GRUs are generally faster to train due to their smaller number of parameters and simpler architecture.
- **Performance:** LSTM were shown to be very strong in capturing long-term dependencies, while GRUs are often competitive, but may not perform as well on tasks that require modeling very long sequences.

9.5 Multi-Layer RNNs

To increase the representational capacity of RNNs, we can stack multiple RNN layers on top of each other, creating a **multi-layer RNN** (also known as a deep RNN). In this architecture, the hidden state of each layer serves as the input to the next layer, allowing the model to learn hierarchical representations of the sequential data.

9.5.1 Bi-directional RNNs

Very popular in the speech recognition community are the Bi-directional RNNS. They allow processing input sequences in both forward and backward directions, enabling the model to capture context from both past and future time steps.

Altough bi-directional RNNs are powerful, they are not suitable for real-time applications, as they require access to the entire input sequence before making predictions.

10 Sequence Models

In the preceding chapter, we explored how recurrent architecture, specifically LSTM and GRU cells, were developed to alleviate the vanishing gradient problem and better capture long-term dependencies in sequential data. Despite these advancements, RNN-based models still encounter significant challenges as input sequences grow in length.

This limitation is particularly prominent in the **encoder-decoder** (or sequence-to-sequence) framework. In this setup, an encoder RNN compresses the entirety of the input sequence into a single, fixed-length context vector, which the decoder RNN then utilizes to generate the output. Forcing the full semantic meaning of a variable-length input into a solitary vector creates a severe **information bottleneck**: as the sequence length increases, it becomes exceedingly difficult for this vector to retain all relevant details.

To resolve this bottleneck, the **attention mechanism** was introduced well before the advent of the Transformer architecture. Rather than relying on a static summary vector, attention enables the decoder to access the complete sequence of encoder hidden states. This allows the model to dynamically weigh and focus on the most pertinent information at each step of the generation process.

In this chapter, we examine sequence models that retain the RNN as their foundational building block but augment it with attention. We explore this synthesis through two representative tasks: **image captioning** and **neural machine translation**. In the subsequent chapter, we will discuss how the attention mechanism ultimately rendered recurrence obsolete, paving the way for the Transformer architecture.

10.1 Image Captioning

Image captioning is the task of generating a textual description for a given image. This task combines **computer vision** for the image understanding and **natural language processing** for generating the caption. The main challenge in image captioning is to effectively capture the relevant features of the image and generate a coherent and contextually appropriate caption.

The usual approach to image captioning involves using a CNN to extract features from the image, and then using an RNN to generate the caption based on those features.

10.1.1 No Attention - Show and Tell

Show and Tell is one of the early models for image captioning. Following the encoder-decoder architecture proposed for machine translation, Show and Tell uses an architecture combining two main components:

- **Encoder**: A CNN (such as Inception or ResNet) is used to extract a fixed-length feature vector from the input image. This feature vector captures the high-level visual information of the image.
- **Decoder**: An RNN (such as LSTM) is used to generate the caption word by word. The RNN takes the feature vector from the encoder as input and generates the caption sequentially, predicting one word at a time until it generates an end token.

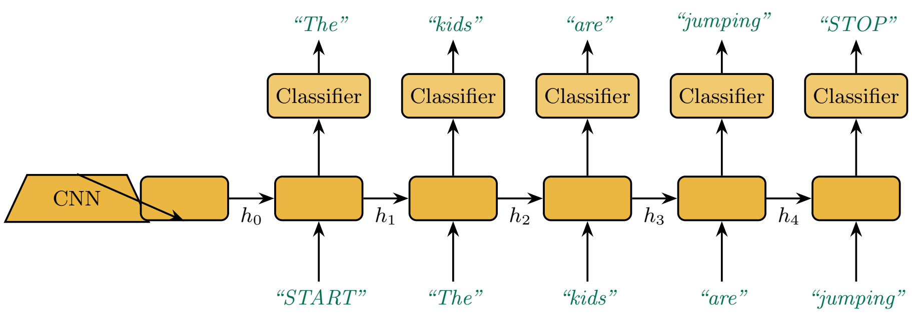


Figure 38: Show and Tell: a CNN encodes the image into a feature vector that initializes the RNN decoder, which generates the caption word by word.

At the first time step, the RNN receives as input the **feature vector** from the encoder and a special start token. At each subsequent time step t, the RNN receives as input the word generated at time step $t-1$ and the hidden state from the previous time step. The RNN continues to generate words until it produces an end token, indicating the end of the caption.

Limitations Despite its success, this model faced several inherent limitations:

- **Information Bottleneck**: The fixed-length feature vector may not capture all relevant details, especially in complex images with multiple interacting objects.
- **Vanishing Visual Context**: Because the RNN only "sees" the image at the first time step, the visual influence can weaken as the sentence grows, leading the model to rely more on language probability than the actual image.
- **Static Features**: The feature vector is global and static; the model cannot "shift its gaze" to different parts of the image as it describes different objects.

10.1.2 Inference

At inference time, two different strategies can be used to generate captions:

- **Sampling**: At each time step, the model samples a word from the predicted probability distribution. The corresponding word is then fed as input to the next time step. This process continues until an end token is generated or a maximum length is reached. While this can lead to more diverse and creative captions, it also increases the risk of generating grammatically incorrect or nonsensical sequences.
- **Beam Search**: Instead of sampling, beam search keeps track of the top k most likely sequences at each time step, where k is the beam width. At each step, the model expands each of the k sequences by one word and keeps only the top k resulting sequences based on their cumulative probabilities. Beam search is more computationally expensive than sampling but often results in more coherent and higher-quality captions.

10.1.3 Evaluation Metrics

In order to evaluate the performance of image captioning models, we must be able to compare the generated captions with human-generated reference captions. To address this, the BLEU (Bilingual Evaluation Understudy) metric was introduced.

BLEU is a precision-based metric that measures the overlap between the generated caption and one or more reference captions. It calculates the **n-gram precision**, which is the proportion of n-grams (contiguous sequences of n words) in the generated caption that are also present in the reference captions.

10.1.4 Attention: Show, Attend and Tell

To overcome the limitations of the baseline *Show and Tell* architecture, the *Show, Attend and Tell* model introduces an **attention** mechanism. Rather than compressing the entire image into a single fixed-length vector, attention enables the model to dynamically focus on specific spatial regions of the image as it generates each word of the caption.

For example, consider an image of a dog playing with a ball in a park. When predicting the word "dog," relying on the global image context is suboptimal due to irrelevant background noise. Instead, the attention mechanism allows the network to isolate and "attend" specifically to the pixels containing the dog. By aligning local visual features with the exact word being generated at each time step, this dynamic focusing yields more accurate and context-aware captions.

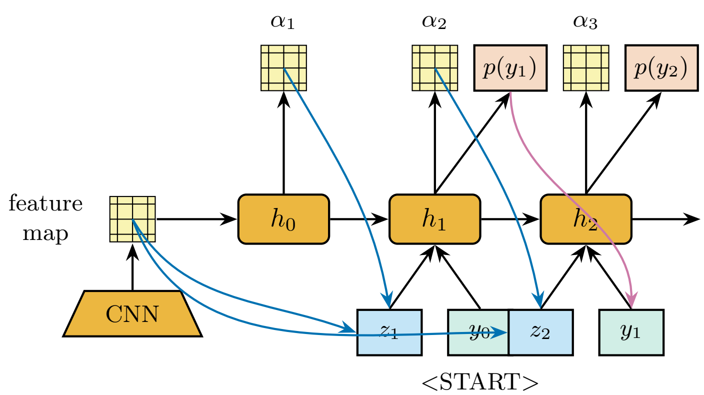


Figure 39: Show, Attend and Tell: at each step the decoder produces an attention map α_t over the CNN feature map, forming a context vector $\boldsymbol{z}_t$ that, together with the previous word, drives the prediction $p(y_t)$.

The architecture of *Show, Attend and Tell* consists of three main components.

Encoder Similar to *Show and Tell*, a CNN is used for feature extraction. However, to preserve spatial information, the encoder avoids the fully connected layers that would otherwise collapse the spatial dimensions. Instead, features are extracted directly from the final convolutional layer, producing a set of spatial feature vectors $\{\boldsymbol{a}_1, \boldsymbol{a}_2, \ldots, \boldsymbol{a}_L\}$. Here, L represents the number of regions in the image (e.g., $14 \times 14 = 196$), and each $\boldsymbol{a}_i$ corresponds to the features of a specific region.

Attention Mechanism At each time step t, an attention network f_{att} determines where the model should focus. To do this, it first computes a raw relevance score for every region i, conditioned on the decoder's previous hidden state $\boldsymbol{h}_{t-1}$ and the corresponding feature vector $\boldsymbol{a}_i$:

$$e_{t,i} = f_{att}(\boldsymbol{a}_i, \boldsymbol{h}_{t-1})$$

where f_{att} is typically a small MLP. These scores are then normalized across all L regions with a softmax, yielding the **attention weights**:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{k=1}^{L} \exp(e_{t,k})}$$

A **context vector** $\boldsymbol{z}_t$ is then computed as a function of the attention weights $\alpha_{t,i}$ and the corresponding feature vectors $\boldsymbol{a}_i$. In the deterministic (soft attention) case, this function is simply a weighted sum:

$$\boldsymbol{z}_t = \sum_{i=1}^{L} \alpha_{t,i}\, \boldsymbol{a}_i$$

though it could also be a more complex non-linear transformation. This context vector dynamically captures the specific visual information relevant for generating the current word.

Decoder An LSTM generates the caption sequentially. At time step t, the decoder receives the dynamic context vector $\boldsymbol{z}_t$, the previous hidden state $\boldsymbol{h}_{t-1}$, and the previously generated word y_{t-1}. The forward pass of the LSTM updates the hidden state and produces a probability distribution over the vocabulary for the next word y_t. By conditioning the generation on $\boldsymbol{z}_t$, the decoder ensures that each word is grounded in the most relevant visual features of the image. Since $\boldsymbol{h}_t$ is in turn used to compute the attention scores at the next time step, the region of the image the model attends to naturally shifts as the caption is generated.

10.2 Attention Types

Attention mechanisms are generally categorized into two primary paradigms, distinguished by how they compute the context vector from the input features:

- **Soft Attention**: This is a deterministic and fully differentiable mechanism. It computes a continuous, weighted average of all input features (e.g., encoder hidden states) based on their respective attention weights. Because the entire operation is continuous and differentiable, the model can be trained end-to-end using standard backpropagation.
- **Hard Attention**: This is a stochastic and non-differentiable mechanism. Instead of a weighted average, it makes a discrete choice to focus on a single element (or a specific subset) of the input at each time step, effectively applying a binary mask over the features. Since this discrete selection process breaks differentiability, the model cannot be trained via standard backpropagation; instead, it relies on reinforcement learning techniques (such as the REINFORCE algorithm) to estimate gradients by treating the attention selection as a stochastic policy.

To illustrate this distinction within the context of image captioning: *soft attention* evaluates the entire image at every step, creating a blended representation by weighting all spatial regions, whereas *hard attention* makes a discrete decision to "crop" and look exclusively at one specific region of the image, ignoring the rest entirely.

10.3 Machine Translation

We will now try to explore the attention mechanism in the context of machine translation, which is a more complex task than image captioning. Again, we will first introduce the baseline model without attention, and then we will analyze how attention can be incorporated to improve performance.

10.3.1 No Attention - Seq2Seq

The **sequence-to-sequence** (Seq2Seq) model is a foundational architecture for machine translation. It consists of two main components:

- **Encoder**: An RNN (such as LSTM or GRU) processes the input sequence, mapping it to a fixed-length context vector;
- **Decoder**: Another RNN generates the output sequence based on the context vector produced by the encoder.

This architecture first showed that deep neural networks could be effectively applied to end-to-end translation tasks, achieving competitive performance with traditional statistical machine translation methods.

Problems Again, the Seq2Seq model suffers from the same information bottleneck as the image captioning model. In fact, it assumes that the last hidden state of the encoder can capture all the semantic information of the input sequence. This is particularly challenging, in particular for long sentences, since the representation of the input sequence is limited to a single vector.

Minor Improvements: RNN Encoder-Decoder An early refinement to the standard Seq2Seq architecture introduced a distinct **context vector**. Rather than directly passing the encoder's final hidden state to the decoder, the context vector is explicitly computed as a function of that hidden state.

Although this added flexibility yielded minor performance gains, it failed to resolve the fundamental information bottleneck. Because the context vector remains a fixed-length representation, the model inherently struggles to compress and retain all relevant information from longer input sequences.

10.3.2 RNN with Attention

Unlike earlier models that rely on a single, fixed context vector to summarize the entire input, the attention mechanism introduces a **dynamic context vector** computed at each decoder time step. This allows the network to adaptively "attend" to the most relevant parts of the input sequence as it generates each target word.

This architecture typically employs a Bi-directional RNN as the encoder. The forward and backward hidden states are concatenated to form a single, context-rich representation for each time step of the input sequence: $\boldsymbol{h}_i = [\overrightarrow{\boldsymbol{h}}_i; \overleftarrow{\boldsymbol{h}}_i]$.

During the decoding step t, the model calculates an attention weight $\alpha_{t,i}$ for each encoder hidden state $\boldsymbol{h}_i$. These weights are derived by scoring the alignment between the decoder's previous hidden state $\boldsymbol{s}_{t-1}$ and each encoder state $\boldsymbol{h}_i$, through an attention network f_{att}, the **alignment model**, which is trained jointly with the rest of the architecture:

$$e_{t,i} = f_{att}(\boldsymbol{s}_{t-1}, \boldsymbol{h}_i)$$

Note that $\boldsymbol{s}_{t-1}$ is used here out of necessity rather than choice: in this architecture the context vector $\boldsymbol{c}_t$ is itself fed into the recurrence that produces $\boldsymbol{s}_t$, so at the point where the attention scores are computed, $\boldsymbol{s}_t$ does not exist yet, only $\boldsymbol{s}_{t-1}$ is available. This is in contrast with the local attention architecture presented later, where the decoder state is computed *before* attention and is therefore free to use $\boldsymbol{s}_t$ directly.

These scores are then normalized across all T_x encoder states with a softmax, yielding the attention weights:

$$\alpha_{t,i} = \text{softmax}(e_{t,i}) = \frac{\exp(e_{t,i})}{\sum_{k=1}^{T_x} \exp(e_{t,k})}$$

The dynamic context vector $\boldsymbol{c}_t$ is then constructed as the weighted sum of all encoder hidden states, scaled by their respective attention weights:

$$\boldsymbol{c}_t = \sum_{i=1}^{T_x} \alpha_{t,i} \boldsymbol{h}_i$$

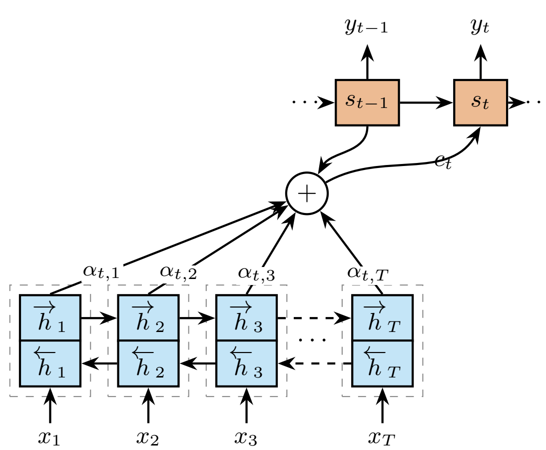


Figure 40: RNN with attention: a bidirectional encoder produces hidden states $\boldsymbol{h}_j$; alignment weights $\alpha_{t,j}$ combine them into the context vector $\boldsymbol{c}_t$ that feeds the decoder state $\boldsymbol{s}_t$ generating y_t.

Attention is Alignment When talking about machine translation, the attention mechanism can be interpreted as a form of **soft alignment** between the source and target sequences. The attention weights $\alpha_{t,i}$ can be viewed as a probability distribution over the source tokens, indicating how much each source token contributes to the generation of the target token at time step t.

10.3.3 Local Attention

Even though the attention mechanism we discussed alleviates the information bottleneck to some extent, it still suffers from the limitation of being a **global attention** mechanism. In global attention, the decoder computes attention weights over all encoder hidden states at every time step, making it computationally expensive for long sequences.

Local attention addresses this by restricting the decoder to attend only to a small window of encoder states, rather than the whole sequence. The idea is to design a mechanism that behaves similarly to *hard attention* (focusing on a limited region) while remaining fully **differentiable**, and therefore trainable via standard backpropagation.

The core of the approach is, for each target time step t, to predict an **aligned position** p_t in the source sequence: an estimate of which input region is most relevant for generating the current output word. The context vector is then computed only from the encoder states inside a fixed-size window $[p_t - D, p_t + D]$ centered on p_t, where D is a hyperparameter.

A two-stage hidden state Unlike the global attention architecture above, here the context vector is *not* an input to the decoder recurrence. Instead, the decoder first computes a preliminary hidden state $\boldsymbol{s}_t$ with a standard RNN step, using only $\boldsymbol{s}_{t-1}$ and the previous output y_{t-1}. Since this $\boldsymbol{s}_t$ is already available, it can be used directly as the query for the rest of the mechanism: first to predict the aligned position p_t, and then to score the encoder states inside the resulting window, as described next. Only once the context vector has been formed from this window is $\boldsymbol{s}_t$ corrected into its final form.

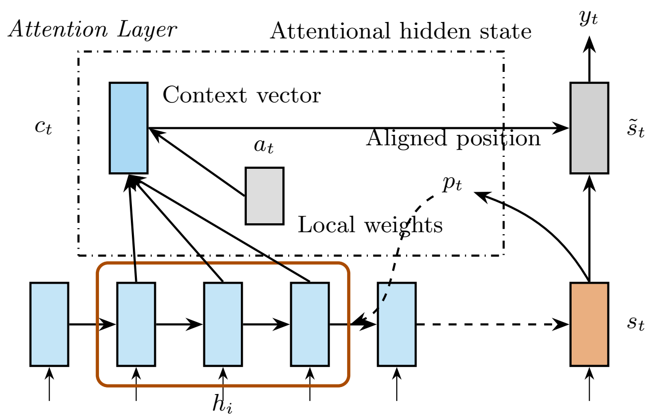


Figure 41: Local attention with predictive alignment. The decoder state $\boldsymbol{s}_t$ predicts the aligned position p_t, which centers a fixed window over the encoder states $\boldsymbol{h}_i$. The context vector $\boldsymbol{c}_t$ is formed from the window only and combined with $\boldsymbol{s}_t$ into the attentional hidden state $\tilde{\boldsymbol{s}}_t$ used to emit y_t.

In the **predictive alignment** variant, p_t is computed from the decoder hidden state $\boldsymbol{s}_t$ through a small learned network:

$$p_t = T_x \cdot \sigma\big(\boldsymbol{v}_p^\top \tanh(\boldsymbol{W}_p \boldsymbol{s}_t)\big)$$

where $\boldsymbol{W}_p$ and $\boldsymbol{v}_p$ are learnable parameters. Since $\sigma(\cdot) \in (0, 1)$, multiplying by the source length T_x rescales the output into a valid position within the input sequence.

The window alignment is enforced by weighting the standard attention scores with a **Gaussian** centered on p_t:

$$\alpha_{t,i} = \text{softmax}(\boldsymbol{h}_i^\top \boldsymbol{W}_a \boldsymbol{s}_t) \cdot \exp\left(-\frac{(i - p_t)^2}{2\sigma^2}\right)$$

where $\boldsymbol{W}_a$ is a learnable scoring matrix and the Gaussian standard deviation is typically set to $\sigma = D/2$. The Gaussian term damps the contribution of encoder states far from p_t toward zero, effectively concentrating attention around the predicted position.

Why is computing p_t tricky? The natural idea, predicting a position and then directly selecting the encoder state at that index, is **not differentiable**, since indexing into a discrete position breaks the gradient flow (this is exactly the issue that hard attention faces). The Gaussian weighting is what resolves this difficulty: because p_t is a real-valued quantity that appears *inside* the smooth exponential term, the alignment remains differentiable with respect to p_t, and therefore with respect to $\boldsymbol{W}_p$ and $\boldsymbol{v}_p$. This allows the model to learn where to look through gradient descent, recovering the focused behavior of hard attention without sacrificing trainability.

Correcting the proposal Once the context vector $\boldsymbol{c}_t$ has been formed, it is used to *refine* the preliminary state $\boldsymbol{s}_t$: the two are concatenated and passed through a tanh layer to produce the **attentional hidden state**

$$\tilde{\boldsymbol{s}}_t = \tanh(\boldsymbol{W}_c[\boldsymbol{c}_t; \boldsymbol{s}_t])$$

which is what is actually used to emit y_t. In other words, $\boldsymbol{s}_t$ is a first proposal for the decoder state, computed with no knowledge of the source alignment, and $\tilde{\boldsymbol{s}}_t$ is the corrected version that incorporates it.

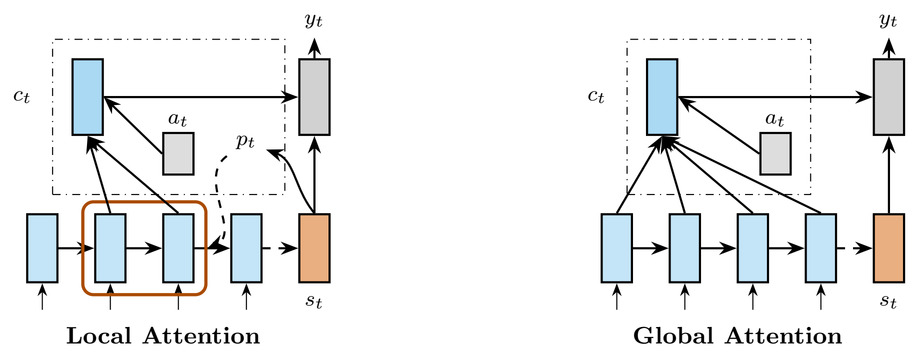


Figure 42: Local vs global attention. Global attention scores all encoder states $\boldsymbol{h}_i$ at every step, whereas local attention restricts the context to a window centered on the predicted position p_t.

11 Attention & Transformers

Early sequence processing models relied primarily on RNN encoder-decoder architectures. In these models, the encoder compresses the input sequence into a fixed-size representation, which the decoder then uses to generate the output. However, this fixed bottleneck limits performance. The introduction of attention mechanisms demonstrated that creating a dynamic, context-aware representation of the input sequence significantly improves results.

Building on the success of attention, researchers sought to eliminate the sequential bottleneck of RNNs entirely. This led to the development of more flexible architectures capable of processing sequences in parallel while still effectively capturing long-range dependencies.

11.1 Convolutional Sequence Models

CNNs offer a natural alternative for sequence processing. By using convolutional filters, they efficiently capture local dependencies, and stacking these layers allows the model to expand its receptive field to capture longer-range relationships.

CNNs provide two primary advantages over RNNs:

- **Parallelization:** Unlike RNNs, which must process tokens sequentially, CNNs process the entire sequence hierarchically. This enables massive parallelization and significantly faster training times.
- **Parameter Efficiency:** The parameter-sharing nature of convolutional filters makes CNNs highly efficient, reducing both the total number of parameters and overall computational cost.

Despite these strengths, traditional CNNs are constrained by a fixed input size. This limitation complicates their application to variable-length sequences, often requiring workarounds that make them less flexible in certain scenarios.

Dilated Convolutions To capture long-range dependencies without inflating the parameter count, CNNs frequently employ **dilated convolutions**. Unlike standard convolutions, a dilated filter is applied over a broader area by introducing gaps between the sampled input values. The size of these intervals is determined by the *dilation rate*. This technique effectively expands the network's receptive field while preserving both parameter efficiency and computational tractability.

For instance, consider a standard 3×3 convolutional filter applied to adjacent pixels. This configuration corresponds to a dilation rate of 1, yielding a 3×3

receptive field. If the dilation rate is increased to 2, the filter skips one pixel between each consecutive sample. Consequently, the filter's effective receptive field expands to a 5×5 area, despite still relying on the same 9 trainable parameters.

11.1.1 Temporal Convolutional Networks

The **Temporal Convolutional Network (TCN)** is a prominent architecture that applies dilated convolutions directly to sequence modeling. By addressing the historical limitations of standard CNNs, TCNs demonstrated that convolutional architectures can be highly competitive with RNNs, achieving comparable or superior performance in various sequence tasks.

TCNs achieve this through two core architectural mechanisms:

- **Hierarchical Dilated Convolutions:** To capture long-range dependencies, TCNs utilize a stack of convolutional layers where the dilation rate increases with depth. The initial layer processes the input with a small receptive field, while subsequent layers progressively expand this field. This hierarchical approach allows the network to model relationships across extended time spans while maintaining parameter efficiency.
- **Causal Convolutions:** To ensure strict adherence to temporal ordering, TCNs employ causal convolutions. This guarantees that the network's prediction at time step t is computed using only inputs from time step t and earlier, fundamentally preventing any information leakage from future states.

11.2 Transformers

Instead of using attention as a supplementary mechanism within RNNs or CNNs, the **Transformer** architecture relies entirely on attention to model dependencies between input and output sequences. Transformers are neural network architectures based on the *self-attention* mechanism. This architecture has been shown to be more efficient and parallelizable than RNNs and CNNs, leading to faster training and strong performance on a wide range of sequence modeling tasks.

11.2.1 The Architecture

The architecture of the Transformer still follows the encoder-decoder paradigm, but it entirely rethinks the way each component is implemented.

- **Encoder:** The encoder receives the entire input sequence and processes it in parallel, producing a sequence of contextualized representations, one for each input token.
- **Decoder:** The decoder predicts the output sequence one token at a time in an autoregressive manner, using previously generated tokens and the output of the final encoder layer as context.

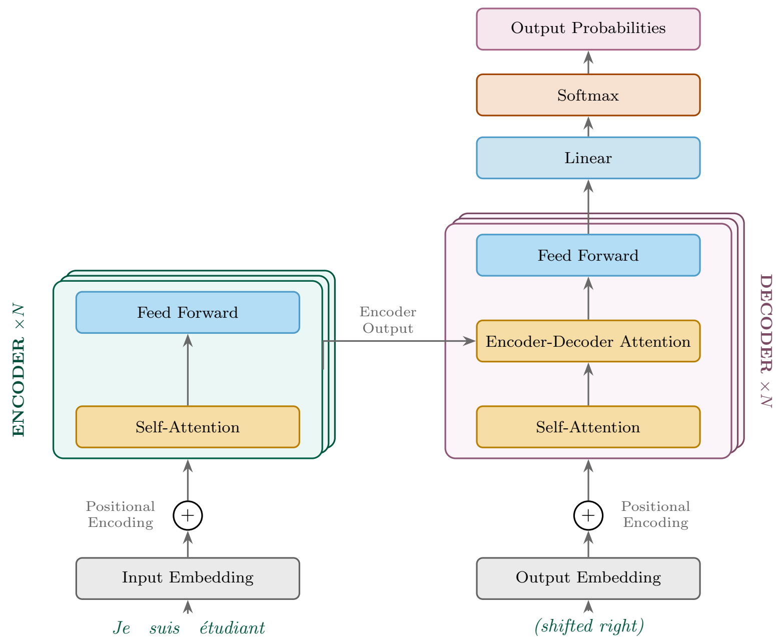


Figure 43: The Transformer architecture. A stack of N encoders maps the input sequence into contextualized representations, which feed the encoder-decoder attention of a stack of N decoders. The decoder generates the output sequence one token at a time.

Let's now go deeper into the details of each component of the Transformer architecture.

11.2.2 Encoder Layer

The encoder maps the input tokens into a sequence of **contextualized representations** by stacking N identical encoder layers, each one refining the representations produced by the previous one. The information flows as follows:

1. **Embedding and Positional Encoding:** The full input sequence is embedded in parallel: each token is first mapped to a high-dimensional vec-

tor through an **embedding layer**, which captures its semantic meaning. Since these embeddings carry no information about the order of the tokens, a **positional encoding** vector is added to each one to inject this information (discussed in detail below).

2. **Self-Attention:** The **self-attention** mechanism is applied, allowing each token to capture dependencies with the other tokens in the sequence that are relevant to its own representation.

3. **Feed-Forward:** A **feed-forward layer** then processes the output of the self-attention mechanism, applying a non-linear transformation to each token's representation independently. Since self-attention has already mixed information across tokens, this layer refines each token's representation without further token interaction, enabling efficient parallel computation.

Steps 2 and 3 are repeated at every one of the N stacked layers, each layer consuming the previous layer's output. The output of the final encoder layer is the encoder's contextualized representation of the input sequence, which is passed as the Keys and Values of the encoder-decoder attention in every decoder layer.

11.2.3 Positional Encoding

The first encoder layer receives a sequence of token embeddings, but these embeddings do not inherently contain any information about the order of the tokens. To address this, positional information is provided through an additive **positional encoding** vector, which shares the same dimensionality as the token embeddings.

With positional encoding, the representation of words is influenced not only by their semantic similarity but also by their relative positions in the sequence.

This is usually implemented using combinations of sine and cosine functions of different frequencies, which allow the model to learn relative positions and generalize to sequences longer than those seen during training.

11.2.4 Self-Attention

The self-attention mechanism is the core of the Transformer architecture. It allows the creation of arbitrarily **long-range dependencies** between tokens in a sequence, regardless of their distance from each other. It allows each token in the input sequence to attend to every other token considered relevant for the current token's representation.

How to compute it From their embeddings, the **self-attention** mechanism creates three vectors for each token:

- **Query ($\boldsymbol{q}$):** What the token is looking for in the other tokens.
- **Key ($\boldsymbol{k}$):** What the token has to offer to the other tokens.
- **Value ($\boldsymbol{v}$):** The actual information that the token carries, which will be used to compute the final representation.

The core idea is that when the query of a token matches the key of another token (yielding a high attention score), the value of that token will heavily influence the final representation of the querying token.

The $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ matrices are computed as linear transformations of the input embeddings $\boldsymbol{X}$, using learned weight matrices:

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q, \quad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K, \quad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V$$

Score Computation Once the $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ matrices are computed, the next step is to calculate the attention scores. Given a query vector $\boldsymbol{q}_i$ for token i and key vectors $\boldsymbol{k}_j$ for all tokens j, the attention score e_{ij} is computed as the dot product of the query and key vectors:

$$e_{ij} = \boldsymbol{q}_i \cdot \boldsymbol{k}_j$$

This score indicates how much attention token i should pay to token j.

These scores are then scaled to ensure that the dot products do not grow too large, which can push the softmax function into regions with extremely small gradients, leading to instability during training. This is done by dividing the scores by the square root of the dimension of the key vectors, d_k:

$$\hat{e}_{ij} = \frac{e_{ij}}{\sqrt{d_k}}$$

Finally, a softmax function is applied to convert the scores into a valid probability distribution:

$$\alpha_{ij} = \text{softmax}(\hat{e}_{ij}) = \frac{\exp(\hat{e}_{ij})}{\sum_k \exp(\hat{e}_{ik})}$$

Output Computation The final step in the self-attention mechanism is to compute a weighted sum of the **value** vectors based on the attention probabilities.

$$\hat{\boldsymbol{v}}_i = \sum_j \alpha_{ij} \boldsymbol{v}_j \tag{11.1}$$

This output vector $\hat{\boldsymbol{v}}_i$ represents the refined representation of token i after considering the context provided by all other tokens in the sequence, giving more importance to tokens that are more relevant according to the attention scores.

Putting Everything Together Putting all the steps together, the self-attention computation for the entire sequence can be expressed cleanly in matrix form as:

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}}\right)\boldsymbol{V}$$

Multi-Head Attention An evolution of the standard self-attention mechanism is **multi-head attention**, which allows the model to focus on different aspects of the input sequence simultaneously. Instead of computing a single set of $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ matrices, the model computes multiple sets (or "**heads**") of these matrices, each with its own learned weight matrices.

This provides the attention layer with multiple representation subspaces, allowing it to capture a richer set of relationships between tokens (e.g., one head might focus on grammatical syntax, while another focuses on semantic relationships).

The final output of the multi-head attention mechanism is obtained by concatenating the outputs of all individual heads and applying a learned linear transformation to combine them back into a single representation.

Although demonstrated to be highly effective, multi-head attention does increase the number of parameters and the computational cost. Furthermore, some heads may end up learning redundant information, which can reduce the overall theoretical efficiency of the model. Currently, we lack a definitive way to enforce diversity among the heads or prune redundant ones *a priori*.

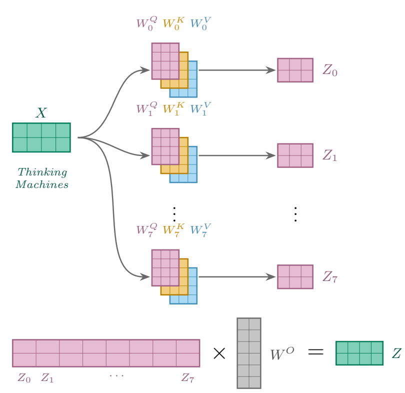


Figure 44: Multi-head attention. The input $\boldsymbol{X}$ is projected by h independent sets of weight matrices $\boldsymbol{W}_i^Q, \boldsymbol{W}_i^K, \boldsymbol{W}_i^V$, producing one attention output $\boldsymbol{Z}_i$ per head. The heads are concatenated and projected by $\boldsymbol{W}^O$ to obtain the final output $\boldsymbol{Z}$.

11.2.5 Layer Normalization and Residuals

Between each sub-layer (e.g., around self-attention and feed-forward networks), the Transformer employs a **residual connection** followed by **layer normalization**. The residual connection allows the model to bypass transformations and learn identity mappings if necessary, which severely mitigates the vanishing gradient problem in deep networks. Layer normalization stabilizes the learning process by normalizing the inputs across the features, ensuring a better gradient flow and faster convergence.

11.2.6 Decoder-Specific Attention Mechanisms

While the decoder shares many architectural similarities with the encoder, it introduces two distinct attention variations to support autoregressive generation:

Masked Self-Attention In the decoder's self-attention layer, a **masking mechanism** is applied. To preserve the causal, autoregressive nature of sequence generation, the model is prevented from "looking ahead." The mask ensures that the prediction for token i can only depend on known outputs at positions less than i, blinding the decoder to future tokens during parallelized training.

Encoder-Decoder Attention Following the masked self-attention layer, the decoder features an **encoder-decoder attention** mechanism (also known as cross-attention). Here, the **Queries** come from the previous decoder layer, while the **Keys** and **Values** come directly from the output of the **final encoder layer**, shared across every decoder layer. This allows every position in the decoder to attend over all positions in the input sequence, effectively bridging the gap between the source material and the generated output.

11.3 Language Pretraining

Similarly to what we have seen in the CNN section, the Transformer architecture could be trained for tasks in NLP. This started a new era in NLP, where large-scale unsupervised pretraining on massive text corpora became the standard approach for building state-of-the-art models. These pretrained models could then be fine-tuned on specific downstream tasks, with some supervised data, achieving remarkable performance across a wide range of NLP benchmarks.

Some of the most notable Transformer-based pretrained models include:

- **BERT (Bidirectional Encoder Representations from Transformers):** an encoder-only model that captures bidirectional context, excelling in tasks like question answering and sentiment analysis.

- **GPT (Generative Pre-trained Transformer):** a decoder-only model that generates coherent and contextually relevant text, widely used for text generation and completion tasks.

11.4 Vision Transformers

Transformers have also been successfully applied to computer vision tasks, leading to the development of **ViTs**. Vision Transformers emerged after the success of Transformers in NLP, since one of the main challenges in applying Transformers to computer vision was how to represent images as sequences of tokens.

11.4.1 ViTs

In Vision Transformers, an image is divided into a grid of non-overlapping patches. These patches are flattened and linearly projected into embedding vectors, which serve as the input tokens of the Transformer. Positional embeddings are added to encode the spatial arrangement of the patches, enabling the model to capture their relative positions within the image.

For image classification, a learnable `[CLS]` token is prepended to the sequence before it is processed by a standard Transformer encoder, which produces contextualized representations for all input tokens. It is randomly initialized and trained to aggregate information from the entire sequence, effectively serving as a summary representation of the input image. The final representation of the `[CLS]` token is then passed through an MLP head to produce the classification output.

Results from Vision Transformers have shown that they can achieve competitive performance with convolutional neural networks, especially when trained on larger datasets.

11.4.2 Data-efficient Image Transformer

The **Data-efficient Image Transformer (DeiT)** is a variant of the Vision Transformer designed to improve data efficiency during training. DeiT introduces a learnable **distillation token**, which is prepended to the input sequence and trained to match the predictions of a pretrained teacher model, typically a CNN. This knowledge distillation strategy enables the Vision Transformer to learn more effectively from limited training data, achieving competitive performance without requiring extremely large training datasets.

11.4.3 Swin Transformer

Standard ViT architectures had a few limitations, such as:

- **Lack of Hierarchical Structure:** ViTs process the entire image as a flat sequence of patches, which can limit their ability to capture multi-scale features and hierarchical representations.
- **Quadratic Complexity:** The global self-attention mechanism in ViTs has a quadratic complexity with respect to the number of patches, making it computationally expensive for high-resolution images.
- **Classification Only:** ViTs were primarily designed for image classification tasks, and adapting them to other vision tasks (e.g., object detection, segmentation) required additional modifications.

Swin Transformers address these limitations by introducing a hierarchical architecture that processes images at multiple scales. The input image is initially divided into non-overlapping patches, while subsequent stages progressively merge neighboring patches to reduce the spatial resolution and increase the feature dimension. Within each stage, self-attention is computed over non-overlapping local windows. To enable information exchange across neighboring windows, the window partitioning is shifted between consecutive Transformer blocks, allowing cross-window connections while maintaining computational efficiency.

This hierarchical architecture makes Swin Transformers suitable as a general-purpose backbone for a wide range of vision tasks, including image classification, object detection, and semantic segmentation.

12 GANs

Before going into the details of GANs, we will first introduce the concept of generative models and their applications.

12.1 Generative Models

What we have seen so far focused only on supervised learning tasks, where the goal was to learn a mapping from an input to an output. In this chapter, we will focus on unsupervised learning generative models, which are a class of machine learning models that aim to generate new data samples that are similar to the training data.

In general, generative models can be divided into two main categories:

- **Non-Probabilistic Models:** These models do not explicitly model the underlying probability distribution of the data. Instead, they learn a mapping from a latent space to the data space. **Autoencoders** are a common example of non-probabilistic generative models.
- **Probabilistic Models:** These models explicitly model the underlying probability distribution of the data. The idea is to use a Neural Network to learn a mapping between a basic probability distribution (e.g., a Gaussian distribution) and a more complex distribution p_{model}. The model is trained to make the distribution as close as possible to the true data distribution p_{data}. **Generative Adversarial Networks (GANs)**, **Variational Autoencoders (VAEs)**, and **Diffusion Models** are examples of probabilistic generative models.

Generative models have a wide range of applications, including image generation, style transfer, and data augmentation.

Probabilistic Models can be further divided into two main categories:

- **Explicit Density Estimation:** These models explicitly define $p_{model}(x)$ and solve for it. The main example is the **VAE**.
- **Implicit Density Estimation:** These models learn to sample from $p_{model}(x)$ without ever explicitly defining it. The main example is the **GAN**, which does not consider any explicit density function and instead focuses solely on the model's ability to sample.

12.2 GAN

GANs are a family of Implicit Density Estimation models. They do not consider any explicit probability distribution, but instead focus on generating samples that are indistinguishable from real data.

GANs are actually a union of two neural networks, the **generator** and the **discriminator**, which are trained simultaneously in an adversarial manner. More specifically:

- **Generator:** The generator network takes a random noise vector as input and generates a sample that resembles the training data. The goal of the generator is to produce samples that are indistinguishable from real data.

  Because it is difficult to sample directly from a complex training distribution p_{data}, the generator first samples from a simple distribution p_z (such as a standard normal distribution). It then learns to transform these simple samples into samples that mirror the training data distribution through a Neural Network.

- **Discriminator:** The discriminator network takes a sample as input (either real or fake) and outputs a **probability that the sample is real** (from the training data) rather than fake (generated by the generator), i.e., $1 \leftrightarrow$ real, $0 \leftrightarrow$ fake.

  The discriminator acts both as a classifier and, implicitly, as a loss function for the generator network, with the goal of correctly classifying samples as real or fake.

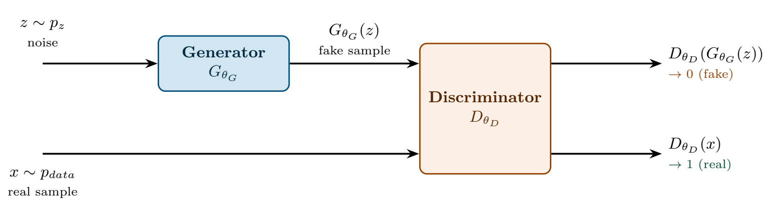


Figure 45: The GAN architecture: the generator G_{θ_G} maps a noise vector $z \sim p_z$ into a fake sample $G_{\theta_G}(z)$. The discriminator D_{θ_D} receives both real samples $x \sim p_{data}$ and fake samples $G_{\theta_G}(z)$, and outputs the estimated probability that each input is real.

12.3 Adversarial Training and Objective Function

The network is trained in an **adversarial manner**, where the generator and discriminator are trained simultaneously. If the discriminator gets better at distinguishing real from fake samples, the generator must improve to produce more realistic samples. This implies that GANs do not consider any explicit probability distribution, but instead focus on generating samples that are indistinguishable from real data.

12.3.1 The Value Function

To formalize this adversarial dynamic, we define a **value function** $V(D, G)$ that measures the performance of the generator and discriminator networks.

$$V(D,G) = \mathbb{E}_{x\sim p_{data}}[\underbrace{\log D_{\theta_D}(x)}_{\text{real term}}] + \mathbb{E}_{z\sim p_z}[\underbrace{\log(1 - D_{\theta_D}(G_{\theta_G}(z)))}_{\text{fake term}}] \tag{12.1}$$

Since $D_{\theta_D}(\cdot) \in [0,1]$, both logarithms are non-positive, so $V(D,G) \leq 0$. Each term behaves as follows:

- **Real term:** $\log D_{\theta_D}(x)$ reaches its maximum value of 0 when $D_{\theta_D}(x) \to 1$, i.e., the discriminator correctly recognizes a real sample as real. It diverges to $-\infty$ when $D_{\theta_D}(x) \to 0$, i.e., the discriminator mistakes a real sample for a fake one.
- **Fake term:** $\log(1 - D_{\theta_D}(G_{\theta_G}(z)))$ reaches its maximum value of 0 when $D_{\theta_D}(G_{\theta_G}(z)) \to 0$, i.e., the discriminator correctly recognizes a generated sample as fake. It diverges to $-\infty$ when $D_{\theta_D}(G_{\theta_G}(z)) \to 1$, i.e., the generator manages to fool the discriminator.

In other words, $V(D,G)$ is large, i.e., close to 0, exactly when the discriminator classifies both real and fake samples correctly.

12.3.2 Min-Max Game

From eq. 12.1, we can see that the generator and discriminator have opposite goals:

- The **discriminator** only controls θ_D, and wants to classify samples as accurately as possible. It wants to **maximize** $V(D,G)$ with respect to θ_D.
- The **generator** only controls θ_G, which appears solely inside the fake term. It cannot influence the real term, but it can push $D_{\theta_D}(G_{\theta_G}(z)) \to 1$ to fool the discriminator, driving the fake term towards $-\infty$. It wants to **minimize** $V(D,G)$ with respect to θ_G.

Because the two networks pursue opposite goals over the same value function, training is naturally framed as a **min-max** game between the generator and discriminator networks:

$$\min_{\theta_G} \max_{\theta_D} [\mathbb{E}_{x \sim p_{data}}[\log D_{\theta_D}(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D_{\theta_D}(G_{\theta_G}(z)))]] \tag{12.2}$$

where θ_G and θ_D are the parameters of the generator and discriminator networks, respectively.

12.3.3 Problems with the Original Objective Function

Although this is a very strong theoretical framework, such objective function had 2 main problems in practice:

- Such min-max game is very hard to optimize in practice, requiring an alternation of gradient ascents on the discriminator and gradient descents on the generator.
- The derivative of the objective function with respect to the generator's parameters heavily suffers from vanishing gradients. In particular, when the discriminator is very good at distinguishing real from fake samples, the generator's gradients become very small, making it difficult for the generator to improve. Instead, gradient signals are very strong when the discriminator is poor at distinguishing real from fake samples, which can lead to unstable training dynamics.

12.3.4 A Solution: Non-Saturating Loss

To fix both problems, a common approach is to use a **non-saturating loss** for the generator, which maximizes $\log D_{\theta_D}(G_{\theta_G}(z))$ instead of minimizing $\log(1 - D_{\theta_D}(G_{\theta_G}(z)))$. The resulting objective function is as follows:

$$\max_{\theta_G} \max_{\theta_D} [\mathbb{E}_{x \sim p_{data}}[\log D_{\theta_D}(x)] + \mathbb{E}_{z \sim p_z}[\log(D_{\theta_D}(G_{\theta_G}(z)))]] \tag{12.3}$$

12.3.5 Inference

After training, the generator can be used to generate new samples by sampling from the simple distribution p_z and passing these samples through the generator network. Usually, the discriminator is discarded after training, as it is no longer needed for generating new samples.

12.4 Properties Of GANs

In this section, we will discuss some of the properties of GANs that make them a powerful tool for generative modeling. We will also discuss some of the challenges that arise when training GANs, and how these challenges can be addressed.

12.4.1 Latent Space Arithmetic

It was observed that the latent space of GANs exhibits interesting properties, such as the ability to perform arithmetic operations on the latent vectors. For example, given a latent vector z_1 that generates an image of a smiling woman, a latent vector z_2 that generates an image of a neutral woman and a latent vector z_3 that generates an image of a neutral man, we can perform the following arithmetic operation in the latent space:

$$z_4 = z_1 - z_2 + z_3 \tag{12.4}$$

The resulting latent vector z_4 can then be passed through the generator to produce an image of a smiling man. This property of the latent space allows for interesting manipulations of the generated samples, such as changing facial expressions or adding/removing objects from images.

12.4.2 Quality of Generated Samples

Since GANs are trained to generate samples that are indistinguishable from real data, the quality of the generated samples is often very high. We will see in the next chapters how this isn't the case for other generative models, such as VAEs, which often produce blurry samples due to the use of a Gaussian likelihood function.

12.4.3 No Likelihood Estimation

As discussed earlier, GANs are Implicit Density Estimation models: they never define nor compute an explicit likelihood $p_{model}(x)$, and only learn to sample from it. This is in stark contrast to explicit density models, such as VAEs, which optimize (a bound on) the likelihood of the data directly.

While this makes GANs very flexible, since they are not constrained by the tractability of an explicit density function, it also means that there is no likelihood value to monitor during or after training. This is one of the main reasons why evaluating GANs is so challenging, as we will see in Section 12.8.

12.4.4 Mode Collapse

We would like the generator to learn to generate samples that cover the entire data distribution. However, in practice, earlier versions of GANs suffered from a problem called **mode collapse**. The generator would learn to produce only a limited variety of samples, often collapsing to a single mode of the data distribution.

For example, if the training data consists of images of different animals, the generator might learn to produce only images of cats, ignoring other animals in the dataset.

12.5 GAN Architectures

What we have discussed so far is the basic GAN architecture, but we still need to discuss how these components are implemented in practice. We will now discuss some of the most important GAN architectures that have been proposed in the literature.

12.5.1 DCGAN

Deep Convolutional GANs (DCGANs) are a class of GANs that use deep convolutional neural networks for both the generator and discriminator networks.

Generator: Starting from a random noise vector, the generator network uses a series of convolutional layers to upsample the noise vector into a final $64 \times 64 \times 3$ image.

Discriminator: The discriminator network again uses a series of convolutional layers to downsample the input image into a single scalar output. To do so, it doesn't use any pooling layers, but instead uses strided convolutions to downsample the input image. A final fully connected layer, along with a softmax activation function, is used to output a probability that the input image is real or fake.

12.5.2 LSGAN

Least Squares GANs (LSGANs) are a class of GANs that replace the binary cross-entropy loss used in the original GAN formulation with a least squares loss:

$$\min_{\theta_D} \frac{1}{2}\mathbb{E}_{x \sim p_{data}}[(D_{\theta_D}(x) - 1)^2] + \frac{1}{2}\mathbb{E}_{z \sim p_z}[D_{\theta_D}(G_{\theta_G}(z))^2] \tag{12.5}$$

$$\min_{\theta_G} \frac{1}{2}\mathbb{E}_{z\sim p_z}[(D_{\theta_D}(G_{\theta_G}(z)) - 1)^2] \tag{12.6}$$

The discriminator is trained to push its output towards 1 on real samples and towards 0 on fake samples, while the generator is trained to push $D_{\theta_D}(G_{\theta_G}(z))$ towards 1, i.e., to fool the discriminator.

This change had 2 main advantages:

- Least squares loss provides stronger gradients for the generator when the discriminator is confident, which helps to mitigate the vanishing gradient problem we discussed earlier.
- The stronger gradients also allow the generator to escape from local minima, mitigating the mode collapse problem we discussed earlier.

After this work, many other GAN variants have been proposed, each with its own advantages and disadvantages.

12.5.3 Wasserstein GAN

Wasserstein GANs (WGANs) try to further improve the training stability of LSGANs by using the Wasserstein distance. The original GAN formulation is equivalent to minimizing the Jensen-Shannon divergence between the real and generated data distributions. However, the Jensen-Shannon divergence has been shown to be a poor choice for training GANs, as it can lead to vanishing gradients and mode collapse.

The Wasserstein distance, on the other hand, is a more robust metric for measuring the distance between two probability distributions. The Wasserstein distance has an important property: it changes smoothly, even when the two distributions do not overlap. This property allows the generator to receive meaningful gradients even when the discriminator is very confident, which helps to mitigate the vanishing gradient problem and improve training stability.

12.6 Conditioning the Generation

All the architectures we have discussed so far are unconditional, meaning that the generator produces samples without any control over the output. GANS can be conditioned on additional information, such as class labels, text descriptions, or even other images, to generate samples that satisfy specific conditions.

12.6.1 Conditional GAN

Conditional GANs (cGANs) are a class of GANs that allow for conditioning the generation process trough labels. In particular, the generator takes as input both a random noise vector and a label, and generates a sample that corresponds to the given label. The discriminator also takes as input a sample and a label for both real and generated samples, and outputs a probability that the sample is real given the label.

The objective function for cGANs is similar to the original GAN formulation, but with the addition of the conditioning label:

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}}[\log D(x|y)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z|y)))] \quad (12.7)$$

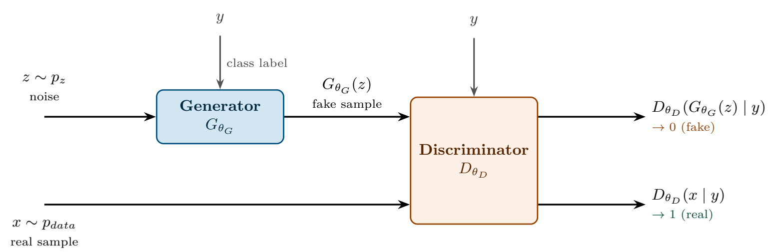


Figure 46: The cGAN architecture: identical to the base GAN in Figure 45, except that both the generator and the discriminator are additionally conditioned on a label y.

12.6.2 Text-To-Image

Following the success of cGANs, several works have proposed to condition the generation process on text descriptions, allowing for **text-to-image synthesis**. In this setting, some text description is appended to the random noise vector and passed through the generator, which produces an image that corresponds to the given text description.

Similarly to the cGANs, the discriminator also takes as input both an image and a text description, and outputs a probability that the image is real given the text description.

12.6.3 Image-to-Image Translation

One of the most popular applications of GANs is **image-to-image translation**, where the goal is to learn a mapping from one image domain to another. For example, given a dataset of black-and-white images and their corresponding color images, a GAN can be trained to learn the mapping from black-and-white to color images.

Such architecture is called **Pix2Pix**, and it uses a CNN-based generator that instead of generating an image from a random noise vector, takes an input image and generates a corresponding output image, of the same size. Again, the discriminator takes as input both the input image and the generated output image, and outputs a probability that the output image is real given the input image.

12.6.4 CycleGAN

CycleGAN is a Generative Adversarial Network (GAN) architecture designed for image-to-image translation without relying on paired training data. The previously discussed Pix2Pix model requires paired datasets, meaning that for every input image, there must be a strict corresponding output image. However, in many real-world scenarios, such perfectly aligned data is unavailable or prohibitively expensive to collect.

To address this, the CycleGAN architecture employs two generators and two discriminators:

- The first generator, G, learns a mapping from the source domain X to the target domain Y, producing generated images $\hat{y} = G(x)$.
- The second generator, F, learns the inverse mapping from the target domain Y back to the source domain X, producing reconstructed images $\hat{x} = F(\hat{y})$.
- The first discriminator, D_Y, is trained to distinguish between real images y from the target domain Y and generated images $\hat{y}$ produced by G.
- The second discriminator, D_X, is trained to distinguish between real images x from the source domain X and generated images $\hat{x}$ produced by F.

To ensure that the generators learn meaningful and structurally consistent mappings rather than random permutations in the target domain, CycleGAN introduces a **cycle consistency loss**. This loss enforces the intuition that translating an image to the target domain and then translating it back should recover the original image (i.e., $F(G(x)) \approx x$ and $G(F(y)) \approx y$).

Note that the cycle consistency loss alone only enforces that the mapping is invertible, not that the intermediate translated image actually looks like a real sample of the target domain; this is why an adversarial loss from a dedicated discriminator is still needed on each side (D_Y for G, D_X for F), so that both translations remain realistic rather than merely reconstructible.

Formally, the full objective function for CycleGAN is defined as follows:

$$\mathcal{L}(G, F, D_X, D_Y) = \mathcal{L}_{\text{GAN}}(G, D_Y, X, Y) + \mathcal{L}_{\text{GAN}}(F, D_X, Y, X) + \lambda \mathcal{L}_{\text{cyc}}(G, F) \tag{12.8}$$

Here, the first two terms represent the adversarial losses for the respective generator-discriminator pairs, and the final term is the cycle consistency loss, weighted by a hyperparameter λ. The cycle consistency loss is typically implemented as an L_1 penalty between the original images and their cyclic reconstructions.

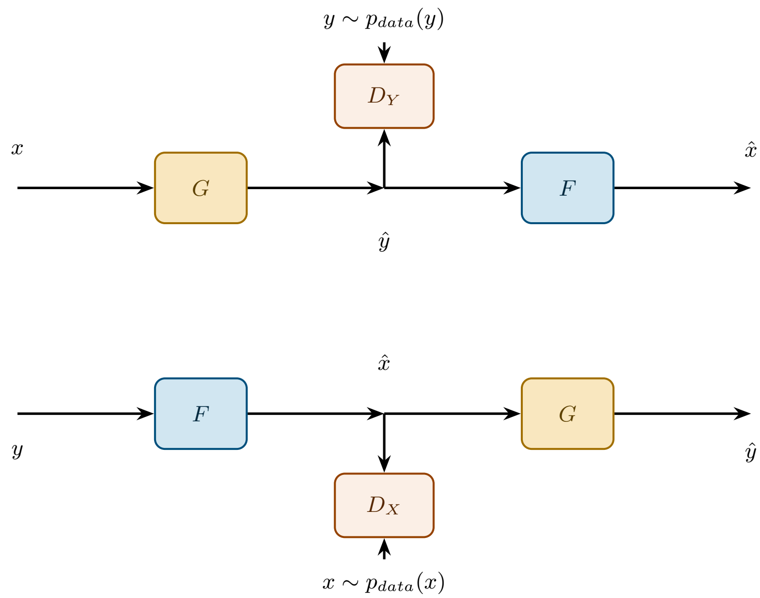


Figure 47: The CycleGAN architecture. In the forward cycle (top), G translates x into the target domain and F maps it back, yielding the reconstruction $\hat{x}$; D_Y distinguishes between real and translated images in domain Y. The backward cycle (bottom) mirrors this process starting from y, with D_X operating in domain X.

12.6.5 StyleGAN

StyleGAN is a GAN architecture proposed by NVIDIA that allows for fine-grained control over the generated images by introducing a style-based generator architecture. To enable this, StyleGAN introduces a **mapping network** that thakes the random noise vector as input and maps it to an intermediate latent space, with better features for controlling the style of the generated images.

The intermediate latent vector w is then passed through a series of **AdaIN** layers, which control the style of the generated image at different levels of detail. This allows for controlling the overall structure of the image, as well as finer details such as textures and colors.

AdaIN Adaptive Instance Normalization (AdaIN) is a normalization technique taken from style transfer literature. Technically, this is an instance normalization layer that instead of using learned parameters from the training data, uses the parameters computed from the style image, which in the case of StyleGAN is the intermediate latent vector w.

12.7 Training Tricks

A few training tricks have been proposed to improve the training stability of GANs.

- **Minibatch Discrimination:** used to reduce mode collapse. It allows the discriminator to look at the similarity of an image x to other images in the same minibatch. The similarity is passed to the discriminator as an additional feature, allowing it to detect if the generator is producing a limited variety of samples.
- **Virtual Batch Normalization:** Due to batch normalization, the generated images are not independent of each other. To mitigate this problem, virtual batch normalization uses a reference batch of real images to compute the normalization statistics for each generated image, instead of using the statistics of the current minibatch.

12.8 Evaluating Generative Models Performance

Since the inception of GANs, evaluating their performance has been a challenging task. To this day, the evaluation of generative models remains an open research problem, and there is no single universally accepted metric to assess GAN performance.

Early approaches relied on human evaluators to assess the quality of generated samples. However, this method is unscalable, expensive, and highly subjective, as different annotators may have conflicting opinions on visual quality.

Additionally, computing the likelihood of generated samples or directly comparing the real and generated data distributions is intractable, as GANs do not explicitly model the underlying probability distribution of the data.

12.8.1 Properties of Generative Models

Before introducing specific evaluation metrics, it is useful to define the key properties that a good generative model should exhibit:

- **High quality samples:** the generated samples should be indistinguishable from real samples, used to train the model;

- **Coverage:** samples should represent the entire data distribution, rather than a limited subset of it;
- **Well-structured latent space:** every latent variable z should correspond to a meaningful data sample x. Most importantly, small changes in the latent variable z should result in small changes in the generated sample x.
- **Efficient likelihood estimation:** if the model is probabilistic, it should be able to efficiently compute the likelihood of a given sample x.

Remark Each of the metrics discussed below is designed to primarily target one (or two) of these properties, rather than assessing overall model performance: the Inception Score and the Fréchet/Kernel Inception Distance mainly capture quality and coverage jointly, Precision and Recall isolate each of them individually, and the Perceptual Path Length evaluates the structure of the latent space. Notably, none of these metrics can assess efficient likelihood estimation, since GANs never compute an explicit likelihood in the first place.

12.8.2 Inception Score

The first metric proposed to standardize and replace human evaluation was the **Inception Score** (IS). The core idea is that a good generator should produce **highly recognizable objects** (high quality) across a **wide variety of classes** (high diversity).

To compute the IS, a pre-trained Inception model classifies the generated samples. For a given generated image $x = G(z)$, the network outputs $p(y|x)$, a conditional distribution over the ImageNet classes y. If x is crisp and easily recognizable, $p(y|x)$ will be a highly peaked distribution (low entropy), serving as a proxy for sample quality. Conversely, if x is blurry or unrealistic, the distribution will be closer to uniform (high entropy).

The marginal distribution, $p(y) = \mathbb{E}_{x\sim p_g}[p(y|x)]$, represents the overall class distribution the generator produces across all samples. If the generator suffers from mode collapse, $p(y)$ will be concentrated on a few classes (low entropy, poor coverage). A diverse generator, however, will produce samples evenly spread across classes, resulting in a uniform $p(y)$ (high entropy, good coverage).

Formally, the Inception Score evaluates these two properties using the following equation:

$$IS = \exp\left(\mathbb{E}_{x\sim p_g}\left[D_{KL}(p(y|x)\,||\,p(y))\right]\right) \tag{12.9}$$

This formula calculates the **Kullback-Leibler (KL) divergence** between the conditional label distribution $p(y|x)$ and the marginal label distribution $p(y)$. This divergence is maximized precisely when the two distributions are optimally opposed: $p(y|x)$ is a spike on a single class (high quality) while $p(y)$ is uniformly spread across all classes (high diversity). Therefore, a higher KL divergence yields a higher, and thus better, Inception Score.

Issues with the Inception Score The Inception Score suffers from two main issues:

- **Memorization:** A GAN that simply memorizes the training data will achieve a high IS, as the generated samples will perfectly match real images (easily classifiable and evenly distributed).
- **Mode Collapse:** If the generator produces only a single realistic sample for each class, the IS can still be high. The samples are evenly distributed across classes and easily recognized, but the model entirely lacks intra-class diversity.

12.8.3 Frechet Inception Distance

The **Fréchet Inception Distance** (FID) was proposed to overcome several shortcomings of the Inception Score. Rather than relying solely on the predicted class probabilities, FID compares the distributions of feature representations extracted from real and generated samples.

To compute FID, both real and generated samples are passed through a pre-trained Inception network, and feature vectors are extracted from one of its intermediate layers. The feature distributions are then approximated as multivariate Gaussian distributions, characterized by their means and covariance matrices.

The FID score is defined as

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\Big(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\Big)\,, \tag{12.10}$$

where (μ_r, Σ_r) and (μ_g, Σ_g) denote the mean vectors and covariance matrices of the real and generated feature distributions, respectively.

This expression corresponds to the Fréchet distance between the two Gaussian distributions. A lower FID score indicates that the generated feature distribution is closer to that of the real data, suggesting higher visual quality and greater diversity of the generated samples.

Compared to IS, FID directly measures the similarity between generated and real data distributions. Nevertheless, it still has limitations. In particular, FID cannot explicitly detect overfitting, as a generator that simply memorizes the training set would achieve an FID score close to zero.

12.8.4 Kernel Inception Distance

Like FID, the **Kernel Inception Distance** (KID) evaluates the similarity between the feature distributions of real and generated samples extracted from a pre-trained Inception network.

Instead of computing the Fréchet distance between Gaussian approximations of the feature distributions, KID measures their discrepancy using the **Maximum Mean Discrepancy** (MMD). Specifically, KID computes the squared MMD between the two sets of feature vectors using a polynomial kernel.

A lower KID score indicates that the generated feature distribution is more similar to the real one.

Compared to FID, the main advantage of KID is that its estimator is **unbiased**, whereas the empirical estimator of FID is biased, particularly when computed from a limited number of samples. Consequently, KID generally provides a more reliable estimate of distribution similarity for smaller datasets, while maintaining comparable performance on larger datasets.

12.8.5 Precision and Recall

The well-known **precision** and **recall** metrics have also been extended to evaluate generative models.

In this setting, **precision** measures the quality (or fidelity) of the generated samples by estimating the fraction of generated samples that lie within the support of the real data distribution. Conversely, **recall** measures the diversity (or coverage) of the generated distribution by estimating the fraction of real samples that lie within the support of the generated distribution.

Pratically, precision and recall are computed by embedding both real and generated samples into a feature space using a pre-trained network (e.g., Inception). The support of each distribution is then estimated using nearest neighbor methods.

12.8.6 Perceptual Path Length

The **Perceptual Path Length** (PPL) metric was introduced to evaluate the quality of the latent space learned by a generative model. In particular, it measures the smoothness of the latent space by quantifying how small changes in the latent vector affect the generated samples.

Intuitively, picking two points in the latent space and interpolating between them should result in a smooth transition between the corresponding generated samples.

12.8.7 Issues with GAN Evaluation Metrics

Despite the variety of metrics proposed to evaluate GANs, there is still no universally accepted standard that comprehensively assesses the performance of generative models. Each metric possesses distinct strengths and limitations, often capturing entirely different aspects of the generated distribution.

A fundamental challenge in evaluating GANs is that the generator's loss does not directly correlate with the visual or structural quality of the generated samples. In traditional Machine Learning, a core principle is that the loss function serves as a reliable proxy for model performance. However, in the adversarial framework of a GAN, the generator's loss is a moving target. Because it is heavily influenced by—and entirely relative to—the discriminator's current performance, a low generator loss simply indicates that it is currently fooling the discriminator, not necessarily that it is producing high-quality, realistic samples.

13 Autoencoders

Autoencoders (AEs) are a class of neural networks trained in an unsupervised manner to learn efficient, compressed representations of data.

The Core Idea Conceptually, autoencoders generalize the idea behind Principal Component Analysis (PCA): both aim to learn a lower-dimensional representation of the input data, known as the **latent space**.

While PCA is restricted to discovering strictly linear transformations, autoencoders leverage non-linear activation functions, allowing them to model much more complex, non-linear data manifolds. In fact, if an autoencoder uses only linear activations and is trained with a mean squared error loss, it learns to span the same principal subspace as PCA.

This lower-dimensional representation is learned by training the network to map the input data x to a compressed space z via an *encoder*, and then reconstruct the original data as accurately as possible through a second network called the *decoder*.

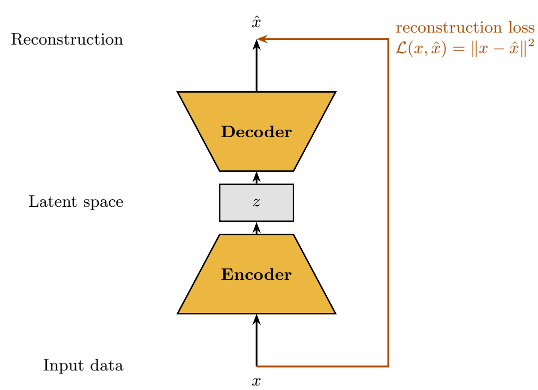


Figure 48: Autoencoder architecture: the encoder compresses the input x into a latent code z, and the decoder reconstructs $\hat{x}$ from it.

To train an autoencoder, we minimize a **reconstruction loss** between the input and the output of the network.

Formally, given an input x, the encoder f_θ maps it to a latent representation $z = f_\theta(x)$, and the decoder g_ϕ reconstructs the input as $\hat{x} = g_\phi(z)$.

A common choice is the MSE:

$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2. \tag{13.1}$$

Depending on the data type, other reconstruction losses may also be used. For example, binary cross-entropy is often preferred when modeling binary or normalized image data.

Implementation Autoencoders give us a lot of flexibility in terms of architecture and design choices. In fact, while the idea is fixed, the implementation can vary significantly depending on the specific use case and data type.

Why AEs? Once trained, the decoder is often discarded and the encoder is used as a feature extractor or for dimensionality reduction. The learned latent representation can then be exploited for a variety of downstream tasks, including visualization, anomaly detection, clustering, and as input features for other machine learning models.

However, standard autoencoders do not explicitly learn a probability distribution over the latent space. As a consequence, although they reconstruct existing samples well, they are not naturally suited for generating new ones.

13.1 Variational Autoencoders

Building upon the idea of autoencoders, we obtain a more powerful generative model called the **VAE**. Unlike standard autoencoders, VAEs learn not only a latent representation of the data but also a probability distribution over that latent space, making it possible to generate entirely new samples.

VAEs belong to the class of **explicit density estimation** generative models, whose goal is to directly model the probability distribution of the observed data.

13.1.1 From Data to Latent Variables

The key idea behind Variational Autoencoders (VAEs) is that the observed data are assumed to be generated from a lower-dimensional *latent* representation z. Rather than modeling the data distribution directly, VAEs first model the latent space and then describe how latent variables generate observations.

Under this assumption, the probability of observing a data point x is obtained by considering all possible latent variables that could have generated it. This gives rise to the marginal likelihood

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz, \tag{13.2}$$

where $p(z)$ is a prior distribution over the latent variable, typically chosen as a standard Gaussian, and $p_\theta(x \mid z)$ is the conditional distribution learned by the decoder.

The objective is to learn the model parameters θ by maximizing the likelihood of the observed data. However, computing the marginal likelihood in Equation 13.2 is generally intractable, as it requires integrating over the entire latent space.

Rather than attempting to evaluate the marginal likelihood directly, it is natural to ask the inverse question: given an observation x, which latent variable z is most likely to have generated it? This is described by the posterior distribution

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z)\, p(z)}{p_\theta(x)}. \tag{13.3}$$

Unfortunately, computing the posterior is just as difficult as computing the marginal likelihood, since it also depends on the intractable quantity $p_\theta(x)$.

To overcome this issue, VAEs introduce a second distribution,

$$q_\phi(z \mid x),$$

parameterized by another neural network, called the *encoder*. This distribution provides a tractable approximation of the true posterior $p_\theta(z \mid x)$.

13.1.2 The Evidence Lower Bound (ELBO)

Introducing the approximate posterior allows us to derive the following decomposition of the log-likelihood:

$$\log p_\theta(x) = \underbrace{\mathbb{E}_z \left[\log p_\theta(x \mid z)\right] - D_{KL}\left(q_\phi(z \mid x) \,\|\, p_\theta(z)\right)}_{\text{ELBO}} + D_{KL}\left(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\right). \tag{13.4}$$

The three terms have the following interpretation:

- The first term is the *reconstruction term*: the expected log-likelihood of the data under the approximate posterior. It measures how well the decoder $p_\theta(x \mid z)$ can reconstruct the observed data x from a latent representation sampled from $q_\phi(z \mid x)$, and maximizing it encourages accurate reconstructions.
- The second term is the *regularization term*: the Kullback–Leibler divergence between the approximate posterior $q_\phi(z \mid x)$ and the prior $p(z)$. Minimizing it encourages the latent representations to remain close to the prior, resulting in a smooth and well-structured latent space.

- The third term is the Kullback–Leibler divergence between the approximate and true posterior. Since this divergence is always non-negative, the sum of the first two terms is a lower bound on the log-likelihood, known as the **Evidence Lower Bound** (ELBO):

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_z \left[\log p_\theta(x|z)\right] - D_{KL}\left(q_\phi(z|x) \parallel p(z)\right). \tag{13.5}$$

Consequently, instead of maximizing the intractable log-likelihood directly, VAEs maximize the ELBO. This simultaneously increases the data likelihood while encouraging the approximate posterior to match the true posterior, and yields a meaningful latent space in which similar inputs are mapped to nearby points.

13.1.3 VAE Architecture

Equation 13.5 maps directly onto the encoder-decoder pair we set out to build, with each network now standing for one of the two distributions involved.

The term $p_\theta(x \mid z)$ represents the probability of generating x given a latent variable z. This is exactly what the **decoder** computes: given a latent code z, it outputs a distribution over reconstructions of x.

Symmetrically, the term $q_\phi(z \mid x)$ represents the (approximate) probability of a latent variable z given the observed data x. This is exactly what the **encoder** computes: given an input x, it outputs a distribution over latent codes z, standing in for the true, intractable posterior $p_\theta(z \mid x)$.

Since we are modeling the probabilistic generation of data, both networks are therefore **probabilistic**: the encoder does not output a single point z, but the parameters of the distribution $q_\phi(z \mid x)$ (a mean $\mu_\phi(x)$ and a standard deviation $\sigma_\phi(x)$), from which z is sampled. The decoder, in turn, does not output a single reconstruction, but the parameters of the distribution $p_\theta(x \mid z)$, from which $\hat{x}$ is sampled. This is illustrated in Figure 49.

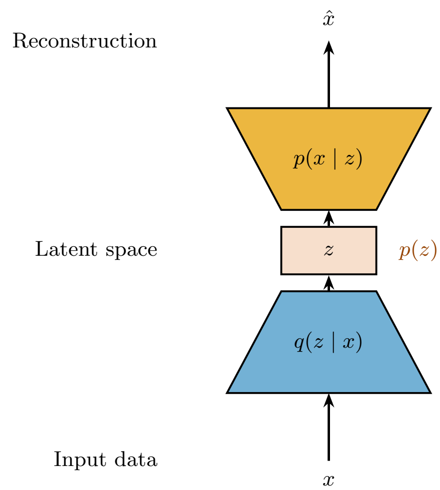


Figure 49: VAE architecture: the encoder computes the approximate posterior $q_\phi(z \mid x)$, from which a latent code z is sampled, and the decoder computes the likelihood $p_\theta(x \mid z)$, from which the reconstruction $\hat{x}$ is obtained. The prior $p(z)$ regularizes the latent space towards a standard Gaussian.

13.1.4 Training Time

At training time, the full architecture shown in Figure 49 is used end-to-end. The encoder receives a real sample x and predicts the parameters of the approximate posterior distribution $q_\phi(z \mid x)$, namely its mean $\mu_\phi(x)$ and standard deviation $\sigma_\phi(x)$. A latent vector z is then sampled from this distribution and passed to the decoder, which reconstructs the original sample.

Since the sampling operation is not differentiable, gradients cannot be propagated through it directly. VAEs overcome this issue using the **reparameterization trick**: instead of sampling z directly, we sample noise $\epsilon \sim \mathcal{N}(0, I)$ and compute

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon,$$

thereby expressing the stochasticity through ϵ while keeping the dependence on the network parameters differentiable.

The encoder and decoder are trained jointly by maximizing the ELBO (Equation 13.5) using gradient-based optimization. The reconstruction term encourages the decoder to accurately reconstruct the input sample, while the KL divergence regularizes the latent space by encouraging the approximate posterior $q_\phi(z \mid x)$ to remain close to the prior distribution $p(z)$.

13.1.5 Inference Time

Unlike at training time, at inference time there is no observed sample x to feed to the encoder, so the encoder is discarded entirely. This is possible because the regularization term encourages $q_\phi(z \mid x)$ to match the prior distribution $p(z)$ for every x in the dataset. As a result, once training is complete, we can generate new data by simply sampling a latent vector directly from the prior,

$$z \sim p(z),$$

and passing it through the decoder alone. This regularized latent space is what gives VAEs their generative capability.

13.1.6 Conditional VAEs

Similarly to how we can condition GANs on additional information, we can also condition VAEs on auxiliary variables. This leads to the **Conditional Variational Autoencoder** (CVAE), which allows us to generate data conditioned on specific attributes. This is achieved by concatenating the conditioning variable c to both the input of the encoder and the latent code fed into the decoder, effectively learning a conditional distribution $p_\theta(x|z, c)$.

13.2 GAN vs VAE

Even though both GANs and VAEs are generative models, they use very different approaches to generate new data, leading to different strengths and weaknesses.

- **GANs**: provide a mechanism to generate samples, encouraging the model to produce progressively more realistic outputs. However, they do not provide an explicit density function for the data distribution, making it difficult to evaluate the likelihood of generated samples.
- **VAEs**: provide an explicit density function for the data distribution. The distribution can then be used both for generating new samples and for evaluating the likelihood of existing samples. However, VAEs often produce blurrier outputs compared to GANs, as they optimize a lower bound on the log-likelihood rather than directly optimizing for sample quality.

We can more analitically compare the two models by looking at their foundational properties:

- **Efficient Sampling:** at inference time, both GANs and VAEs can generate new samples efficiently by sampling from their respective latent spaces;

- **Sample Quality:** GANs are known for producing high-quality, realistic samples, due to the adversarial training process that encourages the generator to produce outputs indistinguishable from real data. VAEs, on the other hand, often produce blurrier samples due to the regularization imposed by the KL divergence term in their loss function;

- **Coverage:** VAEs are designed to cover the entire data distribution, as they explicitly model the underlying probability distribution. GANs, however, can suffer from mode collapse, where the generator produces a limited variety of samples, failing to capture the full diversity of the data;

- **Well-behaved latent space:** GAN were shown to have a well-structured latent space, allowing for arithmetic operations. For VAEs, the latent space is regularized to follow a known prior distribution, which can also lead to meaningful interpolations between samples.

- **Likelihood Estimation:** VAEs provide a tractable lower bound on the log-likelihood of the data, allowing for efficient likelihood estimation. GANs, on the other hand, do not provide a direct way to compute the likelihood of generated samples, making it challenging to evaluate their performance in terms of likelihood.

14 Diffusion Models

Another class of explicit density estimation generative models is **diffusion models**. Although initially proposed in 2015, they have only recently surged in popularity due to their ability to generate exceptionally high-quality samples.

Around 2015, GANs were the state-of-the-art in generative modeling, despite having some serious limitations (as discussed in the previous chapter). It was not until 2020 that breakthroughs in architectures and training techniques allowed diffusion models to surpass GANs in sample quality, establishing them as the current state-of-the-art.

Similar to VAEs, diffusion models use neural networks to transform a simple prior distribution (e.g., a standard Gaussian) into a complex data distribution that ideally matches the training data. Unlike traditional models like GANs, which perform this transformation in a single pass, diffusion models achieve it iteratively, gradually denoising a latent sample step-by-step.

14.1 Idea

The core idea behind diffusion models is actually quite simple: we start with a sample from the data distribution and gradually add noise to it over a series of steps, until it becomes pure noise. Then, we train a neural network to reverse this process, learning to denoise the noisy samples step-by-step until we recover the original data sample.

More formally, we define:

- **Forward diffusion process:** starting from a data sample x_0 from the data distribution, we define a fixed Markov chain that gradually adds noise to the data sample x_0 over T steps, resulting in a sequence of noisy samples $x_1, x_2, \ldots, x_T$, where x_T is nearly pure Gaussian noise.
- **Reverse denoising process:** a neural network that learns to reverse the forward diffusion process, starting from the noisy sample x_T and iteratively denoising it to recover the original data sample x_0.

Let's now dive into the details of the forward diffusion process and the reverse denoising process, and how they are implemented in practice.

14.2 Forward Process

The forward diffusion process can be defined as a fixed Markov chain that gradually adds Gaussian noise to the data sample x_0 over T steps.

Markov chain. A Markov chain is a stochastic process that satisfies the **Markov property**: the probability distribution of the next state depends only on the current state and not on the sequence of previous states. This property is also known as **memorylessness**.

This implies that the forward process requires no learning, since it is completely specified by a predefined noise schedule.

We can formalize the forward diffusion process as follows:

$$q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}x_{t-1}, \beta_t I) \tag{14.1}$$

where β_t is a variance schedule that controls the amount of noise added at each step.

Basically, at each step t, we take the previous sample x_{t-1}, reduce its magnitude by a factor of $\sqrt{1-\beta_t}$, subsituting it with a random Gaussian noise with variance β_t.

14.2.1 A problem

Suppose we have a data sample x_0 and we want to generate its noisy version x_T at step T. We can do this by iteratively applying the forward diffusion (Equation 14.1) process T times, starting from x_0 and generating $x_1, x_2, \ldots, x_T$. This would result in:

$$q(x_T|x_0) = \prod_{i=1}^{T} q(x_i|x_{i-1})$$

However, this approach is computationally expensive, especially for large T, since we would need to sample from the Gaussian distribution T times, which can be slow and inefficient.

14.2.2 Reparametrization trick

Instead, we can use a reparametrization trick to directly sample x_T from x_0 in a single step, without having to go through all the intermediate steps.

First, we can rewrite the forward diffusion process in a more convenient form:

$$q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_0, (1-\alpha_t)I)$$

where $\alpha_t = (1-\beta_t)$.

By recursively applying this equation, we can derive a closed-form expression for $q(x_T|x_0)$:

$$q(x_T|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_T}x_0, (1-\bar{\alpha}_T)I)$$

where $\bar{\alpha}_T = \prod_{t=1}^{T} \alpha_t$.

This makes training and sampling actually feasible, since we can now sample x_T directly from x_0 in a single step, without having to go through all the intermediate steps.

14.3 Reverse Process

The reverse denoising process is the core of the diffusion model, where we train a neural network to learn the reverse of the forward diffusion process. The goal is to start from a noisy sample x_T and iteratively denoise it to recover the original data sample x_0.

We could define the reverse process as a Markov chain that learns to reverse the forward diffusion process $p(x_{t-1}|x_t)$. Applying Bayes' theorem, we can express the reverse process as:

$$p(x_{t-1}|x_t) = \frac{p(x_t|x_{t-1})p(x_{t-1})}{p(x_t)}$$

But this is intractable in practice, since the marginal distribution $p(x_t)$ would require to know the entire data distribution, at every step t, which is not feasible.

Therefore, as previously seen in the VAE chapter, we can train a neural network to approximate the reverse process $p_\theta(x_{t-1}|x_t)$, where θ are the parameters of the neural network.

By assuming $T \to \infty$ and $\beta_t \to 0$, we can show that the reverse process can be approximated as a Gaussian distribution:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_t)$$

where $\mu_\theta(x_t, t)$ is the mean predicted by the neural network and Σ_t is a fixed variance schedule.

Usually, only the mean $\mu_\theta(x_t, t)$ is predicted by the neural network, while the variance Σ_t is fixed and known. It was later shown that learning the variance Σ_t can improve sample quality, but it is not strictly necessary.

14.3.1 Hierarchical VAEs

Diffusion models can also be interpreted as a special case of hierarchical VAEs. While a standard VAE is formulated with a single latent variable z, a diffusion model introduces a hierarchy of T latent variables,

$$x_1, x_2, \ldots, x_T,$$

where each variable corresponds to a progressively noisier version of the data sample x_0.

Under this interpretation, diffusion models exhibit several distinctive characteristics:

- **Fixed encoder:** The encoder is not learned but is instead defined by the forward diffusion process, which gradually corrupts the input with Gaussian noise.
- **Hierarchical latent variables:** The latent variables $x_1, x_2, \ldots, x_T$ form a deep hierarchy, with each variable having the same dimensionality as the original data sample x_0.
- **Shared decoder:** A single neural network is reused at every step of the reverse diffusion process to model the conditional distributions $p_\theta(x_{t-1} \mid x_t)$, with the timestep t provided as an additional conditioning input.

This interpretation immediatly suggests using the same training objective as in VAEs, which is the ELBO.

14.3.2 Loss

Following the VAE framework, we can derive the loss function for diffusion models by maximizing the ELBO. The ELBO can be expressed as:

$$\mathcal{L} = \mathbb{E}_q \left[\underbrace{D_{KL}(q(x_T|x_0) \| p_\theta(x_T))}_{\text{Ensure final Standard Gaussian}} + \underbrace{\sum_{t=1}^{T} D_{KL}(q(x_{t-1}|x_t, x_0) \| p_\theta(x_{t-1}|x_t))}_{\text{KL divergence term}} - \underbrace{\log p_\theta(x_0|x_1)}_{\text{Reconstruction term}} \right]$$

Where:

- **First term:** The first term measures the KL divergence between the final noisy sample x_T and the prior distribution $p_\theta(x_T)$, which is typically a standard Gaussian. This term ensures that the final latent representation is as close as possible to the prior distribution.
- **Second term:** The second term sums the KL divergences between the true posterior $q(x_{t-1}|x_t, x_0)$ and the learned reverse process $p_\theta(x_{t-1}|x_t)$ for each timestep t. This term encourages the model to learn a reverse process that closely approximates the true posterior, effectively guiding the denoising process.

- **Third term:** The third term is the reconstruction term, which measures how well the model can reconstruct the original data sample x_0 from the first noisy sample x_1. This term ensures that the model learns to generate samples that are faithful to the original data distribution.

From the ELBO, we can derive a simplified loss function that is more practical for training diffusion models, that focuses on predicting the noise ϵ added to the original data sample x_0 at each timestep t.

$$\|\epsilon - \epsilon_\theta(x_t, t)\|^2 \tag{14.2}$$

Where $\epsilon_\theta(x_t, t)$ is the noise predicted by the neural network at timestep t.

By applying some further simplifications and reparametrizations, we can express the final loss function as:

$$\mathcal{L} = \mathbb{E}_{x_0,\epsilon,t}\left[\|\epsilon - \underbrace{\epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)}_{\text{Predicted noise}}\|^2\right]$$

Where, at a high level, we are basically picking a random timestep t, a random noise ϵ, and a random data sample x_0. In the forward process, we generate a noisy sample x_t by adding the noise ϵ to the original data sample x_0 according to the forward diffusion process. Then, we feed this noisy sample x_t into the neural network to predict the noise $\epsilon_\theta(x_t, t)$, and we compute the mean squared error between the predicted noise and the true noise ϵ.

This transforms the original problem of learning a complex data distribution into a simpler regression problem, where the model learns to predict the noise added to the data sample at each timestep.

14.3.3 Sampling

At sampling time, we start from a sample x_T drawn from the prior distribution (typically a standard Gaussian) and iteratively apply the learned reverse process, which is defined as:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right) + \sigma_t z \tag{14.3}$$

This formulation allow to re-weight the predicted noise $\epsilon_\theta(x_t, t)$ based on the current timestep t. In fact, early in the reverse process, when t is large, the updates are conservative, whereas later in the process, when t is small, the updates become more aggressive, allowing the model to refine the generated sample as it approaches the original data distribution.

14.4 The Generative Trilemma

The generative models landscape is often characterized by a trade-off between three key aspects: sample quality, diversity, and inference speed. This trade-off is commonly referred to as the **Generative Trilemma**.

In particular:

- **GANs:** known for their high sample quality and fast inference speed, but they often suffer from mode collapse, leading to low diversity in generated samples.
- **VAEs:** known for their high diversity and fast inference, but they often produce samples of lower quality due to the limitations of the variational approximation.
- **Diffusion Models:** known for their high sample quality and high diversity, but they are typically slower in inference due to the iterative nature of the denoising process.

14.5 Architectures

Again, similar to VAEs, diffusion models can be implemented using a variety of neural network architectures. We can distinguish between **unconditional** and **conditional** diffusion models, depending on whether the model is conditioned on additional information (e.g., class labels, text prompts, etc.) or not.

14.5.1 Unconditional Models

U-Net The most common architecture for image diffusion models is the **U-Net**, a convolutional neural network originally introduced for image segmentation. It consists of an **encoder-decoder** structure with skip connections that preserve spatial information across multiple resolutions, enabling the reconstruction of fine-grained image details.

Many diffusion models also incorporate **self-attention** layers within the U-Net to capture long-range dependencies between distant regions of the image, improving the quality and global coherence of the generated samples.

Since the denoising network must condition its predictions on the current diffusion timestep t, the timestep is embedded into a continuous vector representation before being provided to the network. A common approach is to use **sinusoidal embeddings** (also known as Fourier features), which encode t using sine and cosine functions at multiple frequencies.

Transformers - DiT Later, in 2021, researchers proposed using **transformers** as the backbone architecture for diffusion models, leading to the development of **DiT** (Diffusion Transformer). This approach, although more computationally expensive, shows that transformers can effectively model the denoising process, better than convolutional architectures.

14.5.2 Conditional Models

As with conditional GANs and VAEs, the simplest way to condition a diffusion model on additional information c (e.g., a class label or a text prompt) is to feed c as an extra input to the denoising network, $\epsilon_\theta(x_t, t, c)$.

This works, but in practice the network tends to only weakly follow the conditioning signal: since the training objective is a plain regression loss, the model has little incentive to rely heavily on c if it can already reduce the loss using x_t alone. **Guidance** techniques were introduced to fix this, giving us an explicit knob to control how strongly the generated sample adheres to c, at the cost of some diversity.

Background: The Score Function Before introducing guidance, it is useful to recall that diffusion models can equivalently be described in terms of the **score function** $\nabla_x \log p(x)$, i.e., the gradient of the log-density with respect to the data. It can be shown that the noise-prediction network $\epsilon_\theta(x_t, t)$ introduced earlier approximates (a rescaling of) the score of the noisy data distribution:

$$\nabla_{x_t} \log p(x_t) \approx -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}$$

This equivalence is what lets us reason about conditioning in terms of log-probabilities below, and then translate the result back into a correction of the predicted noise ϵ_θ.

Classifier Guidance Applying Bayes' theorem to the conditional distribution:

$$p(x|c) = \frac{p(c|x)p(x)}{p(c)}$$

Taking the log of both sides:

$$\log p(x|c) = \log p(x) + \log p(c|x) - \log p(c)$$

Since the last term does not depend on x, it vanishes when we take the gradient with respect to x:

$$\nabla_x \log p(x|c) = \nabla_x \log p(x) + \gamma \nabla_x \log p(c|x) \tag{14.4}$$

where we introduced a **guidance scale** $\gamma \geq 0$: $\gamma = 1$ recovers the exact conditional score, $\gamma > 1$ amplifies the conditioning signal (trading diversity for fidelity to c), and $\gamma = 0$ ignores it entirely.

The first term is exactly the unconditional score already modeled by $\epsilon_\theta(x_t, t)$, while the second term requires an auxiliary **classifier** $p_\phi(c \mid x_t)$, trained to predict the label c directly from the noisy sample x_t, separately for every timestep t. At sampling time, its gradient is used to shift the predicted noise:

$$\hat{\epsilon}_\theta(x_t, t) = \epsilon_\theta(x_t, t) - \gamma\sqrt{1 - \bar{\alpha}_t}\, \nabla_{x_t} \log p_\phi(c \mid x_t)$$

The main drawback of this approach is that it requires training a separate, timestep-dependent classifier on noisy data, which is expensive and cannot reuse an existing classifier trained on clean images.

Classifier-Free Guidance Classifier-free guidance removes the need for this auxiliary classifier entirely. Starting again from Equation 14.4 and using Bayes' theorem to rewrite the classifier term:

$$\nabla_x \log p(c \mid x) = \nabla_x \log p(x|c) - \nabla_x \log p(x)$$

Substituting back:

$$\nabla_x \log p(x|c) = \nabla_x \log p(x) + \gamma\big(\nabla_x \log p(x|c) - \nabla_x \log p(x)\big) = (1-\gamma)\nabla_x \log p(x) + \gamma\nabla_x \log p(x|c)$$

Notice that the right-hand side only involves the conditional and unconditional *scores*, quantities that a single diffusion model can already provide, with no classifier needed: we simply train one network $\epsilon_\theta(x_t, t, c)$, randomly replacing c with a null token $\emptyset$ (e.g., 10-20% of the time) during training. The same network then acts as a conditional model, $\epsilon_\theta(x_t, t, c)$, or as an unconditional one, $\epsilon_\theta(x_t, t, \emptyset)$, simply depending on what is passed as c.

At sampling time, the two predictions are combined linearly:

$$\hat{\epsilon}_\theta(x_t, t, c) = (1 - \gamma)\, \epsilon_\theta(x_t, t, \emptyset) + \gamma\, \epsilon_\theta(x_t, t, c) \tag{14.5}$$

which is the form used in practice, since it only requires two forward passes of the same network (one conditional, one unconditional) at each denoising step.

Text-to-Image Conditioning: Latent Diffusion Running a diffusion model directly on high-resolution pixels is extremely expensive, since both training and sampling require many forward passes through the denoising network. **Latent Diffusion Models** (LDMs) address this by moving the whole diffusion process into a much smaller latent space:

1. First, a variational autoencoder is trained, independently of the diffusion model, to compress images into a lower-dimensional latent representation $z = E(x)$, with a decoder D reconstructing images from it, $\hat{x} = D(z)$. This is a purely perceptual compression step: it discards imperceptible, high-frequency pixel detail while preserving semantic content.

2. The diffusion model is then trained entirely on z rather than x, which drastically reduces the computational and memory cost of both training and sampling.

3. Conditioning information (e.g., text prompts, encoded by a pretrained text encoder such as CLIP) is injected into the denoising network through **cross-attention** layers interleaved throughout the U-Net: the spatial latent features act as queries, while the text embeddings act as keys and values. This lets the model attend to the relevant words when denoising each region, rather than conditioning on a single, fixed global vector.

4. Finally, once a clean latent $\hat{z}_0$ has been generated, the decoder D maps it back to pixel space to obtain the final image.

This architecture underlies most modern text-to-image systems (e.g., Stable Diffusion).

15 Large Multi-Modal Models

Large Multi-Modal Models (LMMs) are a class of models that can process and generate data across multiple modalities, such as text, images, audio, and video. In particular, we will focus on **Large Vision-Language Models** (LVLMs), which are models that can understand and generate both visual and textual information, usually by combining a **vision encoder** (converting images into embeddings) and a **Large Language Model** (LLM), to address various tasks.

We will first discuss different approaches to designing and training **vision encoder** architectures for LVLMs. Finally, we will discuss the architectures that allow for the integration of vision and language modalities, and how to train these models to perform various tasks.

15.1 Encoders

Over the years, several architectures have been proposed for vision encoders. We can group them into four main categories:

- **CNNs**: based on supervised learning, these models are limited by the fact that the labels need to be pre-defined, and they are not able to learn from unlabeled data.
- **Image-Only (Non-)Contrastive Learning**: based on self-supervised learning, these models learn to generate embeddings for images without the need for labels. This is achieved by training the model to output similar embeddings for different augmentations of the same image.
- **Masked Image Modeling (MIM)**: given an image, the model is trained to predict the missing parts of the image, which allows it to learn a representation of the image that captures its structure and semantics.
- **Contrastive Learning Approaches**: architectures like **CLIP**, that learn to align image and text embeddings in a shared space, allowing for cross-modal retrieval and understanding.

We will discuss each of these approaches in more detail in the following sections.

15.2 Image-Only (Non-)Contrastive Learning

Various self-supervised learning approaches have been proposed to learn image representations without the need for labels.

15.2.1 SimCLR

SimCLR is a contrastive learning approach that learns to generate embeddings for images by training the model to output similar embeddings for different augmentations of the same image, while ensuring that embeddings for different images are dissimilar. Given an image, the model first generates two different augmentations of the image. A base encoder is then used to generate embeddings for both augmentations. These embeddings are then passed through a projection head, which is trained to maximize the similarity between the embeddings of the two augmentations, while minimizing the similarity between embeddings of different images.

For downstream tasks, the projection head is discarded, and the base encoder is used to generate embeddings for images.

The used loss function is the **InfoNCE loss**, which is a contrastive loss that encourages the model to output similar embeddings for positive pairs (different augmentations of the same image) and dissimilar embeddings for negative pairs (different images). It is defined as follows:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{N} \exp(\text{sim}(z_i, z_k)/\tau)} \tag{15.1}$$

where we are basically maximizing the similarity between the embeddings of the two augmentations of the same image (z_i and z_j), while minimizing the similarity between embeddings of different images (z_i and z_k), with τ being a temperature parameter that controls the sharpness of the distribution.

Such approaches have a main limitation: a large batch size is required to have a sufficient number of negative pairs, which can be computationally expensive and memory-intensive.

15.2.2 DINO

Recent approaches such as **DINO** (DIstillation with NO labels) have been proposed to overcome some limitations of contrastive learning methods such as SimCLR. DINO is a self-supervised learning framework based on ViTs that learns meaningful image representations without requiring labeled data or negative sample pairs.

The architecture of DINO consists of two networks: a **student network** and a **teacher network**. The student network is trained to generate embeddings from input images, while the teacher network produces target embeddings that the student aims to match. Rather than being updated through backpropagation, the teacher's parameters are updated as an exponential moving average (EMA) of the student network's parameters, providing stable and slowly evolving targets throughout training.

During training, multiple augmented views of the same image are generated. The student network processes both global and local image crops, whereas the teacher network processes only the global crops. Both networks output feature vectors, which are projected through a small MLP head to produce logits. These logits are converted into probability distributions using a softmax function.

The student is trained to match the probability distribution produced by the teacher for different views of the same image. This is achieved by minimizing the cross-entropy loss between the teacher's output distribution P_t and the student's output distribution P_s:

$$\mathcal{L} = -\sum_i P_t(i) \log P_s(i),$$

where i indexes the output dimensions. Since the teacher's output serves as the target distribution, gradients are computed only with respect to the student network.

The resulting embeddings have been shown to transfer effectively to a wide range of downstream computer vision tasks, including image classification, object detection, and semantic segmentation.

15.3 Masked Image Modelling

This family of approaches is based on the idea of predicting missing parts of an image, which allows the model to learn a representation of the image that captures its structure and semantics.

15.3.1 BEiT

BERT Pre-Training of Image Transformers (BEiT) adapts the masked language modeling objective of BERT to computer vision. First, an image is divided into non-overlapping patches. A pretrained image tokenizer converts each patch into a discrete visual token, which serves as the prediction target during pre-training. Meanwhile, the image patches are embedded as in a standard Vision Transformer, and a subset of the patch embeddings is replaced with a learned mask embedding.

The Vision Transformer processes the resulting sequence together with positional embeddings. During training, the model predicts the discrete visual tokens corresponding to the masked patches by minimizing the cross-entropy loss between the predicted token distributions and the tokenizer-generated targets.

15.3.2 MAE

Masked Autoencoders (MAE) is a self-supervised learning approach that learns to reconstruct missing parts of an image. Instead of predicting discrete visual tokens, MAE directly reconstructs the pixel values of the masked patches.

Usually, a large portion of the input image patches (e.g., 75%) is randomly masked, and the remaining visible patches are processed by an encoder. A decoder then takes the encoded visible patches and the mask tokens (which are included after the encoder) to reconstruct the original image.

15.4 CLIP

We have already discussed the **CLIP** architecture in the previous chapter, which is a contrastive learning approach that learns to align image and text embeddings in a shared space. Although it shows impressive performance on various tasks, there are still some remarkable limitations that need to be addressed:

- **No direct interaction between modalities:** CLIP simply learns to align the embeddings of images and text. It doesn't allow to go from one modality to another, or to generate new data in a different modality.
- **Limited task performance:** CLIP can only perform tasks that can be solved by comparing embeddings, such as image classification or retrieval.
- **No generative capabilities:** CLIP lacks the ability to generate new data, making them unsuitable for tasks that require data generation, such as captioning or image synthesis.

In this section, we will discuss some approaches that address these limitations and allow for more direct interaction between modalities, as well as generative capabilities.

15.4.1 Data Scaling

Various studies have shown that increasing the amount of training data can significantly improve the performance of LVLMs. For example, scaling from 400 million to 2 billion image-text pairs has been shown to improve the performance of CLIP on various tasks.

Scale is not the only factor that affects the performance of LVLMs. Also, the quality of the data is important. Smaller, more curated datasets were shown to outperform larger, noisier datasets.

15.4.2 Model Design

Not only the amount and quality of data, but also the design of the model architecture can significantly affect the performance of CLIP. In particular, new architectures have been proposed for both the vision encoder and the language model:

- **Image Side:** the vision encoder can be improved via the use **masking**. It has been shown that masking a large portion of the input image and only encoding the remaining visible patches can improve the performance of the vision encoder, reducing also the traing time and computational requirements.
- **Text Side:** along with the original text input, additional information such as the definition of the class or a description of the image can be provided to the language model, which can improve the performance of the model on various tasks. This was particularly effective with fine-grained datasets, which typically contain many rare classes.

Some efforts have also been made to improve the interaction between modalities, while maintaining the shared embedding space.

For example, the **ImageBind** architecture allows for the integration of multiple modalities, such as images, text, audio, and video, into a single model. To achieve this, a pretrained version of CLIP is used and kept frozen. A new encoder is used for each new modality, and a contrastive loss is used to align the embeddings of the new modalities with the embeddings of the original CLIP model.

15.4.3 Interpretability

One of the main limitations of CLIP is the limited interpretability of its learned embeddings. To address this issue, **STAIR** introduces a sparse and semantically meaningful representation for both image and text embeddings.

STAIR constructs a concept-based embedding space in which each dimension corresponds to a human-interpretable semantic concept. Instead of representing an image or a text as a dense feature vector, each embedding is expressed as a non-negative set of concept activations, where the value associated with each concept reflects its relevance to the input.

To obtain this representation, STAIR employs a **token projection head** that maps the original CLIP embeddings into the sparse concept space. The projection head produces a non-negative weight for every concept, yielding an interpretable embedding whose active dimensions indicate the semantic concepts most strongly associated with the image or text.

15.4.4 Objective Function

FILIP (Fine-grained Interactive Language-Image Pre-training) introduces a fine-grained supervision mechanism. Instead of representing an image and a text by a single global embedding, FILIP preserves the embeddings of individual image patches and text tokens, enabling interactions at a finer semantic level.

After the image and text encoders produce patch-level and token-level embeddings, FILIP computes a pairwise similarity matrix between every image patch and every text token. Rather than directly comparing global representations as in CLIP, the similarity between an image and a text is obtained through a *cross-modal late interaction* mechanism. Specifically, for each image patch, the maximum similarity over all text tokens is computed, while for each text token, the maximum similarity over all image patches is also calculated.

The resulting similarities are then aggregated (using max pooling followed by averaging) to obtain the final image-text similarity score.

This objective encourages each image region to align with its most relevant textual token and vice versa, leading to more accurate word-patch correspondences.

CoCa (Contrastive Captioner) takes a different route: instead of only refining the contrastive objective, it adds a **generative branch** on top of it. This allows a single model to be jointly pre-trained with a contrastive loss and a captioning loss, unifying the single-encoder, dual-encoder, and encoder-decoder paradigms into one architecture.

CoCa is composed of three modules:

- **Image Encoder:** as in CLIP, it produces a global image embedding as well as patch-level embeddings.
- **Unimodal Text Decoder:** processes the text tokens using only self-attention (cross-attention to the image is omitted), producing a global text embedding used for the contrastive objective.
- **Multimodal Text Decoder:** takes the output of the unimodal decoder and applies cross-attention over the image encoder outputs, learning multimodal representations that are used to generate the caption.

The two decoders share the same weights except for the attention mechanism, and are trained jointly with a weighted combination of the two losses:

$$\mathcal{L} = \lambda_{\text{Con}}\, \mathcal{L}_{\text{Con}} + \lambda_{\text{Cap}}\, \mathcal{L}_{\text{Cap}}$$

where $\mathcal{L}_{\text{Con}}$ is the usual CLIP-style contrastive loss between the image embedding and the unimodal text embedding, while $\mathcal{L}_{\text{Cap}}$ is the captioning loss, which trains the multimodal decoder to autoregressively predict the caption tokens y conditioned on the image x:

$$\mathcal{L}_{\text{Cap}} = -\sum_{t=1}^{T} \log p_\theta(y_t \mid y_{<t}, x)$$

Unlike the dual-encoder approach, which only encodes the text as a whole, the captioning objective forces the model to reason at a finer granularity, since it has to predict the caption word by word. As a result, CoCa is not limited to visual recognition and retrieval tasks (as CLIP is), but can also be directly used for generative tasks such as image captioning and visual question answering.

SimVLM is a precursor of CoCa, proposed by the same research group. It adopts a standard **transformer encoder-decoder** architecture, trained with a single generative objective: given an image (and an optional text prefix), the model is trained to generate the remaining text autoregressively, conditioning on the preceding tokens and the visual input.

Although SimVLM was an important step towards generative vision-language models, using a captioning loss alone was not enough to reach the same alignment quality as CLIP's contrastive objective, which is exactly why CoCa was later introduced to combine both objectives.

We can now summarize the three main design paradigms for Vision-Language Models that we have discussed in this section:

- **Dual Encoder (CLIP):** cannot generate text, but achieves excellent performance on tasks that can be solved by comparing embeddings, such as classification and retrieval.
- **Encoder-Decoder (SimVLM):** can generate text, but does not match CLIP's performance on alignment-based tasks.
- **Fusion Decoder (CoCa):** combines the strengths of both approaches, enabling text generation while retaining strong retrieval and classification performance.

A fourth paradigm, adopted by **Flamingo**, blends the dual-encoder approach with an additional, separately pre-trained LLM, in order to further improve generative and few-shot capabilities. We will discuss this approach in the next section.

15.5 LLMs as a Universal Interface

Multimodal systems offer a more flexible way of interacting with a model than plain text, since a user can provide input by typing, speaking, or simply pointing a camera at something. To build a general-purpose assistant around this idea, we need LLMs that are able to ingest a **multimodal prompt**, containing images and/or videos arbitrarily interleaved with text.

The idea is to equip LLMs with the ability to "see" the world, by training them on vision-conditioned language generation tasks. This allows LLMs to act as a general interface for other modalities, while still leveraging the knowledge acquired during their language-only pre-training.

This capability is usually obtained through two complementary strategies:

- **Multimodal In-Context Learning (M-ICL):** a few demonstrations (e.g., image-text pairs) are provided directly as part of the prompt, allowing the model to perform few-shot learning without updating any of its parameters.
- **Multimodal Instruction-Tuning (M-IT):** the model is fine-tuned on a collection of tasks described through natural language instructions, so that it can generalize to unseen tasks simply by following the instructions given in the prompt.

15.5.1 Frozen LM Prefix

Before discussing Flamingo, it is useful to introduce the concept of **Frozen LM Prefix**, described in "Multimodal Few-Shot Learning with Frozen Language Models".

The model is composed of three components: a vision encoder (NF-ResNet-50), a language embedder, and a language model used for text generation. The last two components are kept **frozen**, meaning that their weights are never updated. Only the vision encoder is fine-tuned, and its output embeddings are directly used as **soft prompts** for the language model: they are concatenated with the text embeddings and fed to the frozen LLM decoder, which generates the textual output autoregressively.

In other words, the visual embeddings act as a hint or guidance for the frozen language model on what it should focus on, without ever changing its weights. This is enough to unlock zero-shot visual question answering, as well as few-shot capabilities such as outside-knowledge VQA and few-shot image classification, purely by conditioning on the prompt.

15.5.2 Flamingo

Flamingo, developed by Google DeepMind, builds on the Frozen LM Prefix idea and represents a significant advance over it, achieving strong performance on zero-shot task transfer and in-context few-shot learning. It bridges powerful, separately pre-trained vision-only and language-only models through a small set of novel architectural components, trained from scratch, while keeping both pre-trained models frozen.

Vision Encoder Flamingo first trains a CLIP-like dual-encoder model from scratch using contrastive learning: a **Normalizer-Free ResNet** (NFNet) is used as the image encoder, while BERT (rather than GPT-2) is used as the text encoder. Text and image embeddings are mean-pooled before being projected into the joint embedding space. Once this pre-training is complete, only the vision encoder is kept (and frozen) for the rest of the pipeline, while the text encoder is discarded, since a separate LLM will be used for language understanding and generation.

Perceiver Resampler Since the frozen vision encoder can produce a variable number of spatial and temporal features, depending on the resolution of the input images or the number of frames of the input videos, we need a way to convert them into a fixed number of visual tokens.

The **Perceiver Resampler** addresses this problem: a small, fixed-size set of learned *latent queries* attends, via cross-attention, to the (variable-length) features produced by the vision encoder, producing a fixed number of output tokens regardless of the size of the visual input. These tokens are the ones later used to condition the language model.

Gated XATTN-DENSE Layers To let the language model attend to the resampled visual tokens, new **gated cross-attention dense** (XATTN-DENSE) layers are inserted between the existing, frozen self-attention layers of the LLM. The output of a gated XATTN-DENSE layer serves as the key, value, and query for the following frozen language model layer.

Each of these new layers is gated by a tanh function, initialized to zero. This means that, at the beginning of training, the model behaves exactly like the original frozen LLM, and the influence of the visual tokens is only gradually introduced as training progresses, without disrupting the language capabilities acquired during pre-training.

Training Flamingo is trained to handle sequences of arbitrarily interleaved visual and textual data. This flexibility allows it to be trained directly on large-scale, heterogeneous multimodal web corpora, mixing web-scraped interleaved image-text data with existing image-text and video-text datasets. This heterogeneity is key to endow Flamingo with strong in-context few-shot learning capabilities, since such interleaved data closely resembles the structure of a multimodal few-shot prompt at inference time.

At both training and inference time, text is predicted autoregressively by the language model (Chinchilla), conditioned on the interleaved visual tokens through the gated XATTN-DENSE layers.

Summary Flamingo shows how strong, independently pre-trained single-modality models can be unified through lightweight, learnable connector modules: a Perceiver-based architecture that produces a fixed number of visual tokens to support both images and videos, and gated cross-attention layers interleaved with the frozen language-only self-attention layers. Despite these strengths, Flamingo is still prone to typical failure modes of generative models, such as hallucinating details that are not present in the visual input.